

**KWAME NKRUMAH UNIVERSITY OF SCIENCE AND
TECHNOLOGY**

COLLEGE OF ENGINEERING



**DESIGN AND VALIDATION OF AN AUTONOMOUS
NAVIGATION SYSTEM FOR A MODULAR ROBOT ON
UNSTRUCTURED FARM TERRAIN.**

ANSU-AMPONSAH EMMANUEL KWABENA 1844822

ASIBU FRANK 1847722

WONTUMI ADJINDA 1862322

Supervisor: Dr. Elvis Twumasi

Department: Electrical / Electronics Department

Date: September 2026

ABSTRACT

Agricultural artificial-intelligence tools deployed in Ghana are trained largely on foreign data, and the local crop imagery needed to correct this does not exist in sufficient volume. Collecting it autonomously requires a robot able to cross the uneven, intercropped ground of a cocoa farm, which remains an unsolved prerequisite. Terrain-aware navigation frameworks predict motion with a kinematic model in which the wheels do not slip, whereas a low-cost skid-steer robot has no steering mechanism and turns only by scrubbing its wheels sideways across the ground. A friction-based model that predicts this slip exists, but serves only for offline prediction, and the frameworks that would use it assume a 3D LiDAR priced beyond the research groups such a robot is meant to serve.

This report presents a four-wheel, two-motor skid-steer robot whose navigation reasons about slip. A friction-based kinematic model is identified from the robot's own driving data on deformable, leaf-littered terrain and embedded as the motion prediction of a Hybrid A*-guided model predictive path integral controller, and a low-cost stereo depth camera replaces the reference 3D LiDAR. The work is carried out and evaluated entirely in simulation, on a model of a Ghanaian cocoa farm.

The model predicts yaw rate on unseen data within 0.10 rad/s, against 0.84 rad/s for the no-slip model it replaces, an improvement of 87.9 %, while remaining less accurate on forward velocity. A bounded solver evaluates it in 2.44 μ s, giving a control cycle of 16 to 19 ms within a 100 ms period and placing a previously offline model inside a real-time controller. In closed loop the benefit depends on tuning: with the navigation framework's original seed settings the friction-based prediction reaches four of six goals against two, and with the settings adopted here both reach all six, so reasoning about slip widens the range of tunings over which navigation succeeds. The stereo camera maps 40.8 percentage points fewer tree trunks than the LiDAR, yet reaches every goal with time to goal and path efficiency statistically equivalent to it and no loss of obstacle clearance. A tree-image survey mission visits five trees and captures twenty images without intervention, holding the camera's range to the trunk to a spread of 10 mm.

Table of Contents

ABSTRACT	1
CHAPTER 1 - INTRODUCTION	7
1.1 Background.....	7
1.2 Problem Statement.....	8
1.3 Aim	9
1.4 Objectives	9
1.5 Scope and Limitations	9
1.6 Significance of the Study.....	10
1.7 Report Outline	11
CHAPTER 2 - LITERATURE REVIEW	13
2.1 Introduction	13
2.2 Agricultural Robots for Field Data Collection	13
2.3 Terrain-Aware Navigation on Rough, Unstructured Ground.....	15
2.4 Skid-Steer Motion Modelling and Friction-Aware Control	16
2.5 Low-Cost and Accessible Sensing for Field Robotics	18
2.6 Agricultural Robotics in Ghana and West Africa.....	19
2.7 Summary.....	20
CHAPTER 3 - DESIGN THEORY AND CONSIDERATIONS	21
3.1 Introduction	21
3.2 Platform and Environment	21
3.2.1 Operating Environment and Design Requirements.....	22
3.2.2 Drivetrain Topology and Actuation.....	23
3.2.3 Simulation as the Test Instrument	25
3.2.4 Representing the Robot in Simulation.....	26
3.2.5 Representing a Cocoa Farm.....	27
3.3 Skid-Steer Motion Modelling.....	29

3.3.1 The No-Slip Kinematic Model	29
3.3.2 The Friction-Based Kinematic Model	31
3.3.3 Parameter Identification	35
3.3.4 Measures of Prediction Accuracy	36
3.4 Terrain-Aware Autonomous Navigation	37
3.4.1 Choice of Navigation Framework	37
3.4.2 Traversability Estimation from Elevation Maps	39
3.4.3 Hybrid A* Search	40
3.4.4 Model Predictive Path Integral Control.....	41
3.4.5 Guided MPPI: Seeding the Sampler from the Planned Path	43
3.4.6 Reaching Goals beyond the Local Map, and the Localisation It Assumes	45
3.4.7 Range Sensing for Terrain Mapping: 3D LiDAR versus Stereo Depth	45
3.4.8 Measuring Navigation Performance and Testing for Equivalence.....	47
3.5 Survey Workflow Design	49
3.5.1 Requirements of a Tree-Image Survey Mission	49
3.5.2 Station Geometry and Camera Coverage	51
3.6 Summary.....	54
CHAPTER 4 - METHODOLOGY	56
4.1 Introduction	56
4.2 Platform and Environment	56
4.2.1 Robot Design: from Requirements to the CAD Model	57
4.2.2 From CAD to Simulation Model	58
4.2.3 Drivetrain and Actuation	60
4.2.4 Building the Cocoa-Farm Worlds	62
4.2.5 Evaluation Worlds and Simulation Settings.....	64
4.3 Friction Model Development	65
4.3.1 Data Collection Protocol	65

4.3.2 Dataset Processing Pipeline.....	67
4.3.3 Parameter Fitting	68
4.3.4 Open-Loop Validation Protocol	69
4.3.5 Adapting the Model for Real-Time Use	69
4.4 Autonomy Stack	70
4.4.1 Staged Implementation	70
4.4.2 Traversability Estimation as Implemented	71
4.4.3 Hybrid A* Planner as Implemented	72
4.4.4 MPPI Controller with the Friction Rollout.....	74
4.4.5 Guided Seeding for a Scrub-Limited Robot	75
4.4.6 Sensor Substitution: the Stereo Depth Configuration	76
4.4.7 Closed-Loop Evaluation Protocol	77
4.5 Survey Workflow	79
4.5.1 Mission Architecture	79
4.5.2 Station Selection and the Camera-Referenced Standoff.....	80
4.5.3 Capture Routine and the Triggered Still Camera	81
4.5.4 Operator Interface.....	83
4.5.5 Output Data and Metadata	83
4.6 Summary.....	84
References (placeholders, to be formatted)	85
CHAPTER 5 - RESULTS AND ANALYSIS	86
5.1 Introduction	86
5.2 Platform and Environment	86
5.2.1 The Corrected Simulation Model	86
5.2.2 Motion Behaviour of the Corrected Model	87
5.2.3 The Cocoa-Farm Worlds	90
5.2.4 Limitations of the Platform and Environment.....	90

5.3 Friction Model	91
5.3.1 Datasets of the Final Model.....	91
5.3.2 Fitted Parameters	92
5.3.3 Open-Loop Prediction Accuracy	93
5.3.4 Computational Cost of the Online Solver	94
5.3.5 Limitations.....	94
5.4 Autonomy Stack	95
5.4.1 Staged Verification	95
5.4.2 Closed-Loop Comparison of the Friction and No-Slip Rollouts.....	96
5.4.3 Control-Loop Timing	98
5.4.4 Sensor Substitution: the Perception Deficit.....	99
5.4.5 Sensor Substitution: Task Completion and Equivalence.....	100
5.4.6 Sensor Substitution: Clearance	101
5.4.7 Limitations.....	102
5.5 Survey Workflow	102
5.5.1 Station Selection Across the Whole Farm.....	103
5.5.2 Turning, Drive Arbitration and the Skip Path	104
5.5.3 The Five-Tree Survey Mission.....	105
5.5.4 Camera Range and Framing	106
5.5.5 Captured Data and Metadata	107
5.5.6 Limitations.....	108
5.6 Summary.....	109
CHAPTER 6 - CONCLUSION	111
6.1 Introduction	111
6.2 Summary of the Study	111
6.3 Conclusions	112
6.4 Constraints of the Study	113

6.5 Recommendations	114
CHAPTER 7 - REFERENCES	115

CHAPTER 1 - INTRODUCTION

1.1 Background

Agriculture supports Ghana's economy and food security. The sector contributes about one-fifth of GDP, employs about one-third of the working population [1], and is dominated by smallholder farmers working small, intercropped plots whose livelihoods depend on rainfall, soil conditions and pest pressure [2]. Two pressures shape the problem. Young people are leaving agriculture in Ghana and across Sub-Saharan Africa at a rate the sector cannot sustain [3], [4]. Climate change is also making rainfall less predictable and adding to pest pressure across the region. Sub-Saharan Africa must more than double agricultural output by 2030 to meet projected demand [5]. Labour is shrinking while the growing conditions are becoming harder. The sector has little room left for slow fixes.

Artificial intelligence is often proposed as a tool for food insecurity, environmental degradation and economic inequality in Sub-Saharan Africa [6], [7]. It may help, but only if the models can see the farms they are meant to serve. Most agricultural AI systems were built around large monoculture farms in the developed world, not the small, intercropped and highly local conditions of Ghanaian smallholder farming [8], [9]. Satellite and remote-sensing tools miss much of the dense, heterogeneous structure of these plots [8]. Smallholder farms and women-led operations are also poorly represented in the training sets behind many models [7], [8]. The result is a practical data problem with a political edge: the farms most in need of local tools are often least visible in the data used to build them. Studies of AI's political economy describe Global South data producers as structurally excluded from, or invisibly extracted into, the pipelines that train systems later sold back to them [10], [11]. A Kenya-focused commentary calls this a data drought: tools trained on data from Europe, India or the United States can misread local crop-disease conditions [12]. In farming, a wrong diagnosis can cost a harvest [12]. Ghana has fewer than five publicly available localised agricultural datasets, and foreign models applied to Ghanaian conditions show accuracy losses of 18 to 32 % [2]. Better algorithms alone will not solve that. The missing input is local, labelled crop imagery collected at field level, in enough volume and consistency to train useful models.

1.2 Problem Statement

A direct way to close this gap is to collect the data locally with an autonomous mobile robot (AMR). The idea is not speculative. The CCMT project collected more than 100,000 annotated crop-disease images from Ghanaian farms without relying on foreign datasets [13]. In another data-scarce setting, a researcher collected five million local training images in two weeks with cameras carried by volunteers [9]. Those examples show that local data can be gathered. They do not solve repeatability. A volunteer or field worker does not follow the same route, hold the same camera pose, or revisit the same plant in a controlled way on every run. A robot can.

Before an AMR can collect data, however, it must be able to reach it, and on the intercropped, uneven terrain of a Ghanaian cocoa farm that is not a solved problem. GPS with real-time kinematic correction (GPS-RTK), the standard navigation approach on nearly half of existing agricultural platforms [14], is unreliable here: dense canopy and closely spaced trees generate the multi-path interference and signal blockage it cannot tolerate [14], [15]. Frameworks that instead build a traversability map from onboard range sensing, and plan and control motion across it, have been demonstrated on rough, unstructured ground [16], but their motion prediction assumes a robot that does not slip. That assumption fails for the low-cost, skid-steer drivetrains available to the research groups working in this context: a skid-steer robot has no steering mechanism and turns only by scrubbing its wheels sideways across the ground, which is exactly the motion a no-slip model cannot predict. A friction-based kinematic model that does predict this slip has been derived and validated on a real platform [17], but its authors leave its integration into a real-time controller as future work. The navigation framework's own reference implementation is built around a 3D light detection and ranging (LiDAR) sensor, a sensing cost that puts the approach out of reach of the research teams it would need to serve [18].

Reconciling what these navigation frameworks assume with how a skid-steer robot actually moves, at a sensing cost researchers in this context can afford, is therefore the unsolved prerequisite to any of the data collection described above. It is the problem this project addresses.

1.3 Aim

The aim of this project is to enable a low-cost, skid-steer autonomous mobile robot to navigate the uneven, intercropped terrain of a Ghanaian cocoa farm reliably enough to collect localised crop data.

1.4 Objectives

1. To design a low-cost four-wheel, two-motor skid-steer robot for the conditions of a Ghanaian cocoa farm, and to represent that robot and the farm as a simulation model faithful enough to test against.
2. To identify a friction-based kinematic model of that robot from its own driving data, and to establish how accurately it predicts the robot's motion against the no-slip model it would replace.
3. To embed that model as the motion prediction of a terrain-aware Hybrid A*-guided Model Predictive Path Integral (MPPI) controller, at a computational cost that fits inside the control period.
4. To evaluate, in closed-loop navigation trials, what reasoning about slip changes about the navigation stack's behaviour.
5. To determine whether a low-cost stereo depth camera can replace the 3D LiDAR that this class of navigation framework assumes, and at what cost.
6. To demonstrate the complete system on the applied task it exists to serve: an autonomous tree-image survey mission.

1.5 Scope and Limitations

The scope of this project is bounded in five respects, each of which shapes how its results should be read. The first and most consequential is that the work is carried out and evaluated entirely in simulation. The robot was designed in computer-aided design and realised as a physics-simulation model; no physical prototype was built or tested. Simulation is used as the test instrument because it allows the terrain, the sensors and the robot's own parameters to be varied and repeated exactly, which a field deployment on a working farm cannot offer within the time and resources of this project.

The platform is a single drivetrain configuration, a four-wheel, two-motor skid-steer robot driven by torque, and the friction model is identified for one terrain type, a deformable,

leaf-littered surface, at one torque limit and one wheel geometry. Whether the identified parameters transfer to other terrains is not established here.

Two range sensors are compared: a 3D LiDAR, which is the sensor the reference navigation framework assumes, and one stereo depth camera. No other sensing modality is evaluated.

The robot's position and orientation are taken from the simulator throughout, so the navigation results are not affected by localisation error, and nothing in this report tests the stack under the drift a real robot would accumulate. Localisation on a working farm, where satellite positioning is unreliable for the reasons given in Section 1.2, falls outside this project and is returned to in Chapter 6.

Finally, the survey mission demonstrates autonomous data collection rather than crop diagnosis. The images it captures are sufficient to show that a complete tree record can be gathered without intervention; they are not intended to support inspection of individual pods, and no disease model is trained or evaluated in this report.

1.6 Significance of the Study

This report presents an autonomous mobile robot built around a motion model that predicts how it actually moves, rather than how an idealised robot would. A friction-based kinematic model, identified from the robot's own driving data on deformable, leaf-littered terrain, is embedded as the motion prediction of a Hybrid A*-guided MPPI controller, so that the planner and controller reason about wheel slip instead of assuming it away. The resulting navigation stack crosses the irregular, intercropped terrain of a cocoa farm using a traversability map built from onboard range sensing. Because a 3D LiDAR is the sensor this class of framework is normally built around, and its cost puts it out of reach of the research groups this robot is meant to serve, the range sensor is treated as a design variable rather than a fixed choice.

The main contribution is methodological: the report puts a slip-aware motion predictor inside a sampling-based controller for a robot that turns by sliding its wheels. The implemented robot, navigation stack and survey workflow form the second contribution. Four specific contributions follow, listed in the order in which they are tested.

1. It identifies a friction-based kinematic model for a four-wheel, two-motor skid-steer robot on deformable agricultural terrain, from commanded-velocity and pose data alone.
2. It embeds that model as the rollout dynamics of a real-time MPPI controller, the integration its authors left as future work, at a cost of microseconds per prediction, so that a model previously used for offline analysis runs inside the control loop.
3. It shows what that substitution buys in closed loop: a controller that reasons about slip succeeds across a wider range of tuning than one that does not, recovering goals the no-slip controller loses and colliding less often on the way, while the shipped configuration reaches every goal.
4. It shows that a substantially cheaper stereo depth camera can stand in for the reference 3D LiDAR with no measurable cost to task completion, although it maps materially less of the farm.

A tree-image survey mission built on this stack demonstrates the complete system end-to-end, as the applied task the navigation exists to serve. The significance of the whole is that it puts autonomous, repeatable, local data collection within reach of the research groups whose absence from the training data created the gap described in Section 1.1.

1.7 Report Outline

The remaining chapters follow the order in which the parts of the system depend on one another. Chapter 2 reviews the literature behind the problem statement above: the robots already built for agricultural field data collection, the navigation frameworks available for unstructured ground, the modelling of skid-steer motion, and what is documented for West Africa, ending with the gaps this project addresses. Chapter 3 develops the design theory those gaps imply, why a skid-steer robot cannot be described by no-slip kinematics, what a friction-based model puts in their place, how a traversability map is built and planned across, and what a tree-image survey requires of the robot that carries it out. Chapter 4 describes how each of these was built and how each was tested: the robot and the farm worlds, the collection of driving data and the identification of the friction model, the navigation stack and the sensor configurations, and the protocols for the closed-loop and survey trials. Chapter 5 reports what those tests produced and analyses what they show, taking the platform, the friction model, the autonomy stack and the survey workflow in turn, with the limitations of each stated beside its results. Chapter 6 draws the conclusions

these results support, returns to the aim stated in Section 1.3, and sets out the work that would extend the system beyond the boundaries given in Section 1.5.

CHAPTER 2 - LITERATURE REVIEW

2.1 Introduction

Chapter 1 argued that closing Ghana's agricultural artificial-intelligence data gap requires an autonomous mobile robot (AMR) able to navigate a Ghanaian cocoa farm reliably, and that reliable navigation for a low-cost, skid-steer platform is itself an unsolved prerequisite rather than a solved precondition.

This chapter examines the literature behind that claim and establishes the technical foundation the following chapters build on. The review is organised by topic rather than by source: work is grouped by the problem it addresses, and each group is assessed for what it contributes, what it assumes, and what it leaves unresolved. Sources were drawn from the recent literature on agricultural robotics, terrain-aware navigation, and skid-steer motion modelling, together with regional studies of agricultural technology in West Africa. Comparative reviews and surveys are used where they exist, so that individual systems are discussed only where they bear directly on a design decision taken later in this report.

Five topics follow. Section 2.2 reviews robots built for agricultural field data collection, to establish the design space this class of machine occupies and the trade-off between precision, autonomy and cost that space has settled into. Section 2.3 reviews terrain-aware navigation frameworks for unstructured ground and compares the approaches available for estimating traversability. Section 2.4 reviews skid-steer motion modelling, where the assumption common to those navigation frameworks is examined against how skid-steer robots actually move. Section 2.5 reviews low-cost and accessible field robotics, where the cost of the sensing these frameworks assume becomes the binding constraint. Section 2.6 reviews what is documented for Ghana and West Africa specifically, and Section 2.7 summarises the gaps the review identifies.

2.2 Agricultural Robots for Field Data Collection

Robotic phenotyping, using a mobile platform to gather crop-trait data instead of measuring it by hand, has been reviewed at two levels. Atefi et al. classify phenotyping robots by operating environment, from indoor and controlled systems to outdoor field systems, and by interaction type, from non-contact sensing to contact-based probing [19]. They report that wheeled platforms account for 88 % of mobile field phenotyping platforms, that Robot Operating System is the dominant middleware at 65 %, and that

navigation without satellite positioning under tall canopy remains unresolved across the systems surveyed. A more recent review of eco-phenotyping groups platforms by monitoring scale, chamber, greenhouse, plot and field, and reports a similar pattern: proximal ground vehicles produce the highest-resolution data but the lowest test efficiency, while wind, variable lighting and uneven terrain reduce the precision achieved indoors [20].

Neither review treats affordability as a category of analysis. Both classify systems by environment, scale and sensing modality, and both describe the field's benchmark systems as capital-intensive research instruments. The consequence for a reader looking for an affordable platform is not that such platforms have been assessed and found wanting, but that the surveys' own organising axes leave the question undescribed.

Two named benchmark systems show what capital intensity buys and what it costs. The PhenoRob UGV of Esser et al. mounts a twenty-camera dome and two industrial laser-line scanners on a Thorvald II base, achieving sub-millimetre three-dimensional reconstruction of individual plant organs in the field, which is the highest phenotyping precision reported among the sources reviewed here [31]. Two qualifications bound what that result demonstrates. The platform is teleoperated rather than autonomously navigated, so the precision reported is evidence about sensing and reconstruction, not about a robot that can position itself; and a single plant scan takes roughly thirty minutes to reconstruct offline on a high-end graphics processor, an RTX 3090, which limits the number of plants a study can cover for reasons independent of the platform's cost. The sensor suite of twenty machine-vision cameras, two industrial laser scanners and dual-antenna real-time kinematic satellite positioning (RTK-GNSS) places the platform beyond the reach of a resource-constrained research group.

The Robotanist of Mueller-Sim et al. trades precision for autonomy: a 140 kg four-wheel skid-steer platform that navigates row crops using RTK-GPS waypoints with Pure Pursuit trajectory tracking, and probes plant stalks with a custom manipulator, demonstrating 44.7 km of autonomous operation with a 96 % stalk-detection rate [21]. Its reported limitation is structural rather than incidental: satellite waypoint navigation fails once the crop grows tall enough to obscure the signal. The failure is therefore seasonal, and it arrives late in the season, when the crop is most developed and the measurements are most worth taking. A navigation method whose reliability declines as the growing season advances is poorly matched to a data-collection task defined by that same season. The platform's cost, three

networked compute units, multiple LiDAR sensors and RTK-GPS, again places it outside smallholder-research reach.

Taken together, these sources describe a field in which the systems capable of very high data quality and those capable of genuine field autonomy are both capital-intensive, and in which every named benchmark assumes structured row-crop geometry, reliable satellite positioning, or both. Intercropped, canopy-dense and uneven terrain of the kind found on a Ghanaian cocoa farm satisfies neither assumption reliably.

2.3 Terrain-Aware Navigation on Rough, Unstructured Ground

Path planning in unstructured environments differs materially from path planning on structured roads. The survey of Wang et al. makes the distinction explicit: where structured planning optimises distance or time over a fixed lane geometry, unstructured planning must first estimate the traversability of the ground itself, its slope, roughness and associated safety, energy or comfort cost, before a route can be chosen at all [22]. How traversability is represented, and what a planner is allowed to assume about the map it is represented on, is where the frameworks below diverge. Table 2.1 compares them on those terms.

Table 2.1: Terrain-aware navigation frameworks compared

Framework	Traversability representation	Prior map assumed	Motion model	Reported domain
Hybrid A*-guided Model Predictive Path Integral (MPPI), Yang et al. [16]	Slope and roughness from an elevation map, as a deterministic cost	Yes; an elevation map of the working area	No-slip kinematic	Rough terrain, with range sensing by 3D LiDAR
STEP, Dixit et al. [23]	Probability distribution over traversability, planned against tail risk by conditional value-at-risk	No; built for ground never fully mapped	No-slip kinematic; hazards static and assessed independently of speed	Subterranean and disaster environments
Gaussian-process elevation, Leininger et al. [24]	Elevation model fitted online from LiDAR returns	No; mapless, with no persistent map retained	No-slip kinematic	Rugged, undulating terrain

The framework of Yang et al. estimates traversability from an elevation map built out of range sensing, quantifying surface slope and roughness to identify traversable regions; a Hybrid A* search then computes a path minimising surface irregularity across that map; and that path seeds the mean control input of an MPPI controller, which performs local trajectory optimisation while tracking the terrain-aware guide path [16]. Two properties matter for what follows. Traversability is treated as a deterministic quantity computed

from a known elevation map, which suits terrain that can be surveyed in advance and is stable between visits, such as a farm, and suits it better than terrain that cannot, such as a disaster site. And the framework predicts the robot's own motion with a standard no-slip kinematic model, an assumption its released implementation inherits along with a dependence on 3D LiDAR for range sensing.

That combination is one point in a wider design space rather than the only one. The STEP framework of Dixit et al., developed for the DARPA Subterranean Challenge, treats traversability as a probability distribution rather than a fixed cost, because collapsed mines and rubble are never fully mapped in advance, and plans against the tail risk of that distribution using conditional value-at-risk [23]. The framework buys robustness to unmapped ground at two stated costs: traversability is evaluated identically regardless of the robot's speed, and only static hazards are considered. Leininger et al. take a third approach, fitting a Gaussian-process elevation model online from LiDAR returns without building any persistent map, and report that this mapless method succeeds on rugged, undulating terrain where a plane-fitting baseline, PUTN, fails every trial [24]. Retaining no map is what makes the method robust to unfamiliar ground, but it also means nothing is carried between visits: a robot that surveys the same site repeatedly cannot reuse what it learned the previous time.

Two observations follow. First, the three frameworks bound the design space along two axes, mapped against mapless, and deterministic against risk-aware, and the mapped, deterministic corner is the one suited to an environment that can be surveyed once and revisited. Second, the result of Leininger et al. is a useful negative finding in its own right: it demonstrates that a flat-plane traversability assumption breaks down specifically as ground irregularity increases, which is the condition uneven, root-covered ground presents.

What none of the three does is predict the robot's own motion as anything other than an idealised, non-slipping vehicle. Whether that assumption holds for the class of robot under consideration is the subject of Section 2.4.

2.4 Skid-Steer Motion Modelling and Friction-Aware Control

A skid-steer robot has no independent steering mechanism: it turns only by driving the wheels on one side faster than those on the other, so every turn scrubs the tyres sideways across the ground. The review of skid-steer wheeled mobile robots (SSWMR) by Khan et al. surveys how the field has modelled this behaviour, dividing kinematic approaches into

three families, velocity-vectoring, coordinate-transformation and geometric, and notes that several named instances across those families, including a widely cited planar model for the Pioneer P3-AT, omit the skid effect from the model altogether [25]. Where skid is modelled, the review reports, it is usually captured through empirical relations fitted to one specific platform, and it observes that such relations are not physically interpretable and do not generalise to other robots. This is a methodological problem as much as a gap in coverage: a fitted curve that cannot be reasoned about cannot be transferred to a different robot, a different surface, or a different load, and so has to be re-established empirically for every case.

The same review treats one model differently. Rabiee and Biswas present a friction-based kinematic model for slippage prediction on a three-degree-of-freedom skid-steer robot [25]. Because the model comes from a physical friction relationship rather than a fitted curve, its terms have physical meaning.

Rabiee and Biswas derive that model to predict skid-steer slip from the wheel-ground friction relationship rather than assuming it away or fitting it empirically, and validate it on more than 6 km of driving data collected from a Clearpath Jackal, reporting lower translational and rotational prediction error than the kinematic baselines they compare against [17]. Three facts about the work bear on its use. The validation establishes prediction accuracy on one platform, so transfer across platforms and surfaces is not demonstrated by it. The model is a motion predictor rather than a planner or controller, and says nothing about how a navigation stack should use its predictions. And the authors name integration of the model into the control of skid-steer robots, and the development of a closed-loop control system based on it, as future work rather than as something the paper demonstrates.

The two threads of this chapter converge on one point. The Hybrid A*-guided MPPI framework provides a terrain-aware planner and controller built for rough ground, but predicts motion with a no-slip model that a skid-steer robot cannot satisfy, since such a robot turns only by slipping. The friction-based kinematic model predicts that slip from physically interpretable terms, but has not been placed inside a real-time control loop, by its authors' own account. No source reviewed here closes both gaps at once: a friction-aware motion model serving as the rollout dynamics of a terrain-aware controller, evaluated in closed loop on uneven, obstacle-dense ground.

2.5 Low-Cost and Accessible Sensing for Field Robotics

The cost of the frameworks reviewed above is not incidental to them. The review of sensing, control and networking in agricultural robots by Nkwocha et al. finds that state-of-the-art systems typically assume fifth-generation cellular connectivity, cloud offloading and multi-modal LiDAR sensor arrays, and states that this dependency is a barrier "particularly for small-scale farmers", recommending cost-effective design, local manufacturing and open-source development in response [18]. Its comparison of locomotion types bears directly on drivetrain choice: wheeled platforms are the cheapest and most energy-efficient option but lose traction in mud and on uneven ground. That trade-off frames a design question the review does not itself resolve, whether a wheeled platform's traction limits are better avoided by changing locomotion type, at cost, or accommodated by the way the robot is controlled.

Two low-cost platforms bracket what affordable agricultural robotics has demonstrated and what it still assumes. Table 2.2 places them beside the two benchmark systems of Section 2.2.

Table 2.2: Field data-collection platforms, from research-grade to low-cost

Platform	Navigation	Data collection	Infrastructure and terrain assumed	Cost class
PhenoRob UGV [31]	Teleoperated	20 cameras and 2 laser-line scanners; sub-millimetre reconstruction, about 30 min per plant offline	RTK-GNSS on platform; field rows	Research-grade
Robotanist [21]	Autonomous; RTK-GPS waypoints with Pure Pursuit	Contact probing of stalks; 44.7 km autonomous, 96 % stalk detection	RTK-GPS, degrading under tall canopy; structured row crops	Research-grade
RBR-1 [26]	Autonomous; LiDAR-based simultaneous localisation and mapping (SLAM) on ROS 2	Modular manipulator arm	RTK-GPS infrastructure	About USD 2,200
DARob [27]	Autonomous	Terrestrial crop imaging for training-data collection	Bicycle wheels, degraded in mud; uncovered camera	About USD 850

RBR-1, a six-wheeled rocker-bogie rover, shows that a capable, multi-purpose agricultural unmanned ground vehicle can be built for roughly USD 2,200, an order of magnitude below research-grade platforms, with LiDAR-based SLAM, ROS 2, and a modular manipulator arm [26]. Its reported limitation is that it still depends on RTK-GPS

infrastructure, so its cost reduction is achieved in computing and mechanics while the infrastructural assumption identified in Section 2.2 is left untouched. DARob goes further on cost, at roughly USD 850, by narrowing the task from a general-purpose platform to autonomous capture of terrestrial crop images for training data [27]. Its reported failure modes locate the price of that reduction precisely: bicycle wheels struggle in mud, reducing autonomy and disturbing navigation, and an uncovered camera admits shadow and lighting artefacts into the captured images. The camera issue is also a data-quality problem, since artefacts enter the images the platform exists to collect.

Read together with Section 2.4, these platforms show that affordability and reliability on uneven ground have not been demonstrated in the same system. The cheapest general-purpose platform retains the infrastructure dependency, and the cheapest purpose-built data-collection platform adopts a drivetrain that fails on wet, uneven ground. Two routes to affordability are therefore visible in the literature, reducing computing and mechanical cost, and narrowing the task, while a third is not examined in the sources reviewed here: holding the navigation capability fixed and reducing the cost of the range sensing that terrain-aware navigation depends on, which Section 2.3 identified as a 3D LiDAR in every framework surveyed.

2.6 Agricultural Robotics in Ghana and West Africa

No deployment of a ground-based autonomous mobile robot on a Ghanaian or West African smallholder farm is documented in the sources reviewed for this report. The regional work located is aerial. A systematic review of regenerative agriculture technology across five major West African cocoa-producing countries, Côte d'Ivoire, Ghana, Nigeria, Cameroon and Togo, covers unmanned aerial systems used for spatial monitoring of canopy vigour, water and nutrient stress, and disease risk, feeding diagnostics that human farmers then act on [28]. It does not report ground-based physical data collection or intervention, and flags the upfront cost of aircraft and analytics as a barrier that pushes the region toward cooperative or subscription service models rather than individual ownership. A broader review of field phenotyping across African crops reports the same pattern continent-wide: the platforms in use are unmanned aerial vehicles, motorised gantries, high-clearance tractors and hand-pushed carts, none of them an autonomous ground vehicle navigating uneven, unstructured terrain under its own perception [29]. That review

names high platform cost and limited digital infrastructure as the reasons practical deployment favours low-cost sensors mounted on already-available platforms.

Two limits on this finding should be stated with it. The claim is an absence of documented deployments in the sources reviewed, which is weaker than an absence of deployments: undocumented or locally reported work would not appear in the literature searched, and the search itself is bounded by the criteria given in Section 2.1. What the reviewed sources do support is narrower and still useful, that the regional literature addresses aerial monitoring, that it identifies cost and infrastructure as the binding constraints, and that it does not report ground-based autonomous data collection for the terrain conditions of a cocoa farm.

2.7 Summary

The review leaves five findings. First, existing agricultural data-collection robots trade precision against autonomy and cost, and the benchmark systems assume structured rows, reliable satellite positioning, or both. Intercropped, canopy-dense farms do not reliably provide either condition (Section 2.2). Second, terrain-aware navigation frameworks for unstructured ground can be deterministic and map-based or risk-aware and mapless. The deterministic, map-based form fits land that can be surveyed once and revisited, but the reviewed frameworks still predict motion with no-slip models and obtain range data from 3D LiDAR (Section 2.3). Third, skid-steer robots slip by construction, and although a physically interpretable friction model exists, its authors leave real-time control integration as future work (Section 2.4). Fourth, low-cost agricultural robotics has reduced computing and mechanical cost or narrowed the task, but has not shown an affordable platform that remains reliable on wet, uneven ground, and has not dealt with the cost of range sensing itself (Section 2.5). Fifth, the documented work in Ghana and West Africa is aerial monitoring; the sources reviewed report no ground-based autonomous deployment (Section 2.6).

These gaps are related rather than separate: a terrain-aware framework that assumes no slip, a slip model never placed in a controller, and a cost structure set largely by the sensor that framework depends on. Chapter 3 develops the design theory that addresses them together.

CHAPTER 3 - DESIGN THEORY AND CONSIDERATIONS

3.1 Introduction

Chapter 2 reviewed approaches to autonomous navigation on rough terrain and to modelling the motion of skid-steer robots, and identified the gap this project addresses. The Hybrid A*-guided Model Predictive Path Integral (MPPI) framework of Yang et al. builds a traversability map from 3D point clouds and uses it to plan and control the robot's motion, but it predicts that motion with a standard kinematic model in which the wheels do not slip [16]. Rabiee and Biswas derived a friction-based kinematic model that predicts the slip of skid-steer robots, and named its integration into the control of such robots as future work [17]. A four-wheel skid-steer robot on a cocoa farm needs both: it can only turn by making its wheels slip, and it must find its way between trees on uneven ground. In addition, the authors' released implementation of the navigation framework obtains its point clouds from a 3D light detection and ranging (LiDAR) sensor [30], which is costly for the research teams the robot is meant to serve.

This chapter sets out the theory behind the system and the considerations that shaped each design choice.

The chapter follows the order in which the parts of the system depend on one another. Section 3.2 begins with the robot and the farm it works in, because every later result depends on how faithfully they are represented, and explains why simulation is used as the test instrument. Section 3.3 then shows why a skid-steer robot cannot be described by the no-slip kinematics of that navigation framework, and introduces the friction-based model that replaces them. Section 3.4 builds on that model: it explains how the robot turns range data into a traversability map, plans a route with Hybrid A*, and follows it with MPPI, whose motion predictions are where the friction model enters; it closes with the two range sensors compared in this work and the measures used to judge navigation. Section 3.5 turns to the task the autonomy exists to serve, the tree-image survey, and Section 3.6 summarises the design choices carried into Chapter 4.

3.2 Platform and Environment

The platform is the robot; the environment is the cocoa farm it works in. Both were designed first and then represented in simulation, which served as the test instrument for the rest of the project. This section sets out the requirements the robot was designed to

meet, the choice of drivetrain and how it is driven, the reasons for testing in simulation, and what has to be considered when a robot and a farm are represented in a physics simulator.

3.2.1 Operating Environment and Design Requirements

The robot is intended for Ghanaian cocoa farms. Cocoa in Ghana is typically planted at about $3 \text{ m} \times 3 \text{ m}$, roughly 1,100 trees per hectare, and older farms are often planted more loosely at about $4 \text{ m} \times 4 \text{ m}$. Most farms are smallholdings with irregular spacing and no clear rows. The trees grow 5 to 8 m tall and form a mid-level canopy, the ground between them is covered by fallen leaves and roots over damp soil, and visibility through the farm is limited to roughly 10 to 20 m. Five requirements follow for a robot that must move between the trees and carry sensors to them.

Passing between trees. For a robot of overall width W passing between two trees a distance d_t apart, with a clearance of at least c on each side,

$$W \leq d_t - 2c. \quad (3.1)$$

With the typical spacing $d_t = 3.0 \text{ m}$ and a clearance of $c = 0.5 \text{ m}$, the robot can be no wider than 2.0 m.

Ground clearance. Uneven ground, roots and wet patches can catch the underside of a low robot. A high ground clearance prevents the chassis from dragging, reduces the build-up of soil and debris around the drivetrain, and leaves space under the body for sensors or tools. The design targets a clearance of 25 cm.

Wheel size. If the underside of the frame sits a height h_f above the wheel axle, the ground clearance h_c and the wheel radius r are related by

$$h_c = r + h_f \implies r = h_c - h_f. \quad (3.2)$$

With $h_c = 0.25 \text{ m}$ and $h_f = 0.15 \text{ m}$, the wheel radius is 0.10 m, a diameter of 20 cm.

Load on each wheel. With the robot's mass m shared equally between its four wheels, the static normal load on each wheel is

$$N = \frac{mg}{4}, \quad (3.3)$$

where g is the gravitational acceleration. The design estimate of 20 kg gives $N \approx 49$ N. This load reappears in the friction model of Section 3.3, where every friction force is proportional to it.

Modular payload. The robot is a platform whose job is to carry the sensors a research team needs to the places it needs them. The design therefore reserves payload space on the chassis and treats the sensor set as interchangeable.

3.2.2 Drivetrain Topology and Actuation

A skid-steer robot has no steering mechanism. Its wheels are fixed parallel to the body, and it changes heading by driving the wheels on one side faster than those on the other. Because the wheels cannot turn to follow a curve, every turn forces the tyres to slide sideways across the ground. This sliding, called scrubbing, makes the ground part of the drivetrain: the same command produces a different turn on a different surface.

Two drivetrain topologies were compared, as summarised in Table 3.1.

Table 3.1: Comparison of skid-steer drivetrain topologies

Criterion	Four motors (one per wheel)	Two motors (one per side)
Motor drivers	Four	Two
Wiring and electronics	More complex	Simpler
Wheel speed control	Each wheel independently; all four must be kept in step in software	One speed per side; supported directly by standard differential-drive controllers
Effect of one motor failing	The robot can still partly operate	One side is disabled
Rough terrain	Torque can be distributed between wheels	Less adaptation at individual wheels
Link between wheels on a side	Not needed	Required: chain, belt or shaft

The two-motor topology was selected. Farm terrain is rough but consistent enough that independent wheel control is rarely needed, the lower cost leaves more of the budget for sensors, and a simpler system is more reliable in the field. It is also the topology of the Clearpath Jackal, the 16 kg robot on which Rabiee and Biswas collected their data, in which each of two motors drives the two wheels on one side [17].

Torque needed to turn in place. The drivetrain must overcome scrub resistance as well as the rolling resistance of straight-line driving. Consider the robot rotating on the spot about its centre, with wheelbase b , track width w and each wheel carrying the load N of Equation (3.3). Each wheel lies ahead of or behind the centre and slides sideways as the robot rotates. With a lateral friction coefficient μ_y , each wheel resists with a force $F_y = \mu_y N$ at a lever arm of $b/2$, giving a total resisting moment of $LBNL/2\mu_y\mu_yNL/2$

$$M_r = 4 \mu_y N \frac{L}{2} = 2 \mu_y N L. \quad (3.4)$$

The motors drive the two sides in opposite directions. A longitudinal force f_x at each wheel acts at a lever arm of $B/2$, giving a turning moment of

$$M_d = 4 f_x \frac{B}{2} = 2 f_x B. \quad (3.5)$$

The robot turns in place only if $M_d \geq M_r$, which requires a longitudinal force at each wheel of

$$f_x \geq \mu_y N \frac{L}{B}. \quad (3.6)$$

Since each motor drives two wheels of radius r , the torque required from each side is

$$\tau_{\text{side}} = 2 f_x r \geq 2 \mu_y N r \frac{L}{B}. \quad (3.7)$$

The longitudinal force a wheel can transmit is itself limited by traction, $f_x \leq \mu_x N$, where μ_x is the longitudinal friction coefficient. Turning in place is therefore possible at all only if

$$\frac{\mu_x}{\mu_y} \geq \frac{L}{B}. \quad (3.8)$$

Equations (3.4) to (3.8) neglect rolling resistance and assume that the sliding force at each wheel acts purely sideways. They nevertheless show the two quantities that set the demand on the drivetrain when turning: the resistance of the ground to sideways sliding, and the ratio of wheelbase to track width.

Actuation. In simulation a wheel joint can be driven through a velocity interface or an effort interface. Through a velocity interface, the simulator imposes the commanded wheel speed and applies whatever torque that requires, so the wheel keeps its speed however little grip the ground provides. Through an effort interface, the controller applies a torque, and the wheel speed follows from the balance between that torque, the wheel's inertia and the friction at the contact. Only the second lets the wheels lose grip as a real motor's wheels do when its torque limit or the available traction is reached. This matters because slip is the quantity the friction model describes. An early test in this project showed that, under

velocity control, the robot's turn rate on flat ground did not change as the ground friction coefficient was varied, so the slip the model needs was absent from the motion.

A real robot sits between the two cases. The Jackal is commanded with wheel velocities, but its motor drivers limit the motor current and therefore the torque (Clearpath Jackal user manual

(https://docs.clearpathrobotics.com/docs_robots/legacy/ros1_robots/outdoor_robots/jackal/user_manual_jackal/)). The drivetrain is therefore modelled as a velocity-tracking loop whose output is a torque for each side, limited to a maximum value: the command is a velocity, as on the real robot, but the wheels slip whenever the torque limit or the available traction is exceeded.

Representing the two-motor topology also requires the two wheels on each side to turn together, as a chain or shaft would force them to. A rigid constraint between the two joints is the obvious representation, but the physics engine used here does not enforce such constraints (Section 3.2.3). The link is instead approximated by a stiff torsional spring and damper acting on the difference in rotation between the two wheels on a side, applied at every physics step, so that the wheels may differ by a small angle but cannot drift apart in speed.

3.2.3 Simulation as the Test Instrument

The navigation system in this project was developed and evaluated in simulation, for five reasons.

- **Repeatability.** The same farm, starting pose, goal and random seed can be run again, so two versions of the system can be compared under identical conditions.
- **Isolation of causes.** A robot that fails in the field may do so because of terrain, sensing, control, localisation, collisions and mechanical limits at once. In simulation these can be introduced and tested one at a time.
- **Ground truth.** The true position of every tree and the true pose of the robot are known exactly, so maps and trajectories can be scored directly.
- **Safety.** Faulty behaviour can be found and corrected without risk to hardware, crops or people.
- **Reuse.** The same worlds, robot model and missions form a benchmark against which later changes can be measured.

The limitation is equally direct. The simulated robot moves according to a physics engine, not real soil, so results obtained in simulation describe the simulated robot, and field testing remains a separate step.

The software used is summarised in Table 3.2.

Table 3.2: Simulation software

Component	Selection	Role
Middleware	Robot Operating System 2 (ROS 2), Jazzy release	Communication between perception, planning and control nodes; <code>ros2_control</code> separates controllers from the joints or motor drivers they command [32]
Simulator	Gazebo Harmonic	Simulates the robot, the world and the sensors, and builds terrain from heightmaps; the Gazebo release paired with ROS 2 Jazzy [33]
Physics engine	Dynamic Animation and Robotics Toolkit (DART), through Gazebo's physics plugin	Rigid-body dynamics and contact

DART was retained over Bullet-Featherstone, the alternative engine tried in this project: Bullet-Featherstone could not load the heightmap terrain on which all the farm worlds are built. Three behaviours of DART, found during development, shaped how the robot and the farm were modelled. Friction coefficients set on the wheels have no effect; only the friction of the ground surface acts, so different terrains are represented by different ground friction values. Contact stiffness and damping parameters are ignored. Constraints that tie the motion of one joint to another, such as mimic and gearbox joints, are not enforced, which is why the side link of Section 3.2.2 is modelled as a coupling torque.

3.2.4 Representing the Robot in Simulation

The robot is described to the simulator in the Unified Robot Description Format (URDF), written with the Xacro macro language. The description is a tree of rigid links connected by joints, and each link carries three separate descriptions, each used by a different part of the simulator:

- **inertial properties, mass, centre of mass and inertia tensor, used by the physics engine to compute motion;**
- **collision geometry,** used by the physics engine to detect and resolve contact;
- **visual geometry,** used by the renderer and by the sensors that work by rendering the scene.

Because the three are independent, any one can be wrong without the others showing it: a robot can look correct while its mass or its contact geometry is not. Two groups of properties need particular care for a skid-steer robot.

Inertial properties. Computer-aided design (CAD) tools can export mass properties automatically, but they compute them from the density of the modelled solids. A chassis modelled as solid blocks, or a cargo bay modelled as a filled volume, then receives a far larger mass than the real robot, so exported values have to be checked against the intended mass. The wheels need a further check. For a wheel whose mass lies mostly towards the rim, as with a tyre on a hub, the moment of inertia about the axle is expected to lie between that of a uniform disc and that of a thin ring of the same mass m and radius r :

$$\frac{1}{2} mr^2 \leq I_{\text{axle}} \leq mr^2 . \quad (3.9)$$

The wheel's centre of mass must also lie on its axle. By the parallel axis theorem, a centre of mass displaced a distance d from the axle raises the moment of inertia about the axle to

$$I = I_c + md^2 , \quad (3.10)$$

where I_c is the moment of inertia about the centre of mass. An offset that cannot be seen in the rendered model can therefore change how the wheels respond to torque.

Collision geometry and wheel-ground contact. A physics engine resolves contact between rigid bodies; it does not model the deformation of a tyre. The shape chosen for a wheel's collision geometry therefore decides where the contact forces act. A cylinder touches flat ground along a line across the full width of the tread, and a sphere touches it at a single point. A mesh shaped like a tyre touches over its tread, and a crowned tread, rounded across its width, touches over a narrow band near its centre. For a robot that turns by scrubbing, this choice changes how strongly the ground resists turning. In this project, a cylinder collision shape stopped the robot turning under velocity control, and narrowing the contact band raised the turn rate the robot could reach; both results are reported in Chapter 5. Whatever the collision shape, its outer radius must match the wheel radius used elsewhere in the model, or the ride height and the relation between wheel speed and ground speed change with it.

3.2.5 Representing a Cocoa Farm

The simulated farm has two parts: the ground and the trees.

Terrain. The ground is represented as a heightmap, a greyscale image in which the brightness of each pixel encodes the elevation at that point, and from which the simulator builds the terrain surface. Using the same heightmap for the visual surface and the collision surface keeps the ground the robot is seen to drive on identical to the ground it physically drives on. Small-scale relief gives the surface the unevenness of a farm floor.

The original terrain also carried a directional slope. This slope was removed from the terrain used for the friction model and for the navigation experiments, because the friction-based model of Section 3.3 describes motion on a plane and has no term for gravity acting along a slope; on sloped ground, the effect of the slope would be attributed to friction. Subtracting the best-fitting plane from the elevation data removes the overall grade while keeping the local relief.

Surface types such as leaf litter and exposed laterite differ in the grip they give. Because only ground friction acts in the physics engine (Section 3.2.3), each surface type is represented by its own ground friction coefficients, together with a matching visual texture.

Trees. Repeating a single tree model across the farm would present the robot with identical obstacles everywhere. The trees are instead generated from one parametric cocoa-tree model whose height, trunk fork height, trunk thickness and pod placement vary from tree to tree. Pods are placed on the trunk and main branches, where cocoa bears its fruit.

Each tree carries the same three kinds of description as the robot's links. Its visual geometry includes the canopy, branches, trunk and pods. Its collision geometry is limited to a tube following the visible trunk up to the fork and an ellipsoid for each pod below a height of 2.5 m; the canopy has no collision geometry. The trees are declared static, so the physics engine does not compute their motion. Because a farm of a hundred trees must still render and simulate quickly enough to be usable, geometric detail was reduced where it is least visible, in the pods rather than the leaves.

The split between visual and collision geometry matters for sensing. The simulated depth camera and 3D LiDAR are both rendering-based sensors: they measure the visual geometry, not the collision geometry (Gazebo Harmonic sensor documentation (<https://gazebo.org/docs/harmonic/sensors/>) and Gazebo Sensors library documentation (<https://gazebo.org/libs/sensors/>)). They therefore return points from the canopy, which has no collision geometry, as well as from the trunks, which do. A map built from these

sensors must tell the two apart, which is addressed in Section 3.4. For the same reason, the trunk's collision tube is built to follow the visible trunk, so that the obstacle the sensors observe and the obstacle the robot can strike are in the same place.

Placement. Trees are placed by Poisson-disk sampling, which keeps a minimum distance between neighbouring trees while leaving the layout irregular, as on a smallholder farm. Each tree must also sit on the terrain surface. Because procedurally varied trees differ in the offset between their model origin and their lowest point, the height at which each tree is placed has to be corrected tree by tree.

3.3 Skid-Steer Motion Modelling

A planner or controller that chooses commands for a robot needs a motion model: a rule that predicts how the robot will move in response to a command. This section explains why the model used in the navigation framework of Yang et al. does not describe a skid-steer robot, presents the friction-based kinematic model that replaces it, and sets out how the parameters of that model are fitted to data and how the accuracy of its predictions is measured.

3.3.1 The No-Slip Kinematic Model

The robot's motion is described in a body frame fixed at its geometric centre, with the x axis pointing forward and the y axis to the left (Figure 3.1). The left and right wheels turn at angular velocities ω_l and ω_r , the effective wheel radius is r and the track width is B . The robot's velocity in the body frame has a longitudinal component v_x , a lateral component v_y and a yaw rate v_ϕ .

If no wheel slips, the contact point of each wheel moves at speed $r\omega$ in the direction the wheel faces, and the body velocity is

$$\begin{bmatrix} v_x \\ v_y \\ v_\phi \end{bmatrix} = \begin{bmatrix} \frac{r}{2} & \frac{r}{2} \\ 0 & 0 \\ -\frac{r}{B} & \frac{r}{B} \end{bmatrix} \begin{bmatrix} \omega_l \\ \omega_r \end{bmatrix}. \quad (3.11)$$

This is the kinematic model of an ideal differential-drive robot. The robot's pose $\mathbf{x} = [x, y, \phi]^T$ in the world frame, where ϕ is the heading, is then predicted over a time step Δt by

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \Delta t \begin{bmatrix} v_x \cos \phi_t \\ v_x \sin \phi_t \\ v_\phi \end{bmatrix}. \quad (3.12)$$

Equations (3.11) and (3.12) together are the motion model used by Yang et al., who write them in terms of the ground speeds of the two wheels, $v_l = r\omega_l$ and $v_r = r\omega_r$ [16]. Because Equation (3.11) can be inverted, the same model can take a forward speed $v = v_x$ and a turn rate $\omega = v_\phi$ as its inputs. In that form it is known as the unicycle model, the name used for it in Chapters 4 and 5.

Why skid-steer turning breaks the model. Equation (3.11) assumes that no wheel slides. For a robot with two driven wheels on a single axle, this assumption is consistent: the robot turns about a point on the line of the axle, and each wheel can roll along a circle about that point. A four-wheel skid-steer robot has two axles a wheelbase L apart, and all four wheels are fixed parallel to the body. A wheel moves without sideways sliding only if the instantaneous centre of rotation (ICR) lies on the line through that wheel's axle, and the ICR cannot lie on both axle lines at once. Whenever the robot turns, at least one pair of wheels must therefore slide sideways. This is the scrubbing of Section 3.2.2, and it has two consequences that Equation (3.11) cannot represent.

First, the lateral friction at the sliding wheels produces a moment that resists the turn, as in Equation (3.4). The drive wheels then slip longitudinally as well, and the robot turns more slowly than $r(\omega_r - \omega_l)/B$. Second, the ICR need not lie level with the geometric centre. If it lies a distance x_{ICR} ahead of the centre, rigid-body motion gives the centre a lateral velocity of

$$v_y = -x_{ICR} v_\phi, \quad (3.13)$$

which Equation (3.11) sets to zero.

The simplest correction, the extended differential-drive model, keeps $v_y = 0$ and the forward velocity of Equation (3.11), but divides the turn rate by an ICR coefficient χ that depends on the terrain [34]:

$$v_\phi = \frac{r(\omega_r - \omega_l)}{\chi B}, \quad \chi \geq 1. \quad (3.14)$$

The robot is treated as a differential-drive robot with a wider effective track, χB , and $\chi = 1$ recovers the no-slip model. Other models make χ a function of the radius of the turn, because skid-steer robots lose proportionally more turn rate in tight turns [35], or fit a general linear map from wheel velocities to body velocity [36]. All of these take the wheel velocities as their only input. None can therefore predict the additional slip that occurs when the robot accelerates or changes the direction of its turn [17]. A sampling-based controller evaluates many command sequences that change from one time step to the next (Section 3.4), which is exactly the regime these models leave out.

3.3.2 The Friction-Based Kinematic Model

Rabiee and Biswas proposed a model that predicts slip from the forces between the wheels and the ground [17]. Its inputs are the commanded wheel velocities $\mathbf{u} = [\omega_l, \omega_r]^T$ and their rates of change $\dot{\mathbf{u}} = [\dot{\omega}_l, \dot{\omega}_r]^T$, and its output is the body velocity $\mathbf{V} = [v_x, v_y, v_\phi]^T$. The model is kinematic in its inputs and outputs, like Equation (3.11), but it uses the robot's equations of motion internally to decide how much the wheels slip. It is built in four steps.

Slip at a wheel. For a wheel of radius r turning at ω , whose centre moves with longitudinal velocity V_{w_x} and lateral velocity V_{w_y} , the slip ratio s and slip angle α are

$$s = 1 - \frac{V_{w_x}}{r\omega}, \quad \alpha = \arctan\left(\frac{V_{w_y}}{V_{w_x}}\right). \quad (3.15)$$

A wheel that rolls without sliding has $s = 0$ and $\alpha = 0$. A wheel that spins without moving has $s = 1$.

Velocity with slip. The model assigns one slip ratio to each side of the robot, s_l and s_r , as suits a robot whose wheels on a side are driven together. This matches the two-motor drivetrain of Section 3.2.2. Adding the two slip ratios and the ICR offset to Equation (3.11) gives

$$\begin{bmatrix} v_x \\ v_y \\ v_\phi \end{bmatrix} = \frac{r}{B} \begin{bmatrix} \frac{(1-s_l)B}{2} & \frac{(1-s_r)B}{2} \\ x_{ICR}(1-s_l) & -x_{ICR}(1-s_r) \\ -(1-s_l) & (1-s_r) \end{bmatrix} \begin{bmatrix} \omega_l \\ \omega_r \end{bmatrix}. \quad (3.16)$$

Slip reduces the ground speed of each side, and the second row reproduces the lateral velocity of Equation (3.13). If the three unknowns $\theta = \{s_l, s_r, x_{ICR}\}$ were known,

Equation (3.16) would give the robot's velocity directly. The remaining steps estimate them.

Forces at a wheel. Wheel i sits at $(x_i, y_i) = (\pm L/2, \pm B/2)$ in the body frame, so its centre moves with

$$V_{w_x,i} = v_x - y_i v_\phi, \quad V_{w_y,i} = v_y + x_i v_\phi. \quad (3.17)$$

Three forces act at the wheel, each proportional to its normal load N_i . The rolling resistance, caused by the deformation of the tyre and the ground, opposes the wheel's direction of travel:

$$R_i = -\mu_r N_i \operatorname{sgn}(V_{w_x,i}). \quad (3.18)$$

For forward motion this is the form $\mathbf{R}_i = -\mu_r N_i \hat{x}$ given by Rabiee and Biswas. The longitudinal tractive force follows a piecewise-linear model and the lateral force an exponential model [37]:

$$f_{x_i} = \operatorname{sgn}(s_i) \operatorname{sgn}(V_{w_x,i}) \min(|s_i| \lambda N_i, \mu_x N_i), \quad (3.19)$$

$$f_{y_i} = -\operatorname{sgn}(V_{w_y,i}) \mu_y \left(1 - e^{-|\alpha_i|/C}\right) N_i. \quad (3.20)$$

The longitudinal force rises in proportion to slip, with slope λ , until it reaches the traction limit $\mu_x N_i$. The lateral force rises with the slip angle, at a rate set by C , and approaches $\mu_y N_i$. The five parameters, summarised in Table 3.3, describe one combination of robot and ground and are found from data (Section 3.3.3). The coefficients μ_x and μ_y play the same roles as in the turning analysis of Section 3.2.2, where Equation (3.8) showed that their ratio decides whether the robot can turn on the spot.

Table 3.3: Parameters of the friction-based kinematic model

Symbol	Meaning	Equation
μ_r	Coefficient of rolling resistance	(3.18)
μ_x	Longitudinal friction coefficient: the traction limit as a fraction of normal load	(3.19)
λ	Slope of longitudinal force against slip ratio, below the traction limit	(3.19)
μ_y	Lateral friction coefficient: the limit of sideways force as a fraction of normal load	(3.20)
C	Slip-angle scale over which the lateral force approaches its limit	(3.20)

Balance of forces. The unknowns θ must be such that these forces produce the robot's motion. Newton's second law along the two body axes, and about the vertical axis through the centre of gravity at (x_{CG}, y_{CG}) , gives three residuals:

$$F_1(\theta) = \sum_{i=1}^4 (f_{x_i} + R_i) - m a_x, \quad (3.21)$$

$$F_2(\theta) = \sum_{i=1}^4 f_{y_i} - m a_y, \quad (3.22)$$

$$F_3(\theta) = \sum_{i=1}^4 [(x_i - x_{CG}) f_{y_i} - (y_i - y_{CG}) (f_{x_i} + R_i)] - I_z a_\phi, \quad (3.23)$$

where m is the mass of the robot, I_z its moment of inertia about the vertical axis through the centre of gravity, and a_x , a_y and a_ϕ the accelerations of the centre of gravity. The body frame rotates with the robot, so the linear accelerations include rotational terms:

$$a_x = \dot{v}_{CG_x} - v_\phi v_{CG_y}, \quad a_y = \dot{v}_{CG_y} + v_\phi v_{CG_x}, \quad (3.24)$$

with $v_{CG_x} = v_x - y_{CG} v_\phi$ and $v_{CG_y} = v_y + x_{CG} v_\phi$. Rabiee and Biswas write the rotational terms using the radius of curvature and the angle γ of the centre of gravity's velocity. For a centre of gravity on the robot's centreline, $y_{CG} = 0$, as in this robot, the form of Equation (3.24) gives the same terms. It also avoids dividing by the radius of curvature, which is unbounded in straight-line motion and zero when the robot turns on the spot.

The actual accelerations are not known when a prediction is made. The model therefore replaces $\dot{v}_{CG_x}$, $\dot{v}_{CG_y}$ and a_ϕ with the accelerations implied by the commands:

$$\dot{v}_{cmd_x} = \frac{r}{2} (\dot{\omega}_r + \dot{\omega}_l), \quad \dot{v}_{cmd_y} = 0, \quad \dot{v}_{cmd_\phi} = \frac{r}{B} (\dot{\omega}_r - \dot{\omega}_l). \quad (3.25)$$

Rabiee and Biswas describe this substitution as a zero-order approximation. A real robot may not reach the commanded acceleration because of slip and motor dynamics, and the commands are assumed to respect the robot's acceleration limits [17]. The substitution is

what allows the model to predict more slip when the robot is commanded to accelerate than when it moves steadily.

Solving for slip. With Equations (3.15) to (3.25), the three residuals $\mathbf{F} = [F_1, F_2, F_3]^T$ depend only on θ , $\mathbf{u}$ and $\dot{\mathbf{u}}$. At each time step the slip state is found as

$$\theta^* = \arg \min_{\theta} \mathbf{F}(\theta)^T \mathbf{F}(\theta), \quad (3.26)$$

by trust-region optimisation, and θ^* is substituted into Equation (3.16) to give the predicted velocity [17].

Considerations in applying the model. Five considerations follow from the structure of the model.

- *A spurious solution.* Equation (3.26) does not have a single solution in every case. When both slip ratios equal one, Equation (3.16) gives zero velocity. Every wheel velocity in Equation (3.17) is then zero, and so are the forces of Equations (3.18) to (3.20). For a steady command, the commanded accelerations of Equation (3.25) are also zero, and all three residuals vanish. This total-slip solution states that the robot does not move whatever it is commanded to do, so the search for θ has to be restricted to slip ratios smaller than one in magnitude.
- *Planar motion.* The model has no term for gravity acting along a slope. This is why the slope was removed from the terrain on which it is used (Section 3.2.5).
- *Normal loads.* The model allows a different load N_i at each wheel. The centre of gravity of this robot lies close to its geometric centre, and on level ground the static load is shared almost equally, so the loads are taken as equal, $N_i = N$ from Equation (3.3). The transfer of load between wheels during acceleration is neglected.
- *The command as input.* The model maps commands, not wheel torques, to motion. Everything between the command and the ground is therefore absorbed into the fitted parameters together with the ground itself: the velocity-tracking loop, the torque limit and the side link of Section 3.2.2. A set of parameters describes one drivetrain configuration on one surface, and must be fitted again if the torque limit or the wheel contact geometry changes.
- *Computational cost.* Each prediction needs the iterative solution of Equation (3.26), whereas Equation (3.11) is a single matrix product. Section 3.4 considers what this

means for a controller that must make a large number of predictions in every control cycle.

3.3.3 Parameter Identification

The five parameters $\mathcal{P} = \{\mu_r, \mu_x, \mu_y, \lambda, C\}$ are found from recorded driving data: the commanded wheel velocities and their rates of change are the inputs, and the robot's measured body velocity is the target. Rabiee and Biswas fitted them by nonlinear least squares, minimising the error in predicted velocity over a training dataset [17]. Because every prediction requires the slip solution of Equation (3.26), the fit is a nested problem. For each candidate $\mathcal{P}$, the outer optimiser obtains a predicted velocity $\hat{V}_j(\mathcal{P})$ for every training sample j by solving Equation (3.26) with that $\mathcal{P}$, and evaluates

$$J_{\text{fit}}(\mathcal{P}) = \sum_{j=1}^n \sum_{k \in \{x, y, \phi\}} \left(\frac{\hat{v}_{k,j}(\mathcal{P}) - v_{k,j}}{\sigma_k} \right)^2, \quad (3.27)$$

where n is the number of training samples, $v_{k,j}$ is the measured velocity in channel k , and σ_k is a scale for that channel. Six considerations shape how this fit is carried out.

- *Weighting of the channels.* The linear velocities are measured in metres per second and the yaw rate in radians per second, and their typical magnitudes differ. Dividing each residual by a scale such as the standard deviation of that channel in the training data stops any one channel from dominating the fit because of its units. The lateral velocity of a skid-steer robot is small, so its scale is given a lower limit; otherwise, a channel with little variation would receive a disproportionate weight.
- *Positive parameters of different magnitudes.* All five parameters are positive, and they differ in magnitude. Optimising the logarithms of the parameters keeps them positive and treats equal relative changes in each alike. Upper and lower bounds keep the search within physically plausible values, which calls for an optimiser that handles bounds, such as a trust-region reflective method [38].
- *No analytic gradient.* The prediction passes through an iterative solution, so the gradient of J_{fit} with respect to $\mathcal{P}$ is estimated by finite differences. Each estimate requires the training samples to be predicted again for every parameter perturbed. The cost of the fit therefore grows with the number of samples, and a random subset of the training data is used rather than every recorded sample.

- *Starting points.* The objective J_{fit} is not convex, and a local optimiser started from different points may settle at different solutions. The fit is therefore run from more than one starting point, and the candidates are compared on data that played no part in fitting.
- *Coverage of the data.* The training data should cover the conditions in which the model will be used: straight driving, turns of different radii and changes of speed. Rabiee and Biswas trained on data driven by hand with a wide range of velocities and accelerations, from wide, slow turns to sharp turns with high acceleration [17].
- *Separation of test data.* Accuracy on the training data overstates accuracy in use. Consecutive samples of a recording made at a high rate are nearly identical, so test samples drawn at random from the same recording would include near-copies of training samples. Held-out data are therefore taken from a later part of the recording than the training data. A separate recording tests the model under conditions different from those it was fitted on. Following the dataset categories of Rabiee and Biswas, this is a set of constant-velocity turns at a range of fixed radii [17].

3.3.4 Measures of Prediction Accuracy

The parameters are fitted to minimise velocity error, and the controller of Section 3.4 predicts the robot's motion by integrating predicted velocities step by step. Prediction accuracy is therefore measured on velocity. For each channel k over n test samples, the root-mean-square error is

$$\text{RMSE}_k = \sqrt{\frac{1}{n} \sum_{j=1}^n (\hat{v}_{k,j} - v_{k,j})^2}. \quad (3.28)$$

Each channel is reported separately, because the units differ and because a model can improve one channel while worsening another. The friction-based model is compared with the no-slip model of Equation (3.11), evaluated on the same commands, by the relative improvement

$$E_k = 100 \left(1 - \frac{\text{RMSE}_k^{\text{friction}}}{\text{RMSE}_k^{\text{no-slip}}} \right) \%. \quad (3.29)$$

A positive E_k means that the friction-based model predicts channel k more accurately than the no-slip model, and a negative value means that it predicts it less accurately. The no-slip

model predicts $v_y = 0$ for every sample, so its lateral RMSE is the root-mean-square of the measured lateral velocity itself.

Two checks accompany the errors. The first is the fraction of samples for which the slip solution of Equation (3.26) converged. The second is the fraction at which a slip ratio reached its bound, which would show the search being held at the edge of its permitted range rather than at a solution.

Rabiee and Biswas measured accuracy differently: they integrated each model over a horizon of 6 s and compared the predicted pose with the true pose, normalising the error by the distance travelled and the angle turned [17]. That measure shows how velocity errors accumulate into position errors, but its value depends on the horizon chosen. In this project the effect of the motion model on the path the robot follows is assessed in closed loop, with the navigation measures of Section 3.4.

3.4 Terrain-Aware Autonomous Navigation

To reach a tree, the robot must build a picture of the ground around it, choose a route, and turn that route into wheel commands, all while it moves. This section first explains the choice of navigation framework. It then follows the framework's flow of data: from range measurements to a map of how passable the ground is, from that map to a planned path, and from the path to commands. It closes with how goals beyond the map are reached, the two range sensors compared in this work, and how navigation performance is measured. Figure 3.2 shows the parts of the system and how they connect.

3.4.1 Choice of Navigation Framework

Sections 3.2 and 3.3 set four requirements for the navigation framework. It must judge the ground as well as detect obstacles, because a cocoa farm has uneven relief as well as trees. It must run in real time on an embedded computer carried by the robot. It must be able to predict motion with a model that represents skid-steer slip, such as the friction-based model of Section 3.3. And it must reach goals among closely spaced trees without becoming trapped. Five approaches were compared against these requirements: Hybrid A*-guided MPPI; Stochastic Traversability Evaluation and Planning (STEP); standard model predictive control (MPC); the dynamic window approach (DWA); and pure pursuit. The comparison is summarised in Table 3.4.

Table 3.4: Comparison of motion planning and control approaches

Criterion	Hybrid A*-guided MPPI	STEP	Standard MPC	DWA	Pure pursuit
Terrain awareness	High: continuous slope and roughness costs from a 2.5D elevation map (one height per grid cell), used by both planner and controller	High: continuous traversability costs from several risk factors, with conditional value-at-risk to guard against rare severe outcomes	Low: terrain enters only if added to the cost or constraints	Low: each cell is either obstacle or free	None: a given path is followed geometrically
Computation on an embedded computer	Suitable: sampled trajectories are independent and can be evaluated in parallel; guidance reduced the samples needed from 2,000 to 800	Demanding: an optimisation problem is solved by sequential quadratic programming at each control step	Moderate: a numerical optimisation at each step, sensitive to solver tuning	Low: a small window of velocities is sampled	Minimal
Slip and robot dynamics	Any motion model that can be simulated forward can be used, including one without derivatives	Dynamic constraints are included in the controller	The model must suit the solver; replacing it means reformulating the problem	Ideal kinematics over a short window	Slip is ignored
Reliability among obstacles	The planned path guides the sampler away from local minima	Late solutions when the solver overruns its time	No native terrain awareness	Reactive, and prone to local minima among clustered obstacles	Drifts or overshoots when the wheels slip

Sources: Hybrid A*-guided MPPI [16]; STEP [23]; MPPI [39]; standard MPC [40]; DWA [41]; pure pursuit [42].

Hybrid A*-guided MPPI was selected. It is the only candidate that uses continuous terrain costs in both planning and control while leaving the motion model free. The second property follows from how MPPI works. MPPI evaluates each candidate command sequence by simulating the robot forward and scoring the result, and it never differentiates the motion model [39]. The friction-based model, whose every prediction requires the iterative solution of Equation (3.26), can therefore replace the no-slip model of Yang et al. without any change to the optimiser. STEP matches it on terrain awareness, but its controller relies on a gradient-based optimisation that is both heavier and less tolerant of a model defined by an iterative solve. The framework also needs no prior map of the farm: it plans on a local map built from the robot's own sensors as it moves [16], which suits farms for which no survey exists.

3.4.2 Traversability Estimation from Elevation Maps

Elevation map. The ground around the robot is represented as a 2.5D elevation map: a grid in which each cell holds one height. The map is updated from 3D point clouds as the robot moves, and the uncertainty in the robot's motion is carried into the uncertainty of each cell [43]. The map covers a square window centred on the robot and moves with it.

Slope and roughness. A height alone does not say whether the ground can be driven. Yang et al. derive two scores from the map [16]. The slope of a cell follows from the vertical component n_z of the unit surface normal $\mathbf{n}(i, j)$ fitted around it:

$$\zeta(i, j) = \arccos(n_z(i, j)), \quad S_s(i, j) = \min\left(\frac{\zeta(i, j)}{\zeta_c}, 1\right), \quad (3.30)$$

where ζ_c is the steepest slope considered passable. The roughness of a cell is the standard deviation of the distances of the n_p valid neighbouring points $\mathbf{p}_k$ from the plane through the cell with normal $\mathbf{n}(i, j)$:

$$\sigma_{\perp}(i, j) = \sqrt{\frac{1}{n_p - 1} \sum_{k=1}^{n_p} (\mathbf{n}(i, j) \cdot \mathbf{p}_k - D)^2}, \quad S_r(i, j) = \min\left(\frac{\sigma_{\perp}(i, j)}{\sigma_{\perp, c}}, 1\right), \quad (3.31)$$

where D is the mean of $\mathbf{n}(i, j) \cdot \mathbf{p}_k$ over the neighbourhood and $\sigma_{\perp, c}$ is the largest roughness considered passable. The two scores are blended into the traversal cost of the cell,

$$l_t(i, j) = w_s S_s(i, j) + (1 - w_s) S_r(i, j), \quad (3.32)$$

with $w_s = 0.8$ reported by Yang et al. as a good balance [16]. A cost of 0 is easy ground and a cost of 1 is impassable.

Four considerations shape how this estimate is applied on a cocoa farm.

- *Noise and surface normals.* A normal fitted to noisy heights tilts at random, and flat ground then shows a false slope. Smoothing the heights before the normals are fitted removes this, at the price of blurring small features.
- *Thin obstacles.* A trunk covers only a few cells. Smoothing spreads it out, and over a neighbourhood of similar size neither slope nor roughness rates it as impassable. A

separate test therefore compares each cell's height $e(i, j)$ with the lowest height $e_{\min}(i, j)$ within a small radius, taken as the local ground:

$$\Delta e(i, j) = e(i, j) - e_{\min}(i, j), \quad \text{cell impassable if } \Delta e(i, j) > h_p. \quad (3.33)$$

The protrusion threshold h_p is chosen relative to the wheel radius, because a wheel cannot roll onto a step as high as its axle. The lowest height is used as the reference, rather than the mean, because a mean over a wide obstacle rises onto its top and would make the obstacle's interior read as flat ground.

- *Returns from the canopy.* The sensors measure visual geometry (Section 3.2.5), and an elevation map keeps only one height per cell. A branch hanging over drivable ground is therefore recorded as a tall obstacle standing on that ground. Before the map is updated, each return is compared with the ground of its own cell, and returns higher above that ground than the robot itself are removed. Overhead canopy is discarded at any height, while a low branch that would strike the robot, and a trunk of any height, are kept.
- *Unobserved ground.* A cell that has never been measured must not read as flat ground of zero cost. Unobserved cells are kept distinct from observed ones throughout the calculation and passed to the planner as unknown.

3.4.3 Hybrid A* Search

The A* algorithm finds the lowest-cost route through a grid by expanding cells in order of $P = G + h$, where G is the cost from the start and h an estimate of the cost remaining [44]. A route through grid cells ignores the robot's heading and how it can move, so it may not be drivable. Hybrid A* keeps a continuous pose (x, y, ϕ) at each node and expands a node by applying short motion primitives, each integrated with a motion model; the grid cell and a heading bin serve only to recognise poses already reached more cheaply [45]. The resulting path is drivable by construction.

In the framework of Yang et al., a node ν is ranked by

$$P(\nu) = G(\nu) + h(\nu), \quad G(\nu) = \sum_{k=0}^{t-1} (d_k + l_t(i_k, j_k)) , \quad (3.34)$$

so that each step pays for both its length d_k and the traversal cost of the ground it crosses. The heuristic is the Reeds–Shepp distance to the goal, the shortest path for a vehicle that has a minimum turning radius and can reverse [16].

A skid-steer robot differs from such a vehicle in ways that change the search.

- *Motion primitives.* The robot can turn on the spot, so the primitives are straight and curved segments driven forwards and in reverse, together with rotations in place. With no minimum turning radius, the Reeds–Shepp distance no longer describes the robot's motion. Turning in place is nevertheless costly for the drivetrain (Section 3.2.2), so rotations, reversing and changes of direction each carry a penalty.
- *Heuristic.* A heuristic that ignores obstacles underestimates the cost of reaching a goal behind a row of trees, and the search then explores many dead ends. A cost-to-go h_D computed over the traversability grid by Dijkstra's algorithm, expanding outward from the goal cell, accounts for both obstacles and terrain:

$$h_D(\mathbf{g}) = 0, \quad h_D(a) = \min_{b \in \mathcal{A}(a)} \left[h_D(b) + d(a, b)(1 + w_T l_t(a)) \right], \quad (3.35)$$

where $\mathcal{A}(a)$ are the neighbouring cells of a , $d(a, b)$ is the distance between cell centres, and w_T weights the terrain cost.

- *Collision checking.* A primitive whose end pose is free may still sweep through a trunk on the way. The planner therefore checks every intermediate pose along a primitive, rather than checking the end pose alone.
- *Robot footprint.* Checking a single point on the robot is fast, but the robot has extent. Obstacles are therefore enlarged by the radius of the circle that encloses the robot about that point, so that a point outside the enlarged obstacles means the whole robot is clear.

3.4.4 Model Predictive Path Integral Control

MPPI control chooses commands by sampling [39]. Over a horizon of T time steps, a nominal sequence of commands $\bar{\mathbf{u}}_0, \dots, \bar{\mathbf{u}}_{T-1}$, with $\mathbf{u} = [v, \omega]^T$, is perturbed to give K candidate sequences:

$$\mathbf{u}_t^{(i)} = \bar{\mathbf{u}}_t + \boldsymbol{\epsilon}_t^{(i)}, \quad \boldsymbol{\epsilon}_t^{(i)} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma}). \quad (3.36)$$

Each candidate is simulated forward with the motion model $\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t)$ and scored by a cost made of running terms q and a terminal term q_T :

$$J^{(i)} = q_T(\mathbf{x}_T^{(i)}) + \sum_{t=0}^{T-1} q(\mathbf{x}_t^{(i)}, \mathbf{u}_t^{(i)}) . \quad (3.37)$$

The candidates are weighted according to their cost,

$$\Omega^{(i)} = \exp\left(-\frac{J^{(i)} - J_{\min}}{\beta}\right) , \quad (3.38)$$

where $J_{\min}$ is the lowest cost among the candidates and β is a temperature. Subtracting $J_{\min}$ leaves the normalised weights unchanged but prevents every weight from underflowing to zero when all costs are large. The new nominal sequence is the weighted average of the candidates,

$$\bar{\mathbf{u}}_t \leftarrow \frac{\sum_{i=1}^K \Omega^{(i)} \mathbf{u}_t^{(i)}}{\sum_{i=1}^K \Omega^{(i)}} , \quad (3.39)$$

and its first command is sent to the robot before the process repeats at the next control step [16]. A small β makes the result follow the best few candidates; a large β averages over many.

Five considerations follow for this robot.

- *The motion model.* The model enters MPPI only through the forward simulation, so the no-slip model and the friction-based model are interchangeable. The friction-based model takes wheel velocities and their rates of change as input (Section 3.3.2). At each simulated step the command (v, ω) is converted to wheel velocities by inverting Equation (3.11),

$$\omega_l = \frac{1}{r} \left(v - \frac{\omega B}{2} \right) , \quad \omega_r = \frac{1}{r} \left(v + \frac{\omega B}{2} \right) , \quad (3.40)$$

and the wheel accelerations are taken from the change in wheel velocity between consecutive steps of the candidate sequence.

- *Computational cost.* One control step requires $K \times T$ predictions. With the friction-based model each prediction is an iterative solve, so the solver must be bounded in effort to fit within the control period, and a defined fallback prediction is needed for

the rare case in which it fails. Because the candidates do not interact, they can be simulated in parallel.

- *The cost function.* The cost rewards progress towards the goal, penalises traversal cost and control effort, and adds a large penalty for entering an enlarged obstacle cell. The obstacle penalty stands in for a hard constraint: a candidate that enters an obstacle receives a very high but finite cost, so the weights of Equation (3.38) remain defined.
- *The command envelope.* Sampled commands are limited to a range of speeds and turn rates. Because a skid-steer robot achieves only part of the turn rate it is commanded (Section 3.3.1), this range may usefully exceed what the robot can actually reach, provided the motion model predicts the turn rate that will result.
- *Repeatability.* The candidates are drawn at random. Comparing two configurations fairly requires that both draw the same random numbers, so the random generator is seeded explicitly and sampled in a fixed order, even when the candidates are simulated in parallel.

3.4.5 Guided MPPI: Seeding the Sampler from the Planned Path

Standard MPPI centres its samples on the solution of the previous step. When that solution leads into a dead end, such as the space in front of a trunk, most samples start from the same poor place. Guided MPPI instead centres the samples on a nominal command computed from the path planned by Hybrid A* [16]. For a target waypoint at position error (e_x, e_y) and heading error e_ϕ from the robot, Yang et al. set

$$v_t = k_v \sqrt{e_x^2 + e_y^2}, \quad (3.41)$$

$$\omega_t = k_\phi e_\phi + k_p \arctan(e_y, e_x), \quad (3.42)$$

where k_v , k_ϕ and k_p are gains and $\arctan(e_y, e_x)$ is the bearing of the waypoint from the robot. In their comparison, guidance reduced the number of samples needed for reliable navigation from 2,000 to 800, and the computation time per step from 9.55 ms to 3.92 ms [16].

Equations (3.41) and (3.42) assume that the next waypoint lies a useful distance ahead and that the robot turns as commanded. Neither holds automatically for a skid-steer robot following a Hybrid A* path, so this project extends the method of Yang et al. in six ways.

- *Waypoint spacing.* A Hybrid A* path holds one pose per primitive. Waypoints are therefore one primitive length apart on driving segments and at almost the same position on rotations in place. A speed proportional to the distance to the next waypoint is then fixed by the length of a primitive rather than by the manoeuvre. Selecting the target by the distance travelled and the heading change accumulated along the path makes the seed independent of how the path was discretised.
- *Reversing.* Equation (3.41) never gives a negative speed, so the seed cannot ask the robot to reverse even where the path does. On a reversing segment the speed must change sign and the bearing must be measured from the rear of the robot.
- *Waypoints at the robot's own position.* During a rotation in place, the target waypoint lies almost exactly at the robot's position, and its bearing is set by small errors in position. Its sign can then change from one step to the next. A torque-controlled drivetrain averages such alternating commands to almost no torque, and the robot stops turning. The bearing term is therefore faded in with distance, so that close targets are steered by heading error alone.
- *Scrub-limited turning.* The robot delivers only part of the turn rate it is commanded, so a seed sized for an ideal robot turns too little. The seed's angular command is scaled up before it is limited to the command envelope.
- *Staying on the path.* In the framework of Yang et al., the path guides the controller only through the centre of the sampling. Terms that penalise distance from the path and misalignment with its heading can be added to the cost of Equation (3.37), so that candidates that leave the path are also scored as worse.
- *Large changes of heading.* A forward U-turn sweeps a wide arc among the trees, whereas a skid-steer robot can rotate in place. The planner therefore penalises forward motion while the robot faces away from the goal, so that a large change of heading is planned as a rotation followed by driving. The controller, once the heading error exceeds an upper threshold, commands a rotation in place in a fixed direction until the error falls below a lower threshold; the gap between the two thresholds stops the command from switching back and forth.

The individual effect of these additions is examined in Chapter 5.

3.4.6 Reaching Goals beyond the Local Map, and the Localisation It Assumes

The map covers only a window around the robot, while a goal on a farm may lie well beyond it. Following Yang et al., a goal outside the window is projected onto the window, and the planner searches towards this local goal [16]. As the robot moves, the window moves with it, the local goal advances, and the path is planned again, so the robot approaches the true goal in a receding-horizon manner without any global map. Ground not yet observed is treated as passable, so that the planner can search into it, and the path is revised as the map fills in.

The projection must preserve the direction of the goal. Clamping each coordinate of the goal separately to the window places the local goal at a corner of the window whenever both coordinates lie outside it, pointing the search away from the true bearing. The goal is instead moved along the line from the robot's position $\mathbf{x}_r$ towards the goal $\mathbf{g}$:

$$\mathbf{g}_{\text{loc}} = \mathbf{x}_r + \kappa^* (\mathbf{g} - \mathbf{x}_r) , \quad (3.43)$$

where $\kappa^* \in [0, 1]$ is the largest value that keeps $\mathbf{g}_{\text{loc}}$ inside the window, less a margin. Two further cases need handling. A goal placed next to a tree, which is exactly where a survey places its goals, can fall inside the enlarged obstacle region, where no pose is admissible; the goal is then moved to the nearest free cell on the robot's side of the obstacle. And when no complete path exists within the window, the planner returns the part of a path that brings the robot closest to the goal, so that the robot moves and the map can reveal a way through.

All of this requires the robot's pose and the goal to be expressed in a common frame. In this work the pose is taken from the simulator's ground truth. On a real farm it would come from satellite positioning, from wheel odometry combined with inertial sensing, or from simultaneous localisation and mapping. Satellite reception and visual recognition of previously seen places are both degraded under a dense canopy, so the navigation results of this project hold on the condition that the robot knows where it is.

3.4.7 Range Sensing for Terrain Mapping: 3D LiDAR versus Stereo Depth

The released implementation of the navigation framework uses a 3D LiDAR [30]. A stereo depth camera costs a fraction as much, and this work compares the two as sources of point clouds for the elevation map. They differ in three ways that matter for mapping: what they can see, how densely they sample it, and how their error grows with range.

Field of view. A spinning multi-beam LiDAR sees all the way around the robot but has only a few vertical beams, each tracing a ring on the ground. The ground between rings is not sampled and is filled in by interpolation. A stereo camera sees a single wedge ahead. Its vertical field of view Φ_V follows from its horizontal field of view Φ_H and the image width n_w and height n_h in pixels:

$$\tan \frac{\Phi_V}{2} = \tan \frac{\Phi_H}{2} \cdot \frac{n_h}{n_w}, \quad (3.44)$$

The image aspect ratio as well as the lens sets how much ground is seen. A camera mounted at height h_s and tilted down by ψ sees the ground from a range of $h_s \psi$

$$\rho_{\text{near}} = \frac{h_s}{\tan(\psi + \Phi_V/2)} \quad (3.45)$$

outwards. If the upper edge of the field of view lies above the horizon, the far limit of the mapped ground is set by the maximum range of the depth measurement.

Sampling density. At range ρ , adjacent samples lie a distance

$$\Delta \ell(\rho) = \rho \Delta \Phi \quad (3.46)$$

apart across the direction of view, where $\Delta \Phi$ is the sensor's horizontal angular resolution. The stereo camera modelled here spreads its 424 pixel columns over 90° , and the LiDAR spreads 900 samples over 360° . Within its field of view the camera therefore samples the ground about twice as finely as the LiDAR.

Depth error. A stereo camera finds depth from the disparity Δu between its two images, $z = f_{px} b / \Delta u$, where f_{px} is the focal length in pixels and b the distance between the cameras. An error σ_u in disparity therefore gives a depth error

$$\sigma_z = \left| \frac{\partial z}{\partial (\Delta u)} \right| \sigma_u = \frac{z^2}{f_{px} b} \sigma_u = k_z z^2, \quad (3.47)$$

which grows with the square of range. For the camera modelled here, the manufacturer's figures of 0.8 % at 2 m and 1.6 % at 4 m both give $k_z = 0.004 \text{ m}^{-1}$ (Orbbec Gemini 330 Series product page (<https://www.orbbec.com/gemini-330-series/>) and Gemini 330 Series datasheet (<https://doc.orbbec.com/documentation/Orbbec%20Gemini%20330%20Series%20Docum> index

entation/Gemini%20330%20Series%20Datasheet%20%28Overall%29)). A LiDAR measures range by time of flight, and its range error does not grow with distance in this way. When n_c independent samples fall in one map cell, the error in the cell's height falls to

$$\sigma_{\text{cell}} = \frac{\sigma_z}{\sqrt{n_c}}. \quad (3.48)$$

Far from the camera, samples are both noisier and fewer per cell. Noise alone can then raise a cell above the protrusion threshold of Equation (3.33) and create an obstacle that does not exist.

Coverage. A trunk can be observed at a given instant only if its bearing lies within the sensor's horizontal field of view. The fraction of bearings observable at once is

$$\xi = \frac{\Phi_H}{360^\circ}, \quad (3.49)$$

which is 1 for the LiDAR and 0.25 for a camera with a 90° field of view. As the robot turns, trunks outside the wedge can enter it, so the fraction of trunks mapped over a whole run exceeds ξ ; but the map moves with the robot and discards what it leaves behind, so the camera's map remains less complete than the LiDAR's. What the substitution costs is coverage, not resolution. Whether that loss of coverage changes how well the robot completes its task is a separate question, examined in Chapter 5.

3.4.8 Measuring Navigation Performance and Testing for Equivalence

Measures. A trial sends the robot from a start pose to a goal, and the following measures are recorded.

- *Reaching the goal:* the robot comes within the goal tolerance before a time limit.
- *Time to goal:* the simulated time taken to reach the goal.
- *Path efficiency:* the ratio of the straight-line distance d_{line} from start to goal to the distance d_{driven} actually driven,

$$\eta = \frac{d_{\text{line}}}{d_{\text{driven}}}. \quad (3.50)$$

A higher value is better. Because a trial ends once the robot is inside the goal tolerance, a run that heads straight for the goal can score slightly above 1.

- *Clearance*: for trunk j with centre $\mathbf{o}_j$ and radius r_j , the distance from the robot's reference point $\mathbf{x}_r$ to the trunk surface is

$$d_j = \|\mathbf{x}_r - \mathbf{o}_j\| - r_j. \quad (3.51)$$

The minimum of d_j over all trunks and the whole trial is recorded, together with the fraction of the trial spent with a trunk surface inside the obstacle enlargement radius of Section 3.4.3 (the near-miss fraction). Distances are measured against the true trunk positions of the simulated world, never against the robot's map.

- *Contact*: the number of separate occasions on which a trunk surface comes within the radius enclosing the chassis. A trial is accepted only if the robot reaches the goal with no contact.
- *Command dither*: the number of changes of sign per second of the commanded turn rate, which measures the alternating commands described in Section 3.4.5.
- *Detection recall*: the fraction of trunks that entered the map window and were marked impassable,

$$\text{recall} = \frac{\text{trunks marked impassable}}{\text{trunks that entered the map window}}. \quad (3.52)$$

A trunk that never entered the window was never an opportunity for detection, so it is excluded from the denominator.

Paired trials. Two configurations are compared on the same set of goals, from the same starting poses and with the same random seed for the controller. Each trial in one configuration then has a partner in the other, and the comparison is made on the differences within each pair. This removes the differences between goals from the comparison. Because distributions of time and distance from a sampling-based controller need not be symmetric, differences are summarised by their median, and tested with the Wilcoxon signed-rank test, which does not assume a normal distribution [46].

Testing for equivalence. The sensor comparison of Section 3.4.7 asks whether replacing the LiDAR with a camera makes no practical difference to task completion. Failing to find a significant difference does not show that there is none, because a small experiment may simply lack the power to detect it. The two one-sided tests (TOST) procedure addresses

this directly [47]. An equivalence margin Δ_{eq} is fixed in advance, and two null hypotheses are tested about the true difference δ :

$$H_{01} : \delta \leq -\Delta_{\text{eq}}, \quad H_{02} : \delta \geq +\Delta_{\text{eq}}. \quad (3.53)$$

If both are rejected, the difference lies within $\pm\Delta_{\text{eq}}$ and the configurations are equivalent within that margin. If either is not rejected, the result supports only the weaker statement that no difference was detected. Because the goals differ in length, a margin in seconds would be strict for a long goal and loose for a short one. Time and path efficiency are therefore compared through the relative difference within each pair i , where X_i is the value of the measure,

$$\delta_i = \frac{X_i^{\text{camera}} - X_i^{\text{LiDAR}}}{X_i^{\text{LiDAR}}}, \quad (3.54)$$

and each one-sided hypothesis is tested with a one-sided Wilcoxon signed-rank test on $\delta_i + \Delta_{\text{eq}}$ and $\delta_i - \Delta_{\text{eq}}$. The success rate, a proportion, is compared as a difference in percentage points. The margin states how large a difference would change how the robot is used on a farm, so it is set from the task before any trial is run and is not widened after the results are seen. Equivalence is claimed only for task completion. Clearance is expected to worsen when the map is less complete, so it is tested for a difference, not for equivalence.

3.5 Survey Workflow Design

The navigation system of Section 3.4 moves the robot to a pose it is given. The robot exists to collect data, however, so a further layer is needed that turns a researcher's request into navigation goals and into the data recorded at each of them. This section first sets out what a tree-image survey requires, and then derives where the robot must stop and how its camera must be aimed so that every tree is recorded completely and in the same way. Figure 3.3 shows how the parts of the survey fit together.

3.5.1 Requirements of a Tree-Image Survey Mission

The robot is intended for agricultural researchers, who need data from local farms to develop tools such as disease monitoring and crop assessment. Farmers benefit from those tools rather than operating the robot. Collecting such data by hand is slow and hard to repeat: the route, the distance from which each tree is photographed, and the set of trees

visited can all change from one visit to the next. Drones cover large areas quickly but see the crop mainly from above the canopy, whereas a ground robot can stop beside a tree and record it at close range. The robot is designed as a modular platform that can carry whatever sensor a study needs. The form of data collection implemented in this project is the tree image.

Seven requirements follow for a survey mission, and each shapes the design, as summarised in Table 3.5.

Table 3.5: Requirements of a tree-image survey and their consequences for the design

Requirement	Consequence for the design
The researcher chooses the trees, and can give the list to a robot that is already running, follow its progress and cancel it	The mission is sent as a long-running request with progress feedback and cancellation, not fixed in a file at start-up
The robot visits every tree without being driven	Each tree is turned into a navigation goal for the system of Section 3.4
The record of each tree covers every direction around it, with the tree itself in one known image	A fixed number of images at equal heading steps, the first facing the trunk
Images are taken from the same distance and headings at every tree and on every visit	A standoff distance measured at the camera, and headings measured from the direction of the trunk
Every image can be traced to its tree, its time and the place it was taken	Each image is stored with the tree identifier, the time, and the poses of the camera and the robot, written as the image is taken
A tree that cannot be reached or photographed does not end the survey	The tree is skipped, the reason is recorded, and the mission continues
The survey does not load or interfere with navigation	A separate camera that renders only when triggered, and a rule deciding whether navigation or capture drives the robot

Tree positions instead of tree detection. The operator supplies the position of each tree, so the heading that faces the trunk is computed from that position rather than detected in an image. If the supplied position is in error by $e_{\perp}$ across the line of sight, the trunk stays within a horizontal field of view Φ_H seen from a camera range ρ_c provided

$$|e_{\perp}| \leq \rho_c \tan \frac{\Phi_H}{2}. \quad (3.55)$$

With a 90° field of view, the permitted error equals the range itself, 2.4 m at the design standoff given below. Tree positions taken from the simulated world are exact, and a careful survey of a farm would be well inside this bound, so no trunk detector is needed. Positions from a handheld satellite receiver, whose errors can reach several metres, would call for one. Tree positions are expressed in the same frame as the robot's pose (Section 3.4.6); fields for latitude and longitude are reserved in the record but not yet filled.

Selecting trees. Typing coordinates for many trees is slow and error-prone. The trees known to the system are therefore shown in the ROS visualisation tool, RViz, where the operator clicks near each tree in turn. Each click is moved to the nearest known tree, and the order of the clicks sets the order of the visits, since the mission visits trees strictly in the order given and does not optimise the route.

3.5.2 Station Geometry and Camera Coverage

A tree cannot itself be a navigation goal: its trunk is an obstacle, enlarged by the robot's footprint (Section 3.4.3). Each tree is therefore photographed from a station on a circle around its trunk. The design of the station answers four questions: which camera takes the images, how many images are taken and in which directions, how far the camera must be from the trunk, and where on the circle the robot should stop. Figure 3.4 shows the geometry.

The survey camera. The depth camera used for navigation produces images of too low a resolution for a record of the tree, and it is tilted down towards the ground (Section 3.4.7), which puts most of each image on the soil. Raising its resolution would also raise the cost of the point clouds it produces for navigation. The images are therefore taken by a separate camera of higher resolution, mounted on its own and tilted up towards the tree. It renders only when triggered, so it costs nothing while the robot drives.

Number and direction of images. A camera with horizontal field of view Φ_H covers all directions around a station with

$$n_{\text{shot}} = \frac{360^\circ}{\Phi_H} \quad (3.56)$$

images taken at heading steps of Φ_H . The survey camera has a 90° field of view, so four images at 90° steps cover the full circle with neither gap nor overlap. The images are kept as separate records rather than stitched into a panorama, so no overlap is needed. The first image faces the trunk, and the other three record the tree's surroundings in a fixed order.

Camera range and vertical framing. The camera is mounted at height h_{cam} , with its optical axis tilted up by ψ_s . With a vertical field of view Φ_V , the image spans from $\Phi_V/2 - \psi_s$ below the horizontal to $\Phi_V/2 + \psi_s$ above it. The part of a tree from the ground to a height H_{tree} fits in the frame only if the camera is at a range

$$\rho_c \geq \max \left(\frac{h_{\text{cam}}}{\tan(\Phi_V/2 - \psi_s)}, \frac{H_{\text{tree}} - h_{\text{cam}}}{\tan(\Phi_V/2 + \psi_s)} \right) \quad (3.57)$$

or more. The constraint is one-sided: closer than this, the base or the top of the frame cuts off the tree; farther away, the tree only fills less of the frame. For the survey camera, with a 90° horizontal field of view and a 1280×800 pixel image, Equation (3.44) gives $\Phi_V = 64.0^\circ$. Mounted 0.607 m above the ground and tilted up by 16° , the camera must be at least 2.15 m from the trunk to frame the lowest 3 m of a tree, a bound set by the top of the frame. That height takes in the trunk and the main branches on which cocoa bears its pods (Section 3.2.5), rather than the whole of a tree that may grow to 8 m. The design standoff of 2.4 m leaves a margin of about 0.25 m. The image resolution was chosen to show that the robot can collect one type of data, tree images, rather than to support a particular analysis. At 2.4 m each pod occupies too few pixels for close inspection of its surface; a study that needs such detail would fit a camera suited to it, as the modular design of the platform allows.

Camera offset and station radius. The camera is not at the robot's reference point, the point the navigation system brings to the goal. If the camera sits a distance ℓ_f ahead of the reference point along its optical axis and ℓ_l to the side, then, with the camera aimed at the trunk from range ρ_c , the reference point lies at a distance

$$\rho_{\text{st}} = \sqrt{(\rho_c + \ell_f)^2 + \ell_l^2} \quad (3.58)$$

from the trunk. The station circle therefore has radius ρ_{st} , not ρ_c . Placing the reference point itself at the standoff would bring the camera about 0.33 m closer to the trunk on this robot, 14 % of the design standoff, and inside the bound of Equation (3.57). The offsets are read from the robot's description rather than written into the survey software, so that a change to the camera mount cannot silently change the range.

The navigation controller stops within its goal tolerance of the station, a margin larger than the one Equation (3.57) leaves. Before the first image, the range is therefore corrected by driving the robot straight along the camera's optical axis, which leaves the camera aimed at the trunk, until the measured range reaches the standoff.

Choosing the station. Eight candidate stations are spaced evenly on the circle. The first lies on the side of the tree facing the robot's approach, and the others alternate either side of it, so that the robot drives no farther than it must and stops on ground it is likely to have

observed. The first candidate with no obstacle within a clearance radius is chosen. Ground not yet observed counts as free, because at the start of a mission most trees lie outside the robot's map (Section 3.4.6); the choice is checked again while the robot approaches and the map fills in. If every candidate is blocked, the tree is skipped.

Aiming and turning on the station. Four considerations follow from the behaviour of a skid-steer robot turning in place (Sections 3.2.2 and 3.3.1).

- *Drift during turns.* A skid-steer robot does not turn about a fixed point, because its wheels scrub. In an early test the robot moved up to 0.75 m during a sequence of four turns, which at a range of 2.2 m would misdirect the camera by about 19° . The heading to the trunk is therefore computed from the camera's current pose, read from the robot's transform tree, immediately before the first image, rather than from the pose at arrival. Each image is stored with its own pose instead of inheriting the station's.
- *Commanding the turn.* The robot achieves only part of the turn rate it is commanded (Section 3.3.1), and a small command leaves the wheels held by static friction. The turn is therefore commanded well above the rate wanted, with a minimum command large enough to break away.
- *Setting the headings.* Each heading target is a whole number of 90° steps from the aim of the first image, not a step from the heading actually reached after the previous turn. The error in any image is then the error of a single turn rather than the sum of the turns before it.
- *Stillness at the shutter.* An image is taken only after the robot's measured linear and angular speeds have stayed below small thresholds for a short time, so that the image is not blurred and its recorded pose is the pose at which it was taken.

Sharing the drive. Navigation and capture both command the robot's motion, and the navigation controller continues to send zero-velocity commands after it arrives. Commands are therefore passed through a priority selector: while the capture routine runs, its commands take precedence, and when it finishes it falls silent so that navigation resumes. The mission learns of arrival from the controller itself, which publishes a signal when it reaches its goal, rather than repeating the controller's arrival test with a second copy of its tolerance.

The record. Each image is written with a companion file that holds the tree identifier, the heading index, the time in both universal and simulation time, the full pose of the camera

and the pose of the robot, the commanded and achieved headings, the achieved range, the camera's calibration, and the source of the pose, which is the simulator's ground truth in this work. A manifest for the whole mission gains one row as each image is written and is saved to disk at once, so a mission that fails part-way keeps every image already taken. Each mission writes to its own time-stamped directory, and no earlier record is overwritten.

3.6 Summary

This chapter set out the theory and the design considerations on which the rest of the report builds. It started from one fact about the robot, that it can only turn by making its wheels slip, and followed that fact through the design of the platform, the model of its motion and the navigation system that depends on the model.

The platform was designed as a four-wheel, two-motor skid-steer robot sized to pass between cocoa trees, and its drivetrain was represented as a torque-limited velocity-tracking loop so that its wheels slip as a real robot's do. Simulation was adopted as the test instrument for its repeatability and ground truth, with the limitation that its results describe the simulated robot. Representing the robot and the farm showed that details invisible in the rendered scene, such as the collision shape of a wheel, the slope of the terrain and the split between a tree's visual and collision geometry, decide how the robot turns and what its sensors see.

The chapter then showed that the no-slip kinematic model of the chosen navigation framework cannot describe skid-steer turning, and adopted in its place the friction-based kinematic model of Rabiee and Biswas, which predicts slip from the forces at the wheels and the commanded accelerations. It set out a nested least-squares procedure for fitting the model's parameters and a per-channel velocity error for judging its accuracy against the no-slip model.

Hybrid A*-guided MPPI was selected as the navigation framework because it uses terrain costs in both planning and control and places no restriction on the motion model, so the friction-based model can replace the no-slip model in the controller's predictions without changing the optimiser. The traversability estimate, the planner and the controller's guidance were each adapted to a skid-steer robot among trees, and the stereo depth camera was shown to differ from the LiDAR in coverage rather than in resolution. Navigation performance is judged through paired trials, with tests of equivalence where the claim is

that a change makes no practical difference. Section 3.5 then turned to the tree-image survey that the navigation system exists to serve: it set the requirements of a survey mission and derived where the robot must stop and how its camera must be aimed, so that every tree is recorded completely and in the same way.

Chapter 4 describes how these designs were implemented and the experiments used to test them.

CHAPTER 4 - METHODOLOGY

4.1 Introduction

Chapter 3 set out the theory behind each part of the system and the considerations that shaped each design choice. It established that a four-wheel skid-steer robot turns only by scrubbing its wheels across the ground, so a planner and controller built on no-slip kinematics need a motion model that describes slip. It selected the friction-based kinematic model of Rabiee and Biswas as that model and Hybrid A*-guided MPPI as the navigation framework that can use it. It showed that a stereo depth camera gives up bearing coverage, relative to a 3D LiDAR, in exchange for lower cost, and it derived the geometry of a tree-image survey.

This chapter describes how the system was built and how it was evaluated, in enough detail for the work to be repeated: the values used, how each was obtained, and the assumptions behind them. Where the implementation departs from the source papers, the departure is stated together with the reason for it.

The chapter follows the same four-part order as Chapter 3. Section 4.2 describes the robot, from its design to the simulation model used in every test, and the farm worlds it was tested in. Section 4.3 uses that model to collect driving data, identify the friction-based model, and validate the fit before it was used anywhere else. Section 4.4 describes the autonomy stack. Because the source framework could not be adopted as released, it was re-implemented in stages: a Python reference, a port to a two-wheel robot, and an adaptation to the four-wheel skid-steer robot with the friction model inside its controller, after which the 3D LiDAR was replaced by a stereo depth camera. The section ends with the protocol used to test navigation in closed loop. Section 4.5 describes the survey workflow built on top of that stack, and Section 4.6 summarises the methods whose results Chapter 5 reports.

4.2 Platform and Environment

Every result in Chapter 5 was obtained on a simulated robot in a simulated farm, so the fidelity of both bounds what those results can mean. This section describes how the robot was designed, how the design was turned into a simulation model with correct mass properties and contact geometry, how its drivetrain was represented, and how the farm worlds were built and used.

4.2.1 Robot Design: from Requirements to the CAD Model

The robot was designed in Fusion 360, combining parts from the GrabCAD community with components modelled for the project. The design followed the requirements of Section 3.2.1: a four-wheel, two-motor skid-steer platform with high ground clearance, narrow enough to pass between cocoa trees, with a payload bay that can carry different sensors. Its chassis is a deck carried on four corner blocks that descend beside the wheels to the axle mounts, with an $800 \times 900 \times 250$ mm payload bay above the deck and a camera body on a mast at the front (Figure 4.1). Measured against the ground plane, the underside of the deck stands 297.8 mm above the ground across the full 800×937 mm footprint, and the lowest parts of the robot are the four corner plates, each 40×142 mm, at 95.7 mm. Anything low enough to pass under the deck but not under those plates lies in the wheels' own track, so the wheels meet it first.

The drive motors were sized for straight-line driving up a slope. For a robot of mass m on ground with rolling-resistance coefficient μ_r , climbing a design slope ζ_d , with a 30 % margin for debris, acceleration and uneven ground, the total tractive force required is

$$F_{\text{req}} = 1.3 mg (\mu_r + \sin \zeta_d) , \quad (4.1)$$

and, since each motor drives one side through two wheels of radius r , the torque required from each motor is

$$\tau_{\text{motor}} = \frac{F_{\text{req}}}{2} r . \quad (4.2)$$

With the design estimates $m = 20$ kg, $\mu_r = 0.15$ for soft, moist soil, $\zeta_d = 15^\circ$ and $r = 0.10$ m, Equation (4.1) gives 104.3 N and Equation (4.2) gives 5.2 N·m per motor. At the design speed of 2 m/s, the corresponding motor power, with a further 30 % allowance for gearbox losses, is 136 W.

Table 4.1 compares the design targets with the values of the simulation model used in every test. Most targets carried through. The track width did not: the chassis designed around the payload bay placed the wheels 0.9225 m apart, wider than the 60–80 cm target, while keeping the overall width of 1.047 m well inside the 2.0 m limit of Equation (3.1). The torque limit of the simulated drivetrain was set from turning tests on the simulated robot rather than from Equation (4.2) (Section 4.2.3), and the pneumatic tyre of the design

is represented by a rigid collision mesh, since the physics engine does not model tyre deformation (Section 3.2.4).

Table 4.1: Design targets and the values of the simulation model

Quantity	Design target	Simulation model
Drive topology	Two motors, one per side	Two motors, one per side, with the wheels on each side coupled (Section 4.2.3)
Mass	20 kg (estimate)	19.384 kg
Ground clearance	25 cm	297.8 mm under the deck; 95.7 mm under the four corner axle-mount plates beside the wheels
Wheel diameter	20 cm	218.6 mm ($r = 0.1093$ m)
Wheel width	8–10 cm	Tyre 120.5 mm; contact surface of the collision mesh 70.7 mm
Track width	60–80 cm	0.9225 m
Wheelbase	70–90 cm	0.8911 m
Overall width	≤ 2.0 m (Equation 3.1)	1.047 m
Tyre	Pneumatic, ribbed tread	Rigid collision mesh
Drive torque	≥ 5.2 N·m per motor, continuous	Limited at 32 N·m per side

4.2.2 From CAD to Simulation Model

Export and structural repair. The design was exported to URDF with the Fusion-to-URDF exporter. The exported package built but did not work. The exporter wrote an inertia on the root link, which the kinematics library used by ROS 2 cannot handle, and it baked the assembly's world coordinates into the mesh vertices, so the joint origins and the mesh origins did not match. The joint origins were rebuilt from the measured bounding-box centres of the wheel meshes, with each visual and collision origin set to the negative of its mesh centre, which attached the wheels and made them rotate about their own axes; the wheel axis is the link's x axis. A massless root link, `base_footprint`, was added at ground level above `base_link`. Because `base_link` is rotated by 90° about the vertical axis relative to `base_footprint`, the robot's forward direction is the $-y$ axis of `base_link`, and every navigation component refers to `base_footprint` instead. A validation script was written to catch the same export faults mechanically in future exports.

Masses. The exporter computed each link's mass from the volume of its solids at material density, which modelled the robot as a 1,646 kg machine: 221.83 kg for `base_link` and 1,413.0 kg for the payload bay, whose solid box stood in for a hollow volume. The masses were set from the design instead: 7.0 kg for `base_link`, which carries the frame, drivetrain and battery as one body, and 4.0 kg for the payload bay. Each wheel's mass was

reduced from 2.82 kg to 2.0 kg, and its inertia tensor was scaled by the same ratio, $f = 2.0/2.82 = 0.709$, so that the distribution of mass within the wheel was preserved. The wheel's moment of inertia about its axle is then $0.595 mr^2$, between the disc and ring limits of Equation (3.9), as expected for a tyre on a hub. Table 4.2 gives the resulting distribution.

Table 4.2: Mass distribution of the simulation model

Link	Mass (kg)	Share of total
base_link (frame, drivetrain, battery)	7.000	36.1 %
Payload bay	4.000	20.6 %
Wheels, four at 2.000 kg	8.000	41.3 %
Camera body	0.384	2.0 %
Total	19.384	

Centres of mass. The exporter also placed each wheel's centre of mass 0.221 m from its axle. By Equation (3.10), that offset raised the wheel's moment of inertia about the axle from 0.0142 to 0.112 kg·m², a factor of 7.9, so the centre of mass was moved onto the axle. The centre of mass of `base_link` was placed at the centre of the chassis's bounding box rather than at the centroid of its solid volume, because the upper part of that volume is the hollow payload bay, which is a separate link, and the real mass of the frame sits low. The payload bay's centre of mass was placed at the centre of its box. The inertia tensors themselves were checked against equivalent boxes and left unchanged.

Constants for the friction model. The friction-based model of Section 3.3 needs the robot's mass, yaw inertia, track width, wheelbase and the position of its centre of gravity. The yaw moment of inertia about the composite centre of gravity was computed by summing each link's own moment of inertia and its parallel-axis term,

$$I_z = \sum_k [I_{zz,k} + m_k ((x_k - x_{CG})^2 + (y_k - y_{CG})^2)] , \quad (4.3)$$

where m_k , $I_{zz,k}$ and (x_k, y_k) are the mass, vertical moment of inertia and centre-of-mass position of link k . The track width was taken as the distance between the centres of the two wheel contact surfaces,

$$B = d_{in} + d_{wh} , \quad (4.4)$$

where $d_{in} = 802.916$ mm is the gap between the inner faces of the left and right wheels and $d_{wh} = 119.634$ mm is the wheel width, both measured in the CAD model. An earlier

measurement had added only half a wheel width, which understated the track width by about 60 mm. Equation (4.4) gives 922.55 mm, which agrees with the 923.4 mm spanned by the wheel meshes to within 1 mm. Table 4.3 lists the constants used.

Table 4.3: Robot constants used by the friction-based model

Symbol	Quantity	Value	How obtained
m	Mass	19.384 kg	Sum of link masses (Table 4.2)
I_z	Yaw moment of inertia about the centre of gravity	4.776 kg·m ²	Equation (4.3)
r	Wheel radius	0.1093 m	Wheel mesh, outer diameter 218.69 mm
B	Track width	0.9225 m	Equation (4.4)
L	Wheelbase	0.8911 m	Wheel joint origins
x_{CG}, y_{CG}	Centre of gravity relative to the geometric centre	−0.0085 m, 0 m	Composite of link centres of mass
N	Static load per wheel	47.54 N	$mg/4$, Equation (3.3)

Wheel collision geometry. The wheels first carried cylinder collision shapes. Once the wheels' centres of mass had been moved onto their axles, the robot stopped turning under velocity control, and a series of single-variable tests identified the cylinder's line contact as the cause (Chapter 5). The cylinders were replaced by a low-polygon mesh shaped like the tyre, of 1,600 triangles and 121.4 mm wide, centred on the axle.

The tyre mesh was later replaced by a crowned mesh, whose rounded tread touches the ground over a narrower band (Section 3.2.4). It was built from the STEP model of a 10-inch wheel with a crowned tread. The STEP model was converted to a surface mesh with gmsh, the tyre was separated from the rim and hub, recentred on its axle, and scaled uniformly by 0.8625 so that its outer radius, 0.1114 m, matched that of the tyre mesh to within 1 mm. Matching the radius kept the ride height and the relation between wheel speed and ground speed unchanged, so the tread profile was the only variable that changed. The mesh was then reduced to 1,800 triangles. Its contact band is 26.9 mm wide, against 89.5 mm for the tyre mesh, and one mesh serves all four wheels. Friction coefficients set on the wheels were left in the model but have no effect under DART (Section 3.2.3); the wheel–ground friction is set by the ground surface of each world.

4.2.3 Drivetrain and Actuation

Command interface. The four wheel joints are continuous joints driven through `ros2_control` in effort mode: the controller commands a torque, and each joint reports

its position and velocity. Each joint carries a viscous damping of 0.3 N·m·s/rad and no joint friction.

Per-side velocity-tracking controller. A custom controller node converts the robot's velocity command (v, ω) into a torque for each side, running at 100 Hz on the simulation clock. It first converts the command into target wheel speeds ω_l^* and ω_r^* with Equation (3.40). For each side $s \in \{l, r\}$, it compares the target with the average measured speed $\bar{\omega}_s$ of the side's two wheels, and computes a side torque

$$\tau_s = \text{sat}_{\pm\tau_{\max}} \left(K_P e_s + K_I \int e_s dt + \tau_{\text{brk}} \Gamma_s \right), \quad e_s = \omega_s^* - \bar{\omega}_s, \quad (4.5)$$

which is divided equally between the side's two wheels. The integral in Equation (4.5) stops accumulating while the output is saturated, so that it does not wind up. Its last term is a breakaway torque that helps the robot start a turn against the resistance of scrubbing (Section 3.2.2). It acts in the direction of the side's share of a demanded turn and is scaled by the fraction of that share not yet achieved,

$$\Gamma_s = \text{clamp}_{[0, 1]} \left(1 - \frac{\bar{\omega}_s - \bar{\omega}}{\omega_s^* - \bar{\omega}^*} \right), \quad (4.6)$$

where $\bar{\omega}^*$ and $\bar{\omega}$ are the target and measured averages over both sides, so that the breakaway torque is full when the robot has not begun to turn and fades to zero as the turn is reached. The breakaway torque of Equation (4.6) is applied only when the side's share of the demanded turn exceeds a dead band. Commands that stop arriving are replaced by a zero command after a timeout. The parameters are given in Table 4.4.

Side coupling. The two wheels on each side must turn together, as a chain or shaft would force them to, but DART does not enforce joint constraints (Section 3.2.3). Three alternatives were tried and rejected. URDF mimic joints and gearbox joints had no effect, because DART ignores them. Coupling the wheels' velocities in the controller reduced the mismatch between them by only 13 %. Moving to the Bullet-Featherstone engine, which does enforce the constraint, failed because that engine cannot load heightmap terrain. The coupling that was kept acts on the phase of each wheel rather than its speed, because a chain holds the wheels' angles together as well as their speeds. A plugin written in C++ for the simulator accumulates, for each wheel on a side, the angle by which it has drifted from the side's average, i

$$\Psi_i \leftarrow \text{clamp}(\Psi_i + (\omega_i - \bar{\omega}_s) \Delta t) , \quad \tau_{c,i} = \text{sat}_{\pm\tau_{c,\max}}(-k_{\text{phase}} \Psi_i - k_{\text{dphase}} (\omega_i - \bar{\omega}_s)) \quad (4.7)$$

and adds the coupling torque $\tau_{c,i}$ of Equation (4.7) to the torque commanded for that wheel at every physics step. The clamp on the accumulated angle Ψ_i holds the spring term within the torque limit. Because the two wheels on a side deviate from their average by equal and opposite amounts, their coupling torques are equal and opposite, and the coupling adds no net torque to the side. The same coupling applied at the controller's 100 Hz made the mismatch worse, while applied at the physics rate of 1 kHz it held the wheels together, which is why it runs in the simulator rather than in the controller.

Table 4.4: Drivetrain parameters

Parameter	Value
Torque limit per side, $\tau_{\max}$	32.0 N·m (16.0 N·m per wheel)
Proportional gain, K_p	4.0 N·m·s/rad
Integral gain, K_I	8.0 N·m/rad
Breakaway torque, τ_{brk}	8.0 N·m
Breakaway dead band	0.3 rad/s of wheel speed
Controller rate	100 Hz
Coupling stiffness, k_{phase}	80.0 N·m/rad
Coupling damping, k_{dphase}	5.0 N·m·s/rad
Coupling torque limit, $\tau_{c,\max}$	16.0 N·m
Coupling rate	1 kHz (every physics step)
Joint damping	0.3 N·m·s/rad

Torque limit. The torque limit was first set at 16 N·m per side. It was raised to 32 N·m per side on 2026-07-26, at the same time as the crowned wheel mesh was adopted, after in-place turning tests at 16, 32 and 64 N·m per side (Chapter 5). The friction-based model was fitted to data collected at this limit and with this mesh. Because the model's input is the commanded wheel velocity, changing either would change the fitted parameters (Section 3.3.2).

4.2.4 Building the Cocoa-Farm Worlds

Terrain. A generator script produced a 513×513 pixel heightmap covering a 50×50 m plot. The terrain is a planar slope of 4 % with smooth fractal noise of five octaves added on top, scaled to 0.12 m of micro-relief. The noise is generated from a fixed seed, so the terrain can be regenerated exactly. The resulting relief is 2.178 m from the lowest to the highest point, most of it the slope. For the de-sloped terrain of Section 3.2.5, a second

script fitted a plane to the elevation data by least squares and subtracted it. The plane removed had a grade of 2.26° , and the residual relief is 0.2556 m, with a standard deviation of 0.044 m. The de-sloped heightmap is written as a new image, and its worlds scale it to that residual relief, so the local bumps are reproduced at their original size on level ground.

Tree models. The trees were built in Blender from a single cocoa-tree mesh purchased from Sketchfab, which was separated into parts that can be addressed individually, scaled to life size, and given a controllable trunk thickness and fork height, from 1.2 to 2.5 m. Pods were placed on the trunk and main branches with a randomly assigned ripeness colour. Each tree was exported as one visual mesh. Its collision geometry was built in two parts:

- **Trunk.** The trunk collision is a tube built from rings. The fork of the trunk is found by walking up the trunk until the radius of the geometry exceeds 1.6 times the radius at its base. Below the fork, the trunk is divided into eight horizontal bands; in each band, geometry far from the band's median distance is rejected, the ring is centred on the band's own centroid, and its radius is the median distance of the remaining geometry from that centroid. The median rather than the maximum keeps bumps on the bark from inflating the tube.
- **Pods.** Each pod within reach of the sensors is represented by an ellipsoid.

Every tree is declared static.

Placement. Tree positions were generated with Poisson-disk sampling, with a minimum spacing of 3.0 m and no tree within 2.0 m of the plot edge. Candidate positions on exposed laterite were rejected. A smooth noise field with a wavelength of 8 m then removed 12 % of the remaining positions, to create the irregular gaps of a smallholder farm, and the 100 positions with the highest noise values were kept, each assigned a different tree model and a random heading. All steps use a fixed seed. Each tree's height was set from the terrain surface at its position, lifted by that tree model's own offset between its origin and its lowest point, so that every trunk base sits on the ground. The resulting nearest-neighbour spacing is 3.010 m at minimum, 3.424 m at the median and 6.425 m at maximum, and the density over the 50×50 m plot is 400 trees per hectare.

The de-sloped farm world. A further script built the world `cocoa_farm_heightmap_deslope` from the de-sloped leaf-litter terrain, with

ground friction coefficients of 0.60, and the same 100-tree layout: the same position, heading and model for every tree, with only the height recomputed on the de-sloped surface. The script's check reproduced the heights of the original sloped world to within 0.5 mm. The boundary walls of the original terrain were removed, so the trees are the only obstacles, as in Figure 4.2(a). Laterite patches do not appear as surfaces in the de-sloped worlds; the laterite map affected only where the trees could be placed.

4.2.5 Evaluation Worlds and Simulation Settings

Two worlds, both on the same de-sloped terrain, were used for the work reported in Chapter 5 (Table 4.5).

Table 4.5: Worlds used in the reported work

World	Contents	Used for
leaf_litter_heightmap_deslope	De-sloped terrain, ground friction 0.60, no trees	Friction-model datasets (Section 4.3); in-place turning tests
cocoa_farm_heightmap_deslope	The same terrain with the 100 complete tree models, canopy included	Development of the perception chain; survey missions (Section 4.5); closed-loop navigation evaluation (Section 4.4.7)

The evaluation condition. The closed-loop trials of Section 4.4.7 were run in the full cocoa-farm world `cocoa_farm_heightmap_deslope`. The sensors therefore saw the complete tree models, including canopy, branches, trunks and pods, while physical collision remained limited to the trunk and pod geometry described in Section 3.2.5. The tree records used for goals, clearance and survey selection use true trunk positions rather than tree-model origins: across the 100 tree models, the model origin sits a median of 0.50 m from the trunk. The reason for keeping the canopy in the evaluation is that the sensors see the canopy (Section 3.2.5), and returns from it occasionally survive the canopy filter of Section 3.4.2 and land in the map as obstacles standing on drivable ground. During the first sensor-comparison trials, 30 of the 78 cells marked impassable in the live map lay more than 1.5 m from any trunk, and one

goal was blocked by such cells although every goal lay at least 1.75 m from a real trunk.

Simulation settings. All worlds use DART, Gazebo's default physics engine, with a step of 1 ms and a target real-time factor of 1.0. With the full navigation stack running, the achieved real-time factor was about 0.2. Every node runs on the simulation clock, and every timer in the navigation and drive controllers is created on that clock rather than on the computer's clock. The distinction matters: at a real-time factor of about 0.2, a 3 Hz planner timed by the computer's clock replanned at about 14 Hz of simulated time, so rates set on the computer's clock are not the rates the robot experiences. All rates in this report are rates in simulated time. The robot's ground-truth pose is published by the simulator at 100 Hz and supplies the robot's pose to the navigation stack (Section 3.4.6). The robot is spawned 0.5 m above the terrain origin of the de-sloped worlds and settles onto the ground before each trial.

4.3 Friction Model Development

The friction-based model of Section 3.3 has five parameters that describe one combination of robot and ground. This section describes how the driving data were collected, how they were turned into a table the fit could use, how the parameters were fitted and chosen, how the fit was validated before it was used anywhere, and how the model was made fast enough to run inside a controller.

4.3.1 Data Collection Protocol

Plant and world. All data were collected on the robot described in Section 4.2, driven in effort mode through the per-side controller at a torque limit of 32 N·m per side and with the crowned wheel collision mesh. The world was `leaf_litter_heightmap_deslope`: the de-sloped terrain with leaf-litter ground friction of 0.60 and no trees, so that the robot could drive freely and neither slope nor obstacles entered the measurement. Because the model's input is the commanded wheel velocity, the drivetrain configuration is part of what is fitted (Section 3.3.2), so data recorded before the torque limit and the wheel mesh were changed were not reused.

Commands. A generator node published body velocity commands at 45 Hz directly to the controller's command input, bypassing the teleoperation chain, whose relay rescales its input and would have distorted the profile. For each command the generator also recorded

the corresponding wheel velocities and their rates of change, which are the model's inputs, in a comma-separated log.

Two datasets were collected, following the categories of Rabiee and Biswas. The **training** dataset mixes accelerations deliberately, because acceleration-induced slip is what the friction-based model adds over models that take only wheel velocities (Section 3.3.1). Its commands are drawn from the palette of Table 4.6, each segment being a target speed, a target turn rate, a ramp time and a hold time drawn from the ranges shown. The **constant-velocity** dataset does the opposite: each segment holds one pair of linear and angular velocities, ramping to it over 0.6 s and then holding it for 8.0 s, which is longer than the 6 s horizon the source paper predicts over, so that each segment yields a window in which the commanded acceleration is zero. Its segments sweep the turning radius over 0.5, 1, 2, 4, 6 and 8 m at speeds of 0.3, 0.5, 0.7 and 1.0 m/s, together with straight runs and turns on the spot at 0.4, 0.8 and 1.2 rad/s. A flag marks the rows inside the hold, so the ramps can be excluded.

Table 4.6: Command palette of the training dataset

Manoeuvre	Share	Speed (m/s)	Turn rate (rad/s)
Straight cruise	20 %	0.9 to 2.0	−0.15 to 0.15
Wide turn at low speed	16 %	0.3 to 0.8	0.20 to 0.6
Sharp turn at high speed	16 %	1.2 to 2.0	1.0 to 3.0
Gentle curve	12 %	0.6 to 1.4	0.3 to 0.9
Turn on the spot	10 %	−0.1 to 0.3	1.0 to 3.0
Hard stop	10 %	0.0 to 0.15	−0.1 to 0.1
Reverse	8 %	−0.8 to −0.3	−0.4 to 0.4
Random combination	8 %	−2.0 to 2.0	−3.0 to 3.0

The commanded turn rate reaches 3.0 rad/s, which is well above the rate the robot achieves, so the data cover the whole range the controller of Section 3.4 can ask for. Both datasets used the same random seed, and the generator stops when the robot has actually travelled a set distance, measured from its ground-truth position rather than from the commands, so that slip does not shorten the run. The training run was set to 600 m and the constant-velocity run to 300 m.

Boundaries and stalls. The terrain is 50 m square and unfenced. The generator therefore steers the robot back when it comes within 24 m of the centre in either direction, and those recovery segments are kept, being ordinary driving. What must not be kept is a stall: the robot commanded to move while pinned against something, which reads to the model as total slip. The generator flags a stall whenever the commanded speed exceeds 0.3 m/s

while the measured speed stays below 0.08 m/s, and publishes the flag so that it is recorded with the data.

Recording. Each run was recorded as a bag holding the ground-truth odometry, the joint states, the commands, the wheel-command inputs and the stall flag. A first attempt on 25 July was discarded: the recorder was left running for 7,860 s of simulated time after a 695 s run and was never closed properly, so no index was written. The pair used for the final model was re-collected on 26 July, 727.4 s for the training run and 545.3 s for the constant-velocity run.

4.3.2 Dataset Processing Pipeline

Each bag passed through three stages, and every stage was recorded in a log kept inside that dataset's own folder, with the command run, the files in and out, and the counts.

Stage 1, deriving the measured motion. The simulator's odometry publisher computes the yaw rate by differencing successive orientations without making the sign of the quaternion consistent. The physics engine occasionally emits the opposite sign for the same orientation, which the publisher reads as a rotation of about 2π in one time step: the recorded yaw rate reaches ± 620 rad/s while the robot's heading barely moves. The body twist was therefore re-derived from the recorded pose. The sign of each quaternion is made continuous with the previous one, the world-frame velocity is rotated into the body frame, and the yaw rate is taken from the change in heading,

$$v_{\text{fwd}} = v_x^w \cos \phi + v_y^w \sin \phi, \quad v_{\text{lat}} = -v_x^w \sin \phi + v_y^w \cos \phi, \quad v_\phi = \frac{\text{wrap}(\phi_t - \phi_{t-\Delta t})}{\Delta t} \quad (4.8)$$

Any remaining value beyond a physical bound is treated as an outlier and interpolated across. In the training dataset this corrected 29,866 sign flips in 72,737 samples, and the yaw rate fell from a recorded range of ± 620 rad/s to a physical range of -1.592 to 1.470 rad/s, which agrees with the robot's measured turning ceiling. The constant-velocity dataset needed 39,285 such corrections in 54,535 samples.

Stage 2, removing stalls. Stalls were identified from the signals themselves rather than from the generator's label, so that the moment before the generator noticed is also removed:

$$\text{stall} \iff |v_{\text{cmd}}| > 0.3 \text{ m/s} \text{ and } \sqrt{v_{\text{fwd}}^2 + v_{\text{lat}}^2} < 0.08 \text{ m/s}. \quad (4.9)$$

This removed 91 of 72,737 training samples, 0.13 %, and 20 of 54,535 constant-velocity samples. For comparison, the same filter removed 2.77 % of the dataset collected before the drivetrain change, so the robot on the final plant rarely got stuck.

Stage 3, joining inputs to measurements. The model's inputs are in the generator's log at 45 Hz and its targets in the processed odometry at about 100 Hz, so the two were joined on simulation time by nearest match within 0.02 s. Samples with no command inside that tolerance were dropped, which removes the interval recorded before the generator started. Table 4.7 gives the resulting counts.

Table 4.7: Datasets used for the final model

Stage	Training	Constant velocity
Recorded duration	727.4 s	545.3 s
Odometry samples	72,737	54,535
After removing stalls	72,646	54,515
Train-ready rows	69,217	54,249
Of which at constant velocity	—	45,691

4.3.3 Parameter Fitting

The five parameters were fitted by the nested least-squares procedure of Section 3.3.3, minimising the objective of Equation (3.27) over rows drawn from the training table. The configuration is given in Table 4.8. The training table was split by time, four fifths for fitting and the last fifth held back; 900 rows drawn from the first part form the sample the optimiser scores, and 2,500 rows drawn from the held-back part are used only for evaluation. A separate 1,800 rows were drawn from the constant-velocity table, 300 at each of its six turning radii, so that no radius dominates.

Table 4.8: Configuration of the fit

Setting	Value
Rows scored per objective evaluation	900, drawn from the first 80 % of the training table
Held-out rows	2,500, drawn from the last 20 % of the training table
Constant-velocity rows	1,800, balanced at 300 per turning radius
Channel scales σ_k	Standard deviation of each channel, with a lower limit
Parameters optimised	Logarithms of the five parameters, within fixed bounds
Outer optimiser	Trust-region reflective, Jacobian by finite differences with a relative step of 0.02
Outer iteration limit; tolerance	30 evaluations; 5×10^{-4}
Inner solver	Trust-region reflective, at most 60 evaluations per sample, cold-started, slip ratios bounded to ± 0.95 and the ICR offset to ± 0.5 m
Wheel accelerations	Clipped to ± 20 rad/s ²

Three starting points, one choice. Because the objective is not convex (Section 3.3.3), the fit was run from three starting points, and the three results were compared on data none of them had been fitted to. Each was scored on four independent resamples: two draws of 5,000 rows from the held-back fifth of the training table, at two new random seeds, and two draws of 4,800 rows from the constant-velocity table, balanced at 800 per radius, again at new seeds. The parameter set that ranked first on all four resamples was adopted. The same comparison included the parameters fitted before the drivetrain change, which ranked last on all four, confirming that refitting after that change was necessary rather than cosmetic.

Promotion. The adopted parameters were written into the three places that consume them: the defaults of the motion model in the controller's source, the controller node's declared parameters, and the runtime configuration file. The superseded values were kept alongside them so that an earlier configuration can be restored.

4.3.4 Open-Loop Validation Protocol

Before the model was placed inside the controller, it was validated open loop, on data that played no part in the fit. Holding the fitted parameters fixed, the model predicted the body velocity for every row of the held-out set and of the constant-velocity set, and the predictions were scored with the per-channel error of Equation (3.28) and compared with the no-slip model of Equation (3.11) on the same commands through the relative improvement of Equation (3.29). Two further quantities were recorded for each evaluation: the fraction of rows for which the inner solve converged, and the fraction at which a slip ratio reached its bound. The results are reported in Section 5.3.

4.3.5 Adapting the Model for Real-Time Use

The offline fit solves the slip state with the same trust-region method the source paper uses, which costs about 7 ms per sample. The controller of Section 3.4 predicts hundreds of trajectories over tens of steps within each control period, so that solver cannot be used online. The online model therefore solves the same residuals with a bounded Levenberg–Marquardt iteration written in C++, which costs 2.44 μ s per solve.

The online solver keeps the properties the offline one has and adds the guards a real-time loop needs. It is cold-started at zero slip, which excludes the total-slip solution of Section

3.3.2, and it holds the slip ratios within ± 0.95 and the ICR offset within ± 0.5 m, the same bounds as the fit. Its Jacobian is taken by finite differences, stepping inward where a step would leave the bounds. Its damping is reduced by half after an accepted step and tripled after a rejected one. It stops when the sum of squared residuals falls below 10^{-4} or after 60 iterations. If it does not converge, or returns a value that is not finite, the rollout falls back to the no-slip prediction with the lateral velocity set to zero and the yaw rate scaled by 0.12, so that a failed solve degrades the prediction instead of stopping the controller. The C++ implementation is checked against the Python reference on a table of pinned cases, so that the two cannot drift apart.

4.4 Autonomy Stack

Section 3.4 selected Hybrid A*-guided MPPI and set out the theory of each of its parts. This section describes how that framework was built for this robot, what each part was given as parameters, how the 3D LiDAR was replaced by a stereo depth camera, and the protocol under which the result was tested in closed loop.

4.4.1 Staged Implementation

The framework's released implementation was examined and rejected as a basis for this work: it replaces the guided-seeding laws of Equations (3.41) and (3.42) with cost terms that penalise distance from the path, so the published code does not implement the published algorithm. The framework was therefore re-implemented from the paper in four stages, each verified before the next began, as set out in Table 4.9.

Table 4.9: Stages of the implementation

Stage	Platform	What it established
S1	Python reference, no robot	The search structure: the motion primitives, the penalties on reversing, changing direction and turning in place, the Dijkstra cost-to-go heuristic, the discretisation of nodes, and the cycle that replans on a moving local map. Traversability at this stage was a simplified stand-in, not the chain of Section 4.4.2
S2	Two-wheel differential-drive robot	Port to C++ with the search structure unchanged; the forward-alignment penalty added to the planner; the path-tracking cost terms and the turn-in-place latch added to the controller. The seed ran in its source-faithful form, which was sufficient on a robot that does not slip
S3	Four-wheel skid-steer robot	Primitive speeds returned to the robot's measured limits, from 0.6 to 0.5 m/s

		and from 1.2 to 0.4 rad/s; goal tolerance, goal snapping and partial paths; the four additions to the seed of Section 4.4.5; and the substitution of the friction-based model into the controller's rollout. The planner is unchanged by that substitution, which was verified: the commit that introduces the friction rollout touches no planner file
S4	As S3	The sensor substitution of Section 4.4.6: a forward-facing stereo depth camera in place of the 360° LiDAR. The commit touches no planner or controller file

Two properties of this table are stated so that it is not over-read. The stages are attributed by role, not by date: four of the S3 additions were committed after the S4 sensor swap, so the numbering describes what each stage established rather than when. And S1's traversability was a proxy, so although S1 established the search structure, the parameters of the traversability chain come from elsewhere.

4.4.2 Traversability Estimation as Implemented

The perception chain runs in four steps, from the sensor's point cloud to the grid the planner reads.

Removing returns from above the robot. Each cloud first passes through a filter that keeps a return only if it lies within the robot's own height above the ground of the cell it falls in, as described in Section 3.4.2. The ground of each cell is the lowest occupied height seen there, taken as a minimum over a small neighbourhood so that a cell whose own ground is hidden borrows from its neighbours.

Elevation mapping. The filtered cloud updates a robot-centric elevation map of 13×13 m at 0.1 m resolution, which moves with the robot.

The filter chain. A fixed sequence of grid filters turns elevation into traversability, with the values of Table 4.10. Holes are filled by inpainting, which gives the neighbourhood filters a defined value everywhere. A radial mean produces a smoothed surface, and the surface normals are fitted to that smoothed layer rather than to the raw elevation: normals fitted to raw elevation tilt on sensor noise and report about 18° of false slope on flat ground. Slope follows from the normal by Equation (3.30) and roughness from the scatter about the fitted plane by Equation (3.31); the two are blended into a traversal cost by Equation (3.32). A separate mask carries observation status: it is defined where the

elevation was measured and undefined elsewhere, and it multiplies both scores, so ground that has only been inpainted cannot read as good ground. Finally the protrusion test of Equation (3.33) compares each cell with the lowest elevation within 0.4 m and marks anything standing more than 0.12 m above it, which is what makes a trunk an obstacle when smoothing has flattened it into the terrain.

Publication. The result is published as an occupancy grid in which traversable ground is 0, impassable ground is 100, and cells that were never observed are -1 , so that the planner can tell unobserved ground from ground observed to be flat.

Table 4.10: Parameters of the traversability chain

Quantity	Value	Source of the value
Map window; resolution	13×13 m; 0.1 m	This work
Inpainting radius	0.32 m	This work
Smoothing radius; normal-fitting radius	0.3 m; 0.3 m	This work
Roughness radius	0.3 m	This work
Slope limit ζ_c	0.65 rad	The framework's released configuration
Roughness limit $\sigma_{\perp,c}$	0.25 m	The framework's released configuration
Slope weight w_s	0.8	Reported by Yang et al.
Ground-reference radius; protrusion threshold h_p	0.4 m; 0.12 m	This work

Three departures from the source are declared. Normals are fitted to the smoothed layer, for the reason given above. The saturation of each score at 1 before blending, in Equations (3.30) and (3.31), is not applied before the blend, which makes steep ground cost more rather than less. And the protrusion layer has no counterpart in the source at all: it exists because the sensors here measure visual geometry and the trees have canopy (Section 3.2.5).

4.4.3 Hybrid A* Planner as Implemented

The planner runs at 3 Hz on the simulation clock. Each cycle it rebuilds its costmap from the published grid, projects the goal into the window, recomputes the heuristic and searches.

Costmap. Occupancy is read back as a traversal cost, cells above 0.8 are hard obstacles, and those are dilated by a square of radius 10 cells, or 1.0 m, which is the planner's only model of the robot's size: the collision test is a point test, and 1.0 m was sized against the robot's circumscribed radius of 0.951 m about its tracked origin. Cells that were never

observed count as free, so the planner will search into unmapped space and reroute only around ground it has actually seen, and cells outside the window are blocked. Because a robot standing inside its own inflation would have no admissible first move, inflation-only cells within three cells of the robot are cleared, while genuine obstacles there are not.

Motion primitives. Successors are generated by eight primitives, each held for $T_p = 0.5$ s and integrated in eight substeps:

$$\mathcal{M} = \{(\pm v_p, 0), (\pm v_p, \pm \omega_p), (0, \pm \omega_p)\}, \quad v_p = 0.5 \text{ m/s}, \quad \omega_p = 0.4 \text{ rad/s}. \quad (4.10)$$

The speeds are the robot's measured working limits rather than the source's, and the set contains turns on the spot, which is why the Reeds–Shepp heuristic of the source does not apply (Section 3.4.3). Every substep is collision-checked, so a primitive whose end pose is clear but whose swept arc clips a trunk is rejected.

Cost. A primitive that advances a distance Δs across ground of mean traversal cost $\bar{l}_t$, from a node whose bearing error to the goal is e_g , costs

$$c = \Delta s + w_T \bar{l}_t \Delta s + w_{\text{rev}} \Delta s [\text{reverse}] + w_{\text{dc}} [\text{direction change}] + w_{\text{spin}} [\text{spin}] + w_{fa} (\quad (4.11)$$

with the weights of Table 4.11. The terrain, reverse and alignment terms are charged per metre; the direction-change and spin terms are charged once per primitive. The forward-alignment term is what makes a large change of heading resolve as a turn on the spot followed by driving, rather than as a wide forward arc (Section 3.4.5).

Search. Nodes are keyed by their cell and one of 72 heading bins, and a successor is kept only if it reaches its key more cheaply than any earlier node, which allows a node to be expanded again if a cheaper route to it is found. The heuristic is the Dijkstra cost-to-go of Equation (3.35), recomputed over the whole window every cycle, which keeps it tight enough that it needs no inflation, so the heuristic weight is 1. The search stops when a popped node lies within 0.5 m of the goal, which is a position test only, leaving final heading to the controller, and it is limited to 60,000 node expansions.

Goals and failures. A goal outside the window is projected onto it by Equation (3.43), 0.5 m inside the edge. A goal that falls inside the inflated region is moved to the nearest free

cell within 1.5 m, nearer the robot where there is a choice. If no complete path is found, the planner returns the part of the path that brings the robot closest to the goal, provided it closes at least 1.0 m of cost-to-go. If even that fails, the planner publishes an empty path, which the controller treats as a command to stop, but only before any path has succeeded for the current goal or after thirty consecutive failures; in between, the previous path is held, so that a momentary gap in the map does not stop the robot. The controller accepts a path only if it holds at least two poses and is newer than the goal it is driving to.

Table 4.11: Planner parameters

Parameter	Value
Planning rate	3 Hz (simulation clock)
Primitive speed v_p ; turn rate ω_p ; duration T_p ; substeps	0.5 m/s; 0.4 rad/s; 0.5 s; 8
Terrain weight w_r	2.0
Reverse, direction-change, spin, forward-alignment weights	1.0, 0.5, 0.15, 2.0
Heading bins	72
Obstacle threshold; inflation radius	0.8; 10 cells (1.0 m)
Goal tolerance; expansion budget	0.5 m; 60,000
Goal-snap radius; partial-path gate	1.5 m; 1.0 m
Consecutive failures before stopping	30

4.4.4 MPPI Controller with the Friction Rollout

The controller runs at 10 Hz on the simulation clock and follows the method of Section 3.4.4 with the parameters of Table 4.12. Each cycle it draws 800 candidate command sequences over a horizon of 25 steps of 0.1 s, which is 2.5 s of predicted motion, scores them, and applies the first command of the weighted average.

Rollout. Each candidate is simulated with the friction-based model: the commanded speed and turn rate are converted to wheel speeds by Equation (3.40), the wheel accelerations are taken from the change between consecutive steps and clipped to ± 20 rad/s², and the model returns the body velocity, including the lateral component. The alternative arm used in the comparison of Section 4.4.7 replaces this with the no-slip model, integrating

$$x_{t+1} = x_t + v_t \cos \phi_t \Delta t, \quad y_{t+1} = y_t + v_t \sin \phi_t \Delta t, \quad \phi_{t+1} = \phi_t + \gamma_\omega \omega_t \Delta t, \quad (4.12)$$

where γ_ω scales the predicted turn rate. Setting $\gamma_\omega = 1$ gives the ideal no-slip model of Equation (3.12), and that is the value used in the comparison, so that the contrast is between a model that describes slip and one that assumes none.

Cost. The running cost charges squared distance to the goal, the traversal cost of the cell the robot is predicted to occupy, and control effort; a candidate predicted to enter an inflated obstacle cell is charged a further fixed penalty of 10^3 for each such step. A terminal term charges squared distance to the goal at the end of the horizon. When a path is available, two further terms charge the squared distance to the nearest waypoint and the misalignment with that waypoint's heading.

Turning in place. When the heading error to the target exceeds 1.92 rad, the controller commits to turning on the spot at 3.0 rad/s in the shorter direction and holds that commitment until the error falls below 0.9 rad. The gap between the two thresholds prevents the command from alternating.

Repeatability. The candidate sequences are drawn from one random generator in a fixed order before any of them is simulated, and the rollouts are then scored in parallel across processor cores. Drawing serially keeps the sample set identical from run to run whatever the number of threads, which is what makes the paired comparisons of Section 4.4.7 valid. The generator's seed is a parameter of each trial.

Table 4.12: Controller parameters

Parameter	Value
Control rate	10 Hz (simulation clock)
Samples K ; horizon T ; step Δt	800; 25; 0.1 s
Command noise σ_v ; σ_ω	0.25 m/s; 0.30 rad/s
Temperature β	1.0
Command limits	$v \in [-1,1]$ m/s; $\omega \in [-3,3]$ rad/s
Cost weights: goal, terrain, terminal, effort	1.0, 5.0, 30.0, 0.01
Obstacle penalty per step	10^3
Path-tracking weights: cross-track, heading	12.0, 8.0
Turn-in-place thresholds; command	1.92 rad engage, 0.9 rad release; 3.0 rad/s
Rollout fallback yaw scale	0.12

4.4.5 Guided Seeding for a Scrub-Limited Robot

The mean sequence the controller samples around is rebuilt every cycle from the planner's path, following Section 3.4.5. For each step of the horizon, a target waypoint is chosen by walking forward along the path until 0.6 m of arc length or 0.6 rad of heading change has been accumulated, or until the path reverses direction, whichever comes first. The speed and turn rate for that target follow Equations (3.41) and (3.42) with gains $k_v = 1.2$, $k_\phi = 1.0$ and $k_p = 1.0$. The bearing term is faded in over the first 0.15 m, so a target at the robot's own position is steered by heading error alone. On a reversing segment the speed

changes sign and the bearing is measured from the rear. The resulting turn rate is multiplied by $\kappa_\omega = 3.0$ before being limited to ± 3.0 rad/s, because the robot achieves only part of the turn rate it is commanded. An internal pose is then advanced with the same rollout model the samples are scored with, and the next step's target is chosen from there.

Two configurations of these settings are used throughout the evaluation and are defined in Table 4.13. Configuration P is the stack as it arrived from the two-wheel robot: the additions of Section 3.4.5 are switched off, so the seed uses the source's laws unmodified. Configuration A is the shipped configuration, with all of them on. Both carry the terms added at stage S2, the path-tracking cost, the turn-in-place latch, the forward-alignment penalty, so the two differ only in the seed.

Table 4.13: Seed parameters of the two configurations

Parameter	Configuration P	Configuration A
Bearing-term distance gate	0 (off)	0.15 m
Target selection	Nearest waypoint	Budgets of 0.6 m and 0.6 rad
Yaw boost κ_ω	1.0 (off)	3.0
Reverse-aware seed	Off	On
Seed gains k_v, k_ϕ, k_p	1.2, 1.0, 1.0	1.2, 1.0, 1.0

4.4.6 Sensor Substitution: the Stereo Depth Configuration

Two sensor configurations are defined, and exactly one is active in a trial. Both publish a point cloud on the same topic, so everything downstream is identical. Their properties are given in Table 4.14.

The stereo configuration models the Orbbec Gemini 335L. Its field of view follows from Equation (3.44), and its frame tree was replicated from the manufacturer's own robot description so that the frames match those a real driver would publish. Its depth noise is applied outside the simulator, because the simulator supports only a constant standard deviation, whereas stereo error grows with the square of range: a separate node re-noises each point along its ray with the law of Equation (3.47) at $k_z = 0.004 \text{ m}^{-1}$, which reproduces both of the manufacturer's quoted figures.

Two of its settings are measured limits rather than choices. The update rate of 6 Hz is the measured ceiling of the simulated camera, set by serial per-frame work in the simulator rather than by the modelled device, and the resolution was reduced from 848×540 to 424×270 to reach it. The reduction keeps the image's aspect ratio, so by Equation (3.44) the

field of view is unchanged. The elevation mapping's assumed scan duration was matched to the sensor's rate.

The substitution changes four things at once: the horizontal field of view, the depth-error law, the mounting pitch and the sampling pattern. A comparison between the two configurations can therefore be reported as the cost of replacing this LiDAR with this camera, but no part of the difference can be attributed to any one of the four. Intermediate configurations that would separate them, the LiDAR masked to the camera's field of view, and the camera without injected noise, were designed and not run.

Table 4.14: The two sensor configurations

Property	Reference: 3D LiDAR	Substitute: stereo depth camera
Samples	900×16	424×270
Horizontal field of view	360°	90°
Vertical field of view	$30^\circ (\pm 15^\circ)$	64.98° , from the aspect ratio
Range	0.3–30.0 m	0.2–10.0 m
Update rate	10 Hz	6 Hz (measured ceiling)
Mounting	Horizontal, 0.81 m above ground	Pitched 30° down, 0.81 m above ground
Depth noise	The simulator's own model	Applied outside the simulator, $\sigma_z = 0.004 z^2$

4.4.7 Closed-Loop Evaluation Protocol

World and goals. All closed-loop trials were run in the evaluation world of Section 4.2.5. Six goals were fixed in advance by a two-stage procedure. The first stage filtered candidate positions on a 0.25 m grid: between 4 and 20 m from the start, clear of any trunk by more than the 1.0 m inflation plus a margin, so that the planner never has to move a goal; with the straight line from the start blocked by at least one trunk, so that a goal cannot be reached without planning; at least 4 m apart from one another; and one goal in each 30° bin of required heading change over 0 to 180° , because a goal that demands almost no turning cannot distinguish one rollout model from another. The second stage ran each candidate once and kept it only if the robot arrived, the goal was never moved by the planner, and there was no contact. All six were confirmed on 7 September under the shipped configuration with the friction rollout. Table 4.15 lists them.

Table 4.15: The six evaluation goals

Goal	Position (m)	Distance (m)	Heading change	Blocking trunks	Outside the first map window
g1	(13.000, 3.250)	13.400	14.0°	1	Yes
g2	(3.000, -4.000)	5.000	53.1°	1	No
g3	(0.250, -19.000)	19.002	89.3°	1	Yes

g4	(−3.500, 7.000)	7.826	116.6°	1	Yes
g5	(−10.000, −12.750)	16.204	128.1°	1	Yes
g6	(−10.250, −2.750)	10.612	165.0°	1	Yes

A trial. The robot is reset to the start pose and allowed to settle, the goal is published, and the ground-truth track is logged until the controller announces arrival, the trial times out, or the robot contacts a trunk. The time limit is 360 s of simulated time, set at about 2.8 times the slowest run observed on any configuration, since a limit set from a fast configuration would score a slow but successful one as a failure.

Seeds. The controller's random seed is fixed for each trial, at one of five values. Its default behaviour is to draw a seed at random and record it, which leaves each run reproducible but leaves two arms with different noise, and every test in Section 3.4.8 is paired. Fixing the seed is what makes a pair a pair. Five seeds were used, and the same five in every arm.

Experiments. Four closed-loop experiments were run on this goal set, as summarised in Table 4.16. The first two walk the seed configuration from P to A one parameter at a time, once under each rollout model, so that the effect of each addition can be seen separately and under both models. The third runs Configuration P with the friction rollout for comparison with those endpoints. The fourth is the sensor substitution, which is the only experiment repeated across seeds and paired trial by trial.

Table 4.16: Closed-loop experiments

Experiment	Design	Trials
Seed-parameter ladder, friction rollout	Configuration P walked to Configuration A one parameter at a time, one trial per goal per rung	42
Seed-parameter ladder, no-slip rollout	The same ladder with $\gamma_\omega = 1$, whose endpoint is Configuration A's seed with the no-slip model	30
Configuration P with the friction rollout	One trial per goal, with a further variation with the turn-in-place latch off	6 + 6
Sensor substitution	Two sensors $\times$ six goals $\times$ five seeds, paired on the goal and the seed	60

Everything not being varied is held fixed across the ladders: the command envelope, the sampling noise, the sample count, the horizon, the inflation radius, the turn-in-place latch, the path-tracking cost and the forward-alignment penalty. The sensor comparison holds the configuration and the rollout model at their shipped values and varies only the sensor. Before each block of sensor trials, eight checks confirm the world, the sensor, the seed, the

controller's parameters, that only one publisher and one node of each name is present, that the controllers are running, and that the perception chain is the one intended. Trials that aborted for reasons outside the experiment were quarantined and re-run.

Measures and statistics. The measures and tests are those defined in Section 3.4.8. Detection recall is counted while the trial runs rather than afterwards, because the map moves with the robot and discards what it leaves behind. The equivalence margins for the sensor comparison were fixed before the campaign: ± 10 percentage points on success, ± 20 % on time to goal and ± 10 % on path efficiency.

Limits of the design. The ladders and the Configuration P runs are single trials per cell: they show the direction of an effect, not its distribution, and only the sensor comparison is paired and repeated across seeds. In that comparison, each arm plans its own route, so the trunks each sensor meets depend on the route it drove; the detection figures are therefore a property of the sensor operating in the loop, not of the sensor alone.

4.5 Survey Workflow

Section 3.5 set out what a tree-image survey requires and the geometry of a photo station. This section describes the software that carries it out: how a mission is requested and sequenced, how each station is chosen and its range corrected, how the images are taken, how the operator selects trees, and what is written to disk.

4.5.1 Mission Architecture

The survey is a package of three nodes and two request types. An operator sends a survey request naming the trees to visit, their positions and identifiers, the frame those positions are in, the standoff, the number of images and the heading step between them, the time allowed per tree, and a label for the run. The mission server then works through the list, and for each tree in turn:

1. selects a photo station on a circle around the trunk (Section 4.5.2);
2. publishes that station as a navigation goal and waits for the controller to report arrival, re-checking the station while the robot drives;
3. hands the tree to the capture node as a separate **capture request** and waits for it to finish;
4. moves to the next tree.

The server reports progress continuously, which tree, which state, how many images taken, how far the robot still has to drive, and returns the run directory, the manifest, the counts of trees captured and skipped, and the reason for each skip.

Skip, never abort. A tree that cannot be reached within its time limit, or whose capture fails, is recorded with its reason and the mission continues. A survey of sixty trees cannot be ended by the twelfth, and the images already written must survive it. The time limit is measured in wall-clock time, so it scales with the simulation's real-time factor: its default is 180 s, and the multi-tree runs were given 400 s, because a leg of a few metres at a low real-time factor can otherwise exceed the default for no reason but the clock.

Who drives. The mission server never commands motion. Navigation drives the robot between trees, the capture node drives it during a panorama, and the two are arbitrated by a priority multiplexer: capture at priority 100, navigation at 10, each with a 0.5 s timeout, so that navigation resumes automatically half a second after the capture node stops publishing. The capture node is therefore silent whenever it is idle; a stream of zero commands from an idle node at the higher priority would hold the robot still for the whole mission. At the end of a mission, or on cancellation, the server publishes a goal at the robot's own position, which the controller treats as already reached, which is how the robot is parked.

Arrival. The controller announces arrival on its own topic, echoing the goal it arrived at, and the mission server waits for an echo that matches the goal it published to within 0.05 m. This was the one change made to the controller for the survey, and it only publishes: deriving arrival inside the survey code would have meant a second copy of the controller's goal tolerance, free to drift out of step with the one that actually stops the robot.

4.5.2 Station Selection and the Camera-Referenced Standoff

Choosing a station. Station selection is a set of pure functions over the occupancy grid, so it can be checked offline against a world file. Eight candidate bearings are placed around the trunk at the station radius, ordered from the side the robot is approaching from and alternating outwards, and the first candidate with no impassable cell within 0.6 m of it is taken. Cells that were never observed count as free, because the map is a 13 m window that moves with the robot and usually does not cover the tree when the mission starts; requiring observed-free ground would reject all eight candidates and skip every tree before the robot had moved. The choice is re-checked every 3 s while the robot approaches, when

the map does cover the station, and if all eight candidates are blocked the tree is skipped rather than photographed from an obstacle.

The radius. The standoff requested by the operator, 2.4 m by default, is the range from the **camera** to the trunk at the first image, not from the robot's own reference point. The station radius follows from Equation (3.58) with the camera offsets read from the robot's transform tree rather than written into the survey code: 0.330 m ahead of the reference point and 0.194 m to the side, giving a station radius of 2.7372 m. A parameter selects the older robot-referenced behaviour for comparison.

Correcting the range. The controller stops within its goal tolerance of 0.5 m, which is larger than the margin the framing bound of Equation (3.57) leaves, so the range is closed before the first image. After aiming at the trunk, the capture node drives straight along the camera's optical axis under position feedback on the measured range,

$$v = \text{clamp} (k_{\text{pos}} (\rho_{\text{cam}} - \rho_c), -v_{\text{trim}}, +v_{\text{trim}}) , \quad (4.13)$$

with $k_{\text{pos}} = 0.6 \text{ s}^{-1}$ and $v_{\text{trim}} = 0.15 \text{ m/s}$, stopping when the range is within 0.05 m of the target. Position feedback is used rather than a velocity command because the drivetrain controller's integral term keeps pushing after a velocity target is stepped to zero (Section 4.2.3). Three guards bound the manoeuvre: it may not move the robot more than 0.8 m, may not bring the camera closer than 1.9 m, and may not run longer than 30 s. Each guard ends the correction and lets the capture proceed rather than failing it, because an imperfectly ranged image is worth more than no image. A range below 2.25 m is recorded with the image as a warning that the top of the tree may be cut off.

4.5.3 Capture Routine and the Triggered Still Camera

The state machine. A capture runs as: settle, face the trunk, settle, correct the range, settle, face the trunk again, settle, take image 0, turn 90°, settle, take image 1, and so on to image 3. The second aim is needed because the range correction translates the robot; the aim it produces is the reference for all four images.

Aiming and turning. The bearing to the trunk is computed from the camera's pose in the transform tree, never from the robot's heading plus a fixed offset, because `base_link` is rotated by 90° and the camera sits about 0.38 m off the centre of rotation, so a hardcoded

offset would aim every image 90° away from the tree. Both aiming and turning use the same law,

$$\omega = \text{clamp}(k_{\text{turn}} e_\phi, -\omega_{\text{cap}}, +\omega_{\text{cap}}), \quad |\omega| \geq \omega_{\text{brk}} \text{ while } |e_\phi| > e_{\text{tol}}, \quad (4.14)$$

with the values of Table 4.17. The limit of 6.0 rad/s is far above what the robot achieves, commanding it yields about 1.6 rad/s, and the floor of 1.2 rad/s exists because a proportional law alone dies below the torque needed to break away from rest a few degrees short of the target. Each image's heading target is a whole number of 90° steps from the first aim rather than a step from the heading last reached, so the errors of the four turns do not accumulate.

Settling and the shutter. An image is taken only after the robot's measured speeds have stayed below 0.02 m/s and 0.02 rad/s for 0.4 s. The shutter is then triggered and the node waits for a frame; if none arrives within 0.5 s it triggers again, up to twelve attempts, because the simulator drops the first trigger after an idle period.

The camera. Images are taken by a second camera, separate from the navigation sensor and independent of which navigation sensor is fitted. It renders only when triggered, so it costs nothing while the robot drives, and it is mounted on the modelled camera body of the chassis rather than on the navigation sensor's mount: 0.607 m above the ground, turned to face forward, and pitched 16° up so that a whole tree fits in frame at the design standoff (Section 3.5.2). It produces 1280×800 images over a 90° horizontal field of view, with depth clipped between 0.2 and 30 m.

Table 4.17: Survey mission and capture parameters

Parameter	Value
Camera standoff; station radius	2.4 m; 2.7372 m
Station candidates; clearance; re-check period	8; 0.6 m; 3 s
Images per tree; heading step	4; 90°
Turn gain k_{turn} ; limit ω_{cap} ; breakaway floor ω_{brk} ; tolerance e_{tol}	8.0; 6.0 rad/s; 1.2 rad/s; 2°
Settle thresholds; hold	0.02 m/s, 0.02 rad/s; 0.4 s
Range trim: gain, speed limit, tolerance	0.6 s^{-1} ; 0.15 m/s; 0.05 m
Range trim guards: travel, floor, time	0.8 m; 1.9 m; 30 s
Shutter: attempt interval, attempts	0.5 s; 12
Time allowed per tree	180 s by default; 400 s for the multi-tree runs
Drive priorities: capture, navigation; hand-back	100, 10; 0.5 s
Still camera	1280×800 ; 90° horizontal; 0.2–30 m; triggered

4.5.4 Operator Interface

The trees known to the system are published as markers for the visualisation tool, and the operator selects targets by clicking near them with the tool's own point-publishing control. A click within 1.5 m of a known tree is moved onto that tree; a click further away is kept as given. Selected trees are numbered in the order they were clicked, each with its station circle drawn around it, and that order is the order the mission visits them in. Four commands control the selection, start the mission, undo the last pick, clear the list, and cancel a running mission, and the picker colours each tree as the mission reports it captured, skipped or in progress.

The catalogue is built from the world's own record of trunk positions rather than from the poses at which tree models were placed, because a tree model's origin sits a median of 0.50 m from its trunk (Section 4.2.5), and a picker built on model origins would send the robot to a point beside the tree.

4.5.5 Output Data and Metadata

Each mission writes one directory, named for the run and the time it started, holding a manifest for the whole mission, a record of the mission's own parameters and outcomes, and one folder per tree. Each image is a file named for its tree, its heading index and the time it was taken, beside a companion file of the same name holding that image's record. Nothing is ever overwritten.

The record of an image holds what a later analysis needs to use it: the tree's identifier and position, the heading index, the time in both universal and simulation form, the camera's full six-degree-of-freedom pose and the robot's pose, the commanded and measured headings with their error, the camera's achieved range to the trunk and the distance the range correction moved, how far the robot had drifted from its station, the number of shutter attempts, whether the top of the tree was at risk of being cut off, the image's size and encoding, the camera's calibration, and the source of the pose, which is the simulator's ground truth here. Fields for latitude and longitude are present and empty, so that geographic references can be added later without rewriting the dataset.

The manifest gains one row per image, and each row is written and flushed to disk as its image is written rather than assembled at the end, so that a mission interrupted at its fortieth tree still leaves thirty-nine trees of usable data.

4.6 Summary

This chapter described how the system whose design Chapter 3 set out was built and how it was tested.

The robot was designed in CAD against the requirements of Section 3.2.1 and turned into a simulation model whose mass, inertia, centres of mass and contact geometry were each corrected against measurement rather than accepted from the export, and whose drivetrain reproduces a two-motor skid-steer through a torque-limited velocity-tracking loop and a physics-rate coupling between the wheels on each side. The farm was built from a procedural terrain and a library of parametric trees, and the worlds used for the reported work differ only in what stands on the same ground.

Driving data were then collected on that robot in effort mode, cleaned of a simulator artefact that corrupted the recorded yaw rate, filtered of stalls, and joined into tables of commanded wheel motion against measured body motion. The five friction parameters were fitted to 900 rows of that data by a nested least-squares procedure, chosen among three starting points by their accuracy on data none of them had been fitted to, and validated against the no-slip model before being embedded anywhere. A bounded solver was written so that the same model could run inside a controller at a cost of microseconds rather than milliseconds.

The navigation framework was re-implemented from its paper in four stages, because its released code does not implement its published algorithm, and each part was given the parameters set out in Sections 4.4.2 to 4.4.5. The friction model replaces the no-slip model in the controller's predictions and changes nothing else. A stereo depth camera then replaced the 3D LiDAR without touching planner or controller. Navigation was tested on six goals chosen to require planning and to span the range of turns, with fixed seeds so that arms could be paired, in three experiments: two ladders that walk the seed configuration from its ported form to its shipped form under each rollout model, and a paired comparison of the two sensors.

Finally, the survey workflow was built on that stack: a mission server that turns a list of trees into navigation goals and image captures, a station geometry solved for the camera rather than the robot, a capture routine that aims, corrects its range and takes four images around each tree, an operator interface for selecting trees, and a record written per image as it is taken.

Chapter 5 reports what these methods produced: the behaviour of the simulated robot and farm, the accuracy of the identified friction model, the navigation results including the sensor substitution, and the survey mission.

References (placeholders, to be formatted)

Placeholder	Source
—	No new citations in 4.1–4.2; tools named in the text (Fusion 360, GrabCAD, Blender, gmsh, ros2_control, Gazebo, DART) may need citing to their documentation

CHAPTER 5 - RESULTS AND ANALYSIS

5.1 Introduction

Chapter 4 described how the robot, its friction model, the autonomy stack and the survey workflow were built, and how each was tested.

This chapter reports what those tests produced and analyses what the results show. Because the report has no separate discussion chapter, each result is interpreted where it is presented and its limitations are stated beside it.

The results follow the same order as Chapters 3 and 4, because each part depends on the one before it. Section 5.2 reports the behaviour of the simulated robot and the farm it was tested in, which is what the rest of the chapter relies on. Section 5.3 reports how accurately the identified friction model predicts the robot's motion, which is what justifies placing it inside the controller. Section 5.4 reports the autonomy stack: the staged verification, the closed-loop comparison of the friction and no-slip rollouts, and the paired comparison in which a stereo depth camera replaced the 3D LiDAR. Section 5.5 reports the survey workflow that uses the stack, and Section 5.6 summarises what the chapter establishes and what remains open.

5.2 Platform and Environment

Every later result was obtained on the simulated robot and farm of Section 4.2. This section reports what the corrected model does, how its behaviour depends on the choices made in building it, and what the farm worlds contain. All figures here were measured on the corrected model: the wheels' centres of mass on their axles, the masses set from the design rather than from the exporter, and the constants of Table 4.3.

5.2.1 The Corrected Simulation Model

Correcting the model changed it by orders of magnitude rather than by margins. The exported masses described a 1,646 kg machine; setting them from the design gave 19.384 kg, of which 41.3 % is in the wheels, with the centre of mass 234.7 mm above the ground. The wheels' inertia about their axles is $0.595 mr^2$, between the disc and ring limits of Equation (3.9), which is what a tyre on a hub should give. Moving each wheel's centre of mass onto its axle removed an inflation of its spin inertia from 0.0142 to 0.112 kg·m², a factor of 7.9.

Two constants were resolved against measurement rather than taken from one source. The track width from the CAD measurement, 0.862733 m, was 60 mm short because it added half a wheel width where a full width was needed; the corrected value of 0.9225 m from Equation (4.4) agrees with the 0.9234 m spanned by the wheel meshes to within 1 mm. That correction matters beyond tidiness: the earlier value had been feeding a 7 % error into the wheel speeds computed for every recorded dataset, and into the odometry of the drive controller.

5.2.2 Motion Behaviour of the Corrected Model

Contact geometry decides whether the robot turns at all. With the wheels' centres of mass corrected onto their axles and a cylinder as the wheel collision shape, a pure turning command under velocity control produced no yaw. Twenty single-variable experiments ruled out the candidates in turn, wheel inertia, wheel friction, ground friction under those test conditions, the physics engine, the width of the cylinder, and the slip plugin, leaving the cylinder's line contact as the cause. Two findings came out of the same sweeps and both shaped the rest of the work. Friction coefficients set on the wheels have no effect under DART: only the ground surface's friction acts. And on flat ground under velocity control, the robot's turn rate and forward speed did not change as the ground friction was varied, because a velocity-controlled joint imposes wheel speed whatever the contact provides. The second finding is why every friction dataset was collected in effort mode (Section 4.3.1).

A later correction to this result is recorded because it changes its scope. The lock was first called geometric and unbreakable by friction, on the strength of a sweep run in a world whose ground carried no slip. The terrain worlds carry a ground slip of 0.01–0.03, and on those the cylinder does rotate. The lock is therefore a property of the cylinder together with zero-slip ground, not of the cylinder alone. The production fix was neither: the cylinders were replaced by a tyre-shaped collision mesh, which turns while keeping the corrected inertials.

Turning is limited by torque against scrub, not by kinematics. Table 5.1 gives the in-place yaw rate reached under a command of 3.0 rad/s on two surfaces, at three torque limits, for four independent motors and for two linked sides. Yaw rate rises with available torque on both surfaces and under both topologies, which is the signature of a torque limit rather than a kinematic one. Lower friction raises the

whole curve by two to three times, consistent with scrub being the resistance. And at matched torque per wheel the two-motor topology turns faster than four independent motors, by 62 % on laterite at 10 N·m per wheel, because the mechanical link stops the inner wheel stalling and dragging while the outer wheel drives.

Table 5.1: In-place yaw rate (rad/s) under a 3.0 rad/s command, by drivetrain topology and torque

Surface	Torque per wheel	Four independent motors	Two linked sides
Laterite, μ 0.85	6 N·m	0.152	0.160
Laterite, μ 0.85	8 N·m	0.278	0.387
Laterite, μ 0.85	10 N·m	0.324	0.525
Leaf litter, μ 0.60	6 N·m	0.464	0.585
Leaf litter, μ 0.60	8 N·m	0.587	0.669
Leaf litter, μ 0.60	10 N·m	0.562	0.708

The width of the contact band is a second lever on turning. Replacing the flat tyre mesh with the crowned mesh of Section 4.2.2 narrowed the contact band from 89.5 mm to 26.9 mm while leaving the outer radius, and therefore the ride height, unchanged to within 0.1 mm. On the de-sloped leaf-litter terrain at 16 N·m per side, the in-place yaw ceiling rose from 0.306 rad/s to 0.514–0.555 rad/s, an increase of about 68 %, and the gain held across the whole range of commands, as Table 5.2 shows. The flat tread also plateaus earlier: it stops improving beyond a command of about 2 rad/s, while the crowned tread continues to about 3.

Table 5.2: Achieved in-place yaw rate (rad/s) by commanded rate, flat against crowned tread, 16 N·m per side

Commanded (rad/s)	Flat tread	Crowned tread	Gain
1.0	0.193	0.313	+62 %
2.0	0.268	0.446	+66 %
3.0	0.266	0.470	+77 %
4.0	0.272	0.468	+72 %
6.0	0.306	0.514	+68 %

The torque limit was then chosen from the plant's own response. On the crowned tread, raising the limit from 16 to 32 N·m per side lifted the ceiling from 0.555 to 1.610 rad/s, with a peak of 2.71; raising it again to 64 N·m gave 1.425 rad/s, no better. The drivetrain is torque-limited below 32 N·m per side and traction-limited above it, so 32 N·m is the knee of the curve and is the plant on which the friction model was fitted and every navigation result obtained.

This is also where the design's motor sizing can be judged. Equation (4.2) sized the motors at 5.2 N·m each for driving in a straight line up a 15° slope. The robot reaches only 0.555 rad/s against a 3.0 rad/s command at 16 N·m per side, three times that sizing, and needs 32 N·m per side to approach the commanded rate. Turning in place on this robot therefore demands torque far beyond what a straight-line specification provides, because the wheels must scrub sideways across the ground rather than roll along it (Section 3.2.2). A design sized only for gradients would be under-powered for the manoeuvre the robot performs at every tree.

The side coupling reproduces a mechanical link. Table 5.3 gives the median mismatch between the two wheels on a side. Four independent loops driven through one per-side controller left the wheels 5.47 rad/s apart on flat ground and 7.98 rad/s apart on relief, which is not chain-like behaviour. Synchronising their velocities in the controller removed only 13 %. Synchronising their phase at the controller's 100 Hz made the mismatch worse, at 5.11–10.39 rad/s, and the same coupling at the physics rate of 1 kHz brought it to 1.11 and 0.87. The plugin that applies the coupling at every physics step reached 0.58 and 0.74, below the 1 rad/s that counts as chain-like, while leaving the drive loop at its cheap 100 Hz.

Table 5.3: Median mismatch between the two wheels on a side (rad/s)

Coupling	Flat	Relief
None, one controller per side	5.47	7.98
Velocity synchronisation in the controller	3.90	—
Phase coupling in the controller, 100 Hz	5.11–10.39	—
Phase coupling in the controller, 1 kHz	1.11	0.87
Phase coupling in the simulator, every physics step	0.58	0.74

What the robot achieves against what it is commanded. The figure that matters for the rest of the report is the gap between command and achievement: a command of 6.0 rad/s yields about 1.61 rad/s on the final plant, a factor of about four. Every part of the autonomy stack that assumes a commanded turn will be delivered is therefore wrong by that factor, which is what the friction rollout of Section 3.3.2 is for, and what the seed's yaw boost and the widened command envelope of Section 4.4.5 compensate for.

5.2.3 The Cocoa-Farm Worlds

The terrain generator produced a 513×513 pixel heightmap over 50×50 m with 2.178 m of relief, most of it the 4 % planted slope. Removing the best-fitting plane, of 2.263° , left 0.2556 m of micro-relief with a standard deviation of 0.044 m, which is the ground every reported result was obtained on.

The farm holds 100 trees. Placement first produced 45 trees floating 11–118 mm above the surface, because each procedurally generated tree carries its own offset between its model origin and its lowest point; lifting each tree by its own measured offset closed this to zero, while trees whose geometry dips below their origin were left slightly buried, as a root flare is. The resulting nearest-neighbour spacing is 3.010 m at minimum, 3.424 m at the median and 6.425 m at maximum, which is a density of 400 trees per hectare over the plot. Neighbouring trees therefore sit about as close as on a $3 \text{ m} \times 3 \text{ m}$ farm, while the irregular gaps make the whole plot less dense than a regularly planted one.

Each tree's trunk carries a collision tube of 218 vertices and 120 triangles, built by the ring algorithm of Section 4.2.4. The per-band median filter is what makes the tube follow the bark rather than the branches: on the tree used to develop it, the tube's width fell from 0.55 m to 0.33 m once branch geometry was rejected band by band, because the fork's main branches diverge most strongly along one axis.

The de-sloped farm world reproduces the tree layout of the original: re-running the generator's own check reproduces the authored height of all 100 trees to a mean error of 0.0002 m and a maximum of 0.0005 m.

5.2.4 Limitations of the Platform and Environment

Four limitations bound everything that follows.

- **The contact is rigid.** Neither collision mesh deforms, so the contact patch is a geometric intersection rather than a tyre model. Contact pressures computed from it are not a soil-compaction model and are not used as one.
- **Terrain differs only in friction.** Every relief world shares one heightmap for both its visual and its collision surface, so surfaces differ in their friction coefficients and textures over identical geometry. That makes the friction comparisons controlled, but it also means no result here distinguishes a soft surface from a firm one of the same shape.

- **The evaluation ground is level.** The slope was removed because the friction-based model has no gravity term (Section 3.2.5), so the model and the evaluation worlds match by construction. Results on graded ground are therefore not established here, and adding a grade term to the force balance remains future work.
- **The plant is the simulator.** The robot's motion comes from DART's contact physics, and the friction model was fitted to that plant. No claim is made in this report about transfer to a physical robot, and no claim is made that the crowned collision mesh or the 32 N·m per-side limit is the correct physical drivetrain: they are the plant under study.

5.3 Friction Model

This section reports the datasets the final model was identified from, the parameters that were fitted, how accurately the model predicts the robot's motion on data it never saw, what it costs to evaluate, and what the result does not establish.

5.3.1 Datasets of the Final Model

Two recordings were made on the de-sloped leaf-litter world with the crowned wheel mesh and a torque limit of 32 N·m per side: a training run of 727.4 s and a constant-velocity run of 545.3 s. Table 5.4 follows them through the three processing stages of Section 4.3.2.

Table 5.4: The two datasets, stage by stage

Stage	Training	Constant velocity
Recorded duration	727.4 s	545.3 s
Odometry samples	72,737	54,535
Quaternion sign flips corrected	29,866	39,285
Stall samples removed	91 (0.13 %)	20 (0.04 %)
Samples with no command within tolerance	3,402	259
Train-ready rows	69,217	54,249
Of which at constant velocity	—	45,691

Three properties of these datasets bear on what can be concluded from them.

The cleaning step recovered a usable signal from an unusable one. The recorded yaw rate spanned ± 620 rad/s, an artefact of the simulator's odometry publisher rather than motion. After the correction of Equation (4.8) the training run's yaw rate spans -1.592 to 1.470 rad/s. That range is an independent check on the cleaning: it agrees with the 1.61 rad/s in-place ceiling measured separately in Section 5.2.2, which no part of the cleaning uses.

The robot rarely got stuck on this plant. Stalls accounted for 0.13 % of the training samples, against 2.77 % of the dataset recorded before the wheel mesh and torque limit were changed. The filter removed them either way, but the difference is a property of the plant worth recording: the crowned tread at 32 N·m per side has enough authority that it seldom pins itself.

The commands bracket what the robot can do. The training profile reaches 3.0 rad/s, with 19.5 % of rows commanding at least 1.6 rad/s and 14.7 % at least 2.0. Since the robot achieves about 1.61 rad/s, the data cover both the region where commands are met and the region where they are not, which is the region the model exists to describe.

5.3.2 Fitted Parameters

Table 5.5 gives the parameters adopted for the final model. They were fitted from 900 rows of the training table by the nested procedure of Section 4.3.3, and chosen from three starting points by their accuracy on four resamples that played no part in any fit. The choice was unanimous: the adopted set ranked first on all four, and the set fitted before the drivetrain change ranked last on all four.

Table 5.5: Fitted parameters of the friction-based model

Symbol	Quantity	Value
μ_r	Coefficient of rolling resistance	0.0435
μ_x	Longitudinal friction coefficient	1.9545
μ_y	Lateral friction coefficient	1.6635
λ	Slope of longitudinal force against slip ratio	2.0904
C	Slip-angle scale of the lateral force	0.1228

These values calibrate the model to one robot on one surface at one drivetrain configuration. They are not offered as measurements of the friction of the ground: the model's input is the commanded wheel velocity, so the velocity-tracking loop, the torque limit and the shape of the wheel's contact are absorbed into the fit along with the surface (Section 3.3.2). The practical consequence was already seen in the record: when the wheel mesh and the torque limit changed, the parameters fitted before that change became the worst of the four candidates on every resample, which is why they were refitted rather than carried forward. The fit itself cost 12 evaluations of the objective, 47 residual calls and 103 s.

5.3.3 Open-Loop Prediction Accuracy

Holding the parameters fixed, the model predicted the body velocity for every row of two sets it had never seen: 2,500 held-out rows from the last fifth of the training recording, and 1,800 rows drawn from the constant-velocity recording at 300 per turning radius. Table 5.6 compares those predictions with the no-slip model of Equation (3.11) on the same commands, by the measures of Equations (3.28) and (3.29).

Table 5.6: Prediction error against the no-slip model, on data neither model was fitted to

		Friction-based	No-slip	Improvement
Held-out, 2,500 rows	Yaw rate (rad/s)	0.1016	0.8421	+87.9 %
	Forward velocity (m/s)	0.2203	0.1563	-40.9 %
	Lateral velocity (m/s)	0.0540	0.0502	-7.6 %
Constant velocity, 1,800 rows	Yaw rate (rad/s)	0.0880	0.5499	+84.0 %
	Forward velocity (m/s)	0.2050	0.0686	-199 %
	Lateral velocity (m/s)	0.0351	0.0345	-1.5 %

The gain is in yaw, and it is large. The no-slip model is wrong about the turn rate by 0.84 rad/s on the held-out data, a figure of the same size as the turn rates the robot actually reaches, because it assumes every commanded turn is delivered, and this robot delivers about a quarter of it (Section 5.2.2). The friction-based model predicts the same rows to 0.10 rad/s. This is the channel that matters for the use the model is put to: a controller that plans a turn must know how much of it will happen.

The forward channel is worse, and that is reported rather than softened. The no-slip model predicts forward speed well on this plant, 0.156 m/s on the held-out rows and 0.069 m/s on the constant-velocity rows, because forward motion involves little slip when the wheels are rolling. The friction-based model does not match it there, and on the constant-velocity set it is three times worse. The constant-velocity set is where the no-slip assumption is most nearly true, being steady driving with no commanded acceleration, so it is also where the friction model's extra machinery buys least and costs most.

The lateral channel is a near-tie, for a reason worth stating. The no-slip model predicts zero lateral velocity always, so its lateral error is simply the root-mean-square of the robot's measured lateral velocity: 0.050 and 0.035 m/s. That those numbers are small says

the robot's sideways motion is small, and the friction-based model, which does predict a lateral velocity, is marginally worse. Neither model is useful as a predictor of side-slip on this plant.

The errors are the model's, not the solver's. The inner solve of Equation (3.26) converged on 100 % of rows in both sets, no row ended with a slip ratio or an ICR offset at its bound, and the median residual was of order 10^{-21} . The two checks defined in Section 3.3.4 therefore pass, and the errors in Table 5.6 can be attributed to the model rather than to a search that failed or saturated.

Refitting after the plant change was necessary. Against the parameters fitted before the wheel mesh and torque limit changed, the adopted set lowers held-out yaw error from 0.125 to 0.103 rad/s and constant-velocity yaw error from 0.107 to 0.090 rad/s on the same resamples.

5.3.4 Computational Cost of the Online Solver

The offline fit solves the slip state with a trust-region method at about 7 ms per sample. The controller of Section 4.4.4 evaluates 800 candidate sequences over 25 steps, which is 20,000 predictions for a single control step: at the offline solver's cost that would take about 140 s against a control period of 100 ms. The bounded solver written for the online path costs 2.44 μ s per solve, which puts the same 20,000 predictions at about 49 ms of processor time, and spreading the rollouts across the machine's threads brings the measured control cycle to 16–19 ms. The same cycle with the no-slip rollout is 3–4 ms, so the friction model costs roughly five times as much to run and still fits inside the control period. Section 5.4.3 reports the timing in full.

5.3.5 Limitations

- **The forward channel regresses.** The model is adopted for its yaw accuracy, and Table 5.6 shows it is worse than the no-slip model at forward speed. Any use that depends on predicting distance travelled rather than heading would not be served by it.
- **Lateral velocity is not predicted usefully.** Predicting zero is about as accurate on this plant, so the model's side-slip output should not be read as evidence about side-slip.
- **The model was fitted to the simulator it is tested against.** The validation rows are unseen, but they come from the same plant, so what Table 5.6 establishes is prediction

accuracy within this simulation, not physical fidelity. The report makes no claim of transfer to a physical robot.

- **The two plant changes were made together.** The crowned wheel mesh and the rise from 16 to 32 N·m per side were adopted in the same step, so the change in the fitted parameters cannot be attributed to either alone. Separating them would need a dataset with one changed and not the other.
- **The model has no grade term,** so it was fitted and evaluated on level ground by construction (Section 5.2.4). Behaviour on a slope is not established.
- **Transfer between terrains has not been tested.** The experiment designed for it fits parameters on one terrain and deploys them on the others, across three terrains and 108 trials, and it has not been run. Until it is, the parameters are established for the surface they were fitted on and no statement is made about how far they carry.
- **There is no long-distance dataset on this plant.** Validation used held-out rows from the mixed-acceleration recording and rows from the constant-velocity recording; the third category of the source paper, long uninterrupted drives, was not collected after the drivetrain change.

5.4 Autonomy Stack

This section reports what the navigation stack does. It begins with what each implementation stage established, then compares the friction and no-slip rollouts in closed loop, reports what the friction model costs to run, and ends with the three parts of the sensor substitution: what the camera failed to map, whether the robot still completed its task, and what the substitution cost in clearance.

5.4.1 Staged Verification

The Python reference established the search structure, and one result from it shaped the deployed planner. On a world where the goal sits on a raised platform behind a cliff crossed by a single ramp, a straight-line heuristic points through the cliff, and the search probes the cliff face before finding the ramp. Table 5.7 gives the cost of that, measured by re-running the reference.

Table 5.7: Search effort on the ramp world, by heuristic

Heuristic	Heuristic weight	Node expansions	Search time
Straight-line distance	2.0	131,130	20.7 s
Dijkstra cost-to-go	2.0	40	0.006 s, plus 0.021 s to

			build the field
Dijkstra cost-to-go	1.0	324	0.051 s, plus 0.021 s to build the field

Both heuristics find the ramp, and the two routes are similar but not identical. The expansion counts are what matter: an obstacle-aware estimate of the cost to go reduces the search by three orders of magnitude at the same heuristic weight. Since the deployed planner replans several times a second on a map that keeps changing (Section 4.4.3), a heuristic that costs a hundred thousand expansions per cycle is unusable, which is why the cost-to-go field is recomputed over the whole window every cycle.

Three further findings came from the same reference and are carried forward as warnings rather than as results of this robot. Avoiding a soft cost is weaker than avoiding a hard obstacle when the map is built locally: producing the detour that a complete map gave at a terrain weight of about 1 required about 3 when the robot could only see part of the world. Planning without a prior map cost efficiency rather than robustness, in that the robot wandered more but was not trapped. And because the traversability cost of Equations (3.30) and (3.31) uses the magnitude of the slope, it cannot distinguish climbing from crossing a slope, which remains true of the deployed chain.

The transfer from the two-wheel robot to the skid-steer robot is reported qualitatively. The same code reproduced its validated behaviour on the two-wheel robot, which does not slip, and did not reproduce it on the four-wheel skid-steer robot, which turns only by slipping. That outcome was predicted before it was observed, from two facts already established: this robot turns entirely by scrubbing (Section 5.2.2), and the no-slip rollout carries no lateral velocity and integrates heading at the full commanded rate. A rollout that forbids sideways sliding cannot describe a robot whose only means of turning is to slide. No trial is reported for this: it is a retrospective observation about two platforms, not a controlled experiment.

5.4.2 Closed-Loop Comparison of the Friction and No-Slip Rollouts

Two ladders walked the seed configuration from Configuration P to Configuration A one parameter at a time, once with the friction rollout and once with the no-slip rollout, over the same six goals in the evaluation world with everything else held fixed. Table 5.8 gives the outcome of every rung.

Table 5.8: Ladder outcomes, one trial per goal per rung

Seed configuration	Friction: accepted	Friction: collisions	Friction: min gap (m)	No-slip: accepted	No-slip: collisions	No-slip: min gap (m)
Configuration P	4/6	2	1.277	2/6	4	0.994
+ bearing-term gate	3/6	2	1.079	5/6	0	1.282
+ target budgets	3/6	3	1.131	5/6	1	1.345
+ yaw boost	5/6	1	1.257	6/6	0	1.336
+ reverse-aware seed	6/6	0	1.307	6/6	0	1.390
Configuration A	6/6	0	1.400	not run	—	—

The rollout model's benefit is conditional on the quality of the seed. Table 5.9 compares the two arms at the two ends of the ladder. At Configuration P's seeds the rollout model matters a great deal: the no-slip arm collides twice as often, keeps 0.28 m less clearance, and spends three times as long inside the planner's inflation radius. At Configuration A's seeds it stops mattering: both arms reach every goal without contact, and the differences that remain, 2.2 % in total time, a few centimetres of clearance, change sign from goal to goal and are smaller than the variation seen between repeats of one configuration.

Table 5.9: The two rollouts at each end of the ladder

Measure	P seeds, friction	P seeds, no-slip	A seeds, friction	A seeds, no-slip
Goals accepted	4/6	2/6	6/6	6/6
Collisions	2	4	0	0
Minimum trunk gap (m)	1.277	0.994	1.307	1.390
Near-miss fraction	0.035	0.101	0.000	0.000
Reverse fraction	0.132	0.191	0.046	0.015
Energy proxy (N·m·s)	939	1,057	647	614

That reading is consistent with what the two models differ about. Both describe the same robot; they disagree about how much of a commanded turn is delivered. A seed that commands turns the robot cannot make leaves the controller relying on its rollout to notice, and the friction model does; a seed already shaped for a scrub-limited robot commands turns the robot can make, so the two rollouts predict nearly the same motion. On this course, and at these six goals, the friction rollout is a safeguard against a poor seed rather than an improvement on a good one.

Where the two ladders agree. The target-budget parameter is the dither fix on both: the rate at which the commanded turn rate changes sign falls from 2.24 to 1.00 changes per second with the friction rollout and from 2.24 to 0.95 with the no-slip rollout. The dither

rates track each other across every rung, so whatever the rollout model changes, it is not how often the command reverses.

Where they diverge, and why the wider metric set was worth keeping. The bearing-term gate moves nothing measurable on the friction arm. On the no-slip arm it is the single largest improvement in pass or fail anywhere in the study, 2/6 to 5/6, and simultaneously the worst configuration in either ladder on two other measures, taking plan-direction changes from 13 to 56 and the energy proxy from 1,057 to 1,745 N·m·s. The robot stopped committing to a path, and so stopped hitting things. Judged on goals reached alone, that rung is a triumph; judged on how it was driving, it is not.

A single configuration, run twice, did not give the same result. Configuration P with the friction rollout appears in Table 5.8 as 4 of 6 goals with two collisions. The same configuration, run again on the same goals as a separate confirmation, gave 3 of 6 with three collisions, the three failures ending 0.28–0.40 m inside the robot's own chassis envelope, against Configuration A's six clean passes with minimum gaps of 1.17–1.48 m. The disagreement is not a contradiction: with one trial per goal, the outcome of a marginal configuration is a coin toss between the same two regimes. It is the clearest evidence in this chapter for reading the ladders as direction rather than distribution. Within that confirmation run, one failure reproduced across two independent runs against the same trunk on two versions of the goal set, so at least one failure of Configuration P is a property of the configuration rather than of the goal.

Two details of that run are recorded rather than smoothed. Two of its three failures were ended by hand once contact and a wedged robot had been established, instead of waiting out a time limit that would have cost half an hour of wall-clock time to record a failure already proven; the reason for rejection is contact, measured, and the time-out was a formality. And the ladder rungs' failures move between goals, g5 and g6 at Configuration P, g4 at the next two rungs, g5 again at the third, which is what one trial per cell looks like.

5.4.3 Control-Loop Timing

Across the seven configurations of the friction ladder, the controller's mean cycle time was 16.1–18.5 ms, with a 95th percentile of 19.5–21.1 ms, against a control period of 100 ms. Across the five configurations of the no-slip ladder it was 3.1–3.6 ms, with a 95th percentile of 4.1–4.7 ms. The friction rollout therefore costs about five times as much as the no-slip rollout and uses about a fifth of the period available, which is consistent with

the cost derived in Section 5.3.4. Neither arm came close to missing its deadline, and the cycle time varied little with the seed configuration, so the cost is a property of the rollout model rather than of how the robot was being driven.

5.4.4 Sensor Substitution: the Perception Deficit

The substitution was tested as 60 trials, two sensors over six goals and five seeds, paired on the goal and the seed, in the evaluation world, with the rollout model and the configuration held at their shipped values. The premise of the comparison is that the camera maps materially less of the farm, and Table 5.10 establishes it.

Table 5.10: Trunks mapped as impassable, of those that entered the map window

Sensor	Trunks mapped	Recall	Per-goal range
3D LiDAR	458 / 468	97.9 %	—
Stereo depth camera	262 / 459	57.1 %	33 % to 69 %

The deficit is 40.8 percentage points. Its shape matches the geometry of Section 3.4.7 rather than any failure of perception: the camera sees a 90° wedge, so it can observe about a quarter of the bearings around it at any instant, and the per-goal figures rise with how much the robot turned along the way, from 33 % on the goal that required least turning to 69 % on the goal that required a reversal of heading. Detection is also highly repeatable per goal: the camera returned exactly 3 trunks of 9 on one goal across all five seeds, and the LiDAR returned 16 of 18 on another across all five, so these are properties of the route and the geometry rather than of the random seed.

Table 5.11 gives the same measurement goal by goal. Ordered by the heading swept along the route rather than by goal number, the camera's recall rises steadily: the goal demanding least turning sits closest to the 25 % a 90° wedge can observe at any instant, and the goal demanding a 165° reversal is highest, because a robot that turns sweeps its wedge across bearings a stationary wedge never reaches. The LiDAR is at 100 % on every goal but one, and that exception is not noise: it returns 16 of 18 on all five seeds of that goal.

Table 5.11: Trunks mapped per goal, as a percentage of those that entered the map window

Goal	3D LiDAR	Stereo depth camera
g1	100 %	53 %
g2	100 %	33 %
g3	89 %	57 %
g4	100 %	60 %
g5	100 %	62 %

g6	100 %	69 %
----	-------	------

5.4.5 Sensor Substitution: Task Completion and Equivalence

Table 5.12 gives the paired outcome across the same 60 trials.

Table 5.12: Paired comparison of the two sensors, 30 pairs

Measure	Result	Pairs favouring the camera
Goals reached	30/30 with either sensor	—
Tree contacts	0 with either sensor	—
Time to goal	median difference -0.800 s (-4.05 %)	24 / 30
Path efficiency	median difference $+0.017$ ($+1.72$ %)	25 / 30
Minimum trunk gap	median difference -0.006 m	16 / 30 lower

Because the claim being tested is that the substitution costs nothing in task completion, a test that fails to find a difference is not enough (Section 3.4.8). The two one-sided tests were run against the margins fixed before the campaign, and all three completion measures were equivalent within them: the success rate trivially, both arms reaching every goal; time to goal at -4.05 % against a ± 20 % margin, rejecting both one-sided nulls at $p \leq 6.5 \times 10^{-9}$; and path efficiency at $+1.72$ % against a ± 10 % margin, rejecting both at $p \leq 1.7 \times 10^{-5}$.

Two qualifications are recorded with that result. Equivalence is two-sided, and the ordinary difference tests are also significant on time and efficiency, both favouring the camera, so the correct statement is that the two arms differ by less than the margin, with the difference in the camera's favour, not that they are the same. And individual pairs do exceed the margins, one of thirty on time and five of thirty on efficiency, even though the central tendency does not.

Reporting efficiency as a ratio hides where the difference sits, so the same quantity is given as distance in Table 5.13: for each goal, the median distance driven across the five seeds, less the shortest route that completes the task, which is the straight line reduced by the 0.5 m arrival tolerance the robot never has to drive.

Table 5.13: Extra distance driven beyond the shortest route that reaches the goal (m)

Goal	3D LiDAR	Stereo depth camera
g1	0.33	0.20
g2	0.52	0.40
g3	0.80	0.83
g4	0.79	0.69
g5	1.39	0.99
g6	2.81	1.49

The camera's advantage is concentrated on one goal, and its cause is the camera's own deficit. Five goals separate the two sensors by at most 0.4 m, and on one the camera drives slightly further. All five pairs that fall outside the $\pm 10\%$ efficiency margin belong to g6, the goal requiring a near-reversal of heading, where the LiDAR adds 2.2 to 3.4 m of detour against the camera's 0.4 to 1.0 m. The mechanism follows from Section 5.4.4: the LiDAR maps trunks beside and behind the robot that the camera never records, and the planner then routes around obstacles that are absent from the camera's map. The camera is therefore quicker in part because it is ignorant, and the efficiency difference in its favour should be read as a consequence of the perception deficit rather than as better control. It is also the clearest case in this campaign where a measure that looks like an advantage and a measure that looks like a cost are the same fact seen twice.

5.4.6 Sensor Substitution: Clearance

Obstacle clearance was predicted to degrade, because a robot cannot hold a margin against a trunk its map does not contain. Table 5.12 shows the aggregate minimum gap essentially unchanged, a median paired difference of 6 mm. That aggregate was challenged three times as an artefact, on the reasoning that the closest trunk of a run is almost always one the sensor did map, so the trunks it missed would never appear in the statistic. Measuring the missed trunks directly answers it: Table 5.14 partitions every trial's clearance by whether the trunk was mapped.

Table 5.14: Closest approach to trunks, partitioned by whether they were mapped

Sensor	Trunks	Minimum (m)	25th percentile (m)	Median (m)
LiDAR	Mapped	1.070	1.296	1.356
LiDAR	Missed	4.473	4.525	4.526
Camera	Mapped	1.156	1.235	1.341
Camera	Missed	2.977	3.070	4.124

Of the 197 trunks the camera never mapped, none came within 2.977 m of the robot, so none came within the planner's 1.0 m inflation radius, let alone the 0.820 m chassis envelope. The same holds for the ten the LiDAR missed. The reasoning behind the objection was sound and the conclusion it predicted was wrong, and it is reported as withdrawn rather than quietly dropped. The mechanism is now explicit: a forward wedge loses the trunks beside and behind the robot, and on this goal set those are the trunks the route never approaches. Recomputing each trial's minimum from the saved ground-truth

track reproduces the figure the harness recorded to within 0.5 mm, so the partition rests on the same measurement as the aggregate.

5.4.7 Limitations

- **The ladders are single trials.** One trial per goal per rung shows the direction of an effect, and Section 5.4.2 gives a case where two runs of one configuration disagreed on both goals reached and collisions. No distribution, and no significance, should be read from Tables 5.8 or 5.9.
- **The conditional result holds for this course.** That the friction rollout's benefit disappears at good seed settings is established on six goals in one world; it is not a general statement about slip-aware rollouts.
- **The canopy was present in the evaluation world. The sensor comparison therefore includes the simulated foliage returns that survive the canopy filter, but it still remains a simulation result: the canopy is static visual geometry, not a moving crop canopy under changing light and wind.**
- **The sensor substitution changes four things at once, field of view, depth-error law, mounting pitch and sampling pattern, so Tables 5.10 to 5.12 report the cost of replacing this LiDAR with this camera, and no part of the difference can be attributed to the field of view alone, however well the geometry of Section 3.4.7 predicts it.**
- **Detection was measured along each arm's own route.** Both arms planned their own paths, so which trunks entered the map window depended on where each arm drove. The deficit is therefore a property of the camera operating in the loop, not of the camera in isolation; the fixed-route experiment that would separate them was not run.
- **The robot always knew where it was.** Pose came from the simulator throughout, so nothing here tests navigation under the localisation error a real robot would carry (Section 3.4.6).

5.5 Survey Workflow

The survey layer of Section 4.5 turns a list of trees into navigation goals and into images with records attached. It was tested in three settings of increasing realism: offline, against every trunk in the farm at once, where the station geometry can be checked exhaustively; headless, against a substitute for the robot and the simulator, where the paths that are hard

to provoke live, a blocked tree, an unreachable tree, a dropped shutter, can be forced; and live in the simulator, where the whole stack runs together. The live survey runs used the same farm world as the closed-loop trials of Section 5.4, with complete tree models and canopy included, because the canopy is the subject of the images. The robot carried the 3D LiDAR for navigation and the triggered still camera for the survey, and the figures below come from the mission as run on the shipped configuration, with the camera-referenced standoff and the seed settings of Configuration A in place.

5.5.1 Station Selection Across the Whole Farm

Station selection was tested against all 100 trunks of the farm at once, by building an occupancy grid from the world's own record of trunk positions, painting each trunk impassable over a radius of 0.35 m, and asking for a station at every tree in turn. Four properties were checked on every answer: that the station lies at the station radius from its trunk, that no impassable cell lies within the clearance radius of it, that the returned heading points at the trunk, and that the candidate on the side the robot approaches from was the one taken. Table 5.15 gives the outcome.

Table 5.15: Station selection tested against all 100 trunks of the farm

Property checked	Result
Station at the station radius from its trunk	100/100
No impassable cell within 0.6 m of the station	100/100
Returned heading points at the trunk	100/100
Approach-side candidate taken	98/100
Trees with no usable station	0
Control case: trunk ringed at the station radius by 16 obstacles	Refused, and the tree skipped

The control case is what makes the count of zero skips meaningful, since a selector that can never refuse also reports no skips. Presented with a trunk ringed by obstacles at the station radius, the selector returned no station and the tree was skipped, rather than returning a station on an obstacle.

The two trees that were not photographed from the approach side follow from the farm's spacing. Nearest-neighbour distances run from 3.010 m to 6.425 m with a median of 3.424 m, and a station on the circle is blocked when a neighbouring trunk falls between 1.25 and 3.15 m away, this band being the clearance radius and the painted trunk radius on either side of the station circle. Twenty-two of the hundred trees have a neighbour inside that band, so part of their circle is unusable; for twenty of them the blocked arc did not contain

the approach-side bearing, and for two it did, whereupon the search rotated to the next candidate. The fallback therefore fired exactly as often as the geometry required, and the eight-candidate ring was enough to place a station at every tree in this farm.

5.5.2 Turning, Drive Arbitration and the Skip Path

Three behaviours of the capture routine were measured before any mission was run, because each of them fails silently.

Turning accuracy. Driving the four turns as the sole publisher of drive commands, the routine produced exactly 90.0° between consecutive heading commands on all four steps, with a worst error between commanded and measured heading of 0.74° . Because each target is a whole number of steps from the first aim rather than a step from the heading last reached, the accumulated error over three runs came out equal and opposite to the last step's error, at -0.447° , $+0.966^\circ$ and -0.639° against final-step errors of $+0.447^\circ$, -0.966° and $+0.639^\circ$. The error of a four-image sequence is therefore the error of one turn, and re-basing each target on the heading actually reached would have made it compound instead.

Drive arbitration. Repeating the sequence with the navigation controller publishing its idle stream of zero commands, the drive matched the capture routine on all 180 ticks during which the routine was turning, and no navigation command reached the drive. Worst heading error rose from 0.74° to 1.88° , on all four images rather than one, which is the latency of the extra republishing step rather than run-to-run variation. The priority scheme of Section 4.5.1 therefore does what it must: the capture routine owns the drive while it runs, and navigation resumes when it stops.

The skip path. A mission was then run against a substitute for the robot and simulator, whose plant reproduces the measured scrub limit of Section 5.2.2 rather than tracking its command, and whose camera reproduces the simulator's habit of dropping the first trigger after an idle period. Table 5.16 gives the outcome of the mission built to force skips.

Table 5.16: The skip path, with the reason recorded for each tree

Tree	Outcome	Reason recorded
Reachable	Captured, four images	—
Ringed by 16 obstacles at the station radius	Skipped	Station became blocked and no alternative is free
35 m away, with 25 s allowed	Skipped	Approach timed out after 25 s, 23.23 m short

The mission ran to completion and kept the images of the tree it had already photographed, which is the behaviour Section 4.5.1 requires of it: a survey of many trees cannot be ended by one tree it cannot reach. The ringed tree also exercised the re-check of the station during the approach, since its station was chosen while the ring of obstacles was still outside the map window and was rejected three seconds later when the map covered it. On the same run the substitute camera dropped eight triggers and the retry shutter recovered every one, taking ten triggers to produce four images at each tree.

5.5.3 The Five-Tree Survey Mission

Five trees were then selected from the farm and sent to the mission server as one request, with the robot navigating between them under the stack of Section 5.4. The mission captured **5 of 5 trees, 20 images, none skipped**. Table 5.17 reports it tree by tree, in the order visited.

Table 5.17: The five-tree survey mission, in the order the trees were visited

Tree	Transition into the tree (sim s)	Four images (sim s)	Camera range at the first image (m)	Worst heading error (°)	Largest wander during the four turns (m)	Shutter attempts for four images
022	—	13.08	2.445	1.21	0.700	13
027	10.84	7.10	2.449	1.21	0.507	5
070	10.77	9.89	2.447	3.49	0.345	8
060	8.75	10.39	2.452	1.56	0.370	9
037	11.52	9.93	2.442	2.20	0.955	9

Transitions are measured from a tree's last image to the next tree's first image, so each contains the approach, the turn to face the trunk, the range correction, the settling and the shutter; the first tree has none, since the mission began at the robot's spawn pose. The four transitions total 41.9 s of simulated time, and at this run's real-time factor of 0.213 the mission occupied roughly five times that in wall-clock time. All twenty images were taken; the shutter needed between one and five attempts per image, which is the simulator's dropped first trigger appearing throughout rather than only after long idle periods.

A navigation defect the survey exposed. The same five trees in the same order had been run a day earlier, before the seed settings of Configuration A were adopted, and one of the four transitions then took 31.25 s against 9.87 to 10.34 s for the other three. The cause was in the controller rather than the survey layer: with the seed's target

taken one motion primitive ahead, its commanded turn rate was 0.200 rad/s, smaller than the standard deviation of 0.5 rad/s with which the controller samples around it, so the command crossed zero 2.3 times per second, 56 sign changes over 2,438 telemetry samples, and each crossing discarded the torque the effort controller had built up, leaving the yaw stalled for 47 % of the leg. Repeating that leg alone with the target budgets and yaw boost of Table 4.13 reduced it from 22.40 to 3.52 s of simulated time over three runs. In the mission reported above, that transition is 11.52 s, in line with the other three.

Two qualifications keep this from being read as a mission-level speed-up. The station radius also changed between the two runs, from 2.2000 m to 2.7372 m, and because the trees ring the working area a larger radius pulls every station inward: the robot drove 6.938 m in total rather than 10.140 m, a reduction of 1.46 times against a total transition time 1.47 times shorter, so most of the mission-level saving is geometric and not the controller's. Per metre driven, only the pathological leg improved, from 0.080 to 0.118 m/s, while the other three became slower, most probably because the later run's transitions carry the range correction and its legs of 1.0 to 1.5 m are short enough for the fixed cost of facing, settling and shooting to dominate. What improved without qualification is the spread: the ratio of the slowest leg to the fastest fell from 4.79 to 2.38, which is to say no leg remained pathological. This is the same seed defect that Section 5.4.2 measured deliberately; here it appeared as a mission taking three times as long over one leg of clear ground, which is how it was first noticed.

5.5.4 Camera Range and Framing

Equation (3.57) requires the camera to be at least 2.154 m from the trunk for a 3 m tree to fit in frame, and the constraint is one-sided: closer than that and the top of the tree is cut off. Two errors stand between the requested standoff and the range actually achieved. Measured across the four earlier live runs at the first image of each tree, the camera sat 0.335 m nearer the trunk than the robot's reference point, with a standard deviation of 0.003 m, while the controller stopped between 0.47 and 0.58 m short of the station. The first error is deterministic and the second is not, and before the correction of Section 4.5.2 they happened to oppose each other: the camera ranges came out between 2.187 and 2.372 m against a requested 2.2 m, which is to say the requested number was met by cancellation rather than by design, and correcting only the camera offset would have made it less true.

Table 5.18 shows what that left at the first image of each tree, against the same five trees after the station radius was solved for the camera and the range closed in the loop.

Table 5.18: Camera range at the first image of each tree, before and after the range correction

Tree	Before: 2.2 m requested at the robot's reference point	After: 2.4 m requested at the camera
022	2.330	2.445
027	2.245	2.449
070	2.315	2.447
060	2.467	2.452
037	2.046	2.442
Spread	0.421	0.010

The two columns were taken against different targets, so the quantity to compare is the spread and the margin against the framing bound, not the two means. Before the correction the spread was 0.421 m and one image fell to 2.046 m, below the 2.154 m bound, and that image lost roughly the top 0.12 m of a 3 m crown, the hazard predicted from the geometry, firing on the fifth tree of the first full mission. After it, the five ranges lie between 2.442 and 2.452 m, a spread of 0.010 m, every one within 0.052 m of the 2.4 m target and consistent with the correction's own stopping tolerance of 0.05 m. The correction drove between 0.405 and 0.444 m at each tree to close a gap of the same size, and none of the twenty images was flagged as at risk of clipping.

Two further checks support the geometry rather than the run. The station radius solved from the camera offsets read out of the robot's transform tree predicts the camera to sit 0.3372 m inside the robot's reference point, against the 0.335 m measured live, so the model of Equation (3.58) agrees with the robot to about 2 mm. And repeated offline over all 100 trunks, the solved radius of 2.7372 m places the camera at 2.4 m at every one of them, leaving 0.246 m of margin against the clipping bound. The remaining three images of each panorama keep their range swing, between 2.386 and 3.323 m in this mission, because they look outward at the surroundings and have no subject at a fixed range; the four images of a tree therefore do not lie on a circle of constant radius, which is by design rather than an error.

5.5.5 Captured Data and Metadata

The mission wrote 20 images and 20 companion records, one per image, and a manifest of 20 rows, each row written and flushed as its image was written. Every image was 1280 ×

800 pixels in colour, and every record named the simulator's ground truth as the source of its pose, with the fields reserved for latitude and longitude present and empty. No earlier run was overwritten, each mission having written to its own directory named for the run and its starting time.

The records also measure the behaviour that Section 3.5.2 anticipated from a skid-steer robot turning in place. Distance from the pose at which the first image was taken grew to between 0.345 and 0.955 m by the end of a four-image sequence, the wander being larger than the 0.755 m seen in the early test that motivated the design. At a range of 2.4 m, wander of 0.955 m is about 22° of bearing error, so a sequence that computed its headings once on arrival would have pointed the last images well away from the tree. Recomputing the bearing from the camera's live pose at every aim instead held the heading errors of this mission between 0.10° and 3.49° , with a median of 0.90° , and storing each image's own pose with it means the wander is recorded rather than merely survived.

5.5.6 Limitations

- **One mission of five trees.** The figures of Tables 5.15 and 5.16 come from a single run on a single farm. They demonstrate that the workflow completes and show the size of its errors; they support no distribution and no claim of repeatability.
- **Re-stationing has never fired in a live run.** Because a tree is nearly always outside the 13 m map window when its station is chosen, and unobserved ground counts as free, the approach-side candidate is taken every time. The re-check has only ever rejected a station on the substitute stack, where the obstacles were placed to force it.
- **The images demonstrate data collection, not inspection.** At 2.4 m a whole tree fits in a 1280×800 frame, which is what the mission was built to show. An individual pod occupies too few pixels for its surface to be assessed, so the images support the workflow's claim and not yet a disease-monitoring one.
- **The trees were photographed where the world file said they were.** Tree positions were exact, so no error in them was ever passed through Equation (3.55). A survey of a real farm would need surveyed positions or the trunk detector that Section 3.5.1 shows is otherwise unnecessary.
- **The time allowed per tree is wall-clock time,** so it shrinks in usefulness as the real-time factor falls; the multi-tree runs were given 400 s for legs of a few metres, and a

longer leg on a more heavily loaded machine would still risk being skipped for no reason but the clock.

- **Trees are visited in the order given.** The mission does not order them, so the distance driven depends on the order in which the operator clicked, and no route optimisation was attempted.
- **One motion was recorded without being explained.** In a degenerate regression run the robot translated 0.71 m during a state that commands pure rotation, against 0.01 to 0.11 m normally. It is a single sample from an abnormal start, and it is reported rather than diagnosed.

5.6 Summary

This chapter reported what the tests of Chapter 4 produced.

The simulation model was shown to be worth testing against only after correction. Its exported masses and inertias were replaced by measured ones, its track width by the value implied by the wheels themselves, and its wheel contact by a crowned tread; the corrected model turns in place at rates that rise with torque and with contact narrowness, and the torque needed to turn it exceeds the torque that its straight-line motor sizing had implied, which is a result about the design method as much as about the robot. The farm worlds were built on one terrain, and the closed-loop trials used the full cocoa-farm world with complete tree models and canopy present.

The friction-based model was identified from driving data collected on that corrected robot, and it predicts the robot's motion more accurately than the no-slip model does on data neither was fitted to, by margins reported in Section 5.3.3. Its online solver costs microseconds, which is what allows the model to be evaluated inside a sampling controller rather than only offline. The fitted values calibrate the model to this robot on this surface; whether they transfer to another terrain is not established here, and the run that would establish it has not been made.

The autonomy stack was verified stage by stage, and the choice of heuristic was shown to change search effort by orders of magnitude on the same problem. Placing the friction model inside the controller's rollout helped substantially at the seed settings the stack arrived with and not at all at the settings it ships with, which is the chapter's most consequential negative result: the benefit is conditional on how well the controller is seeded, and reporting it without that condition would overstate it. Replacing the 3D

LiDAR with a stereo depth camera cost 40.8 points of trunk detection and nothing measurable in task completion, with time to goal and path efficiency equivalent within the margins fixed before the campaign. The predicted loss of clearance did not appear, and the measurement that would have shown it was made directly rather than inferred: of 197 trunks the camera never mapped, none came within 2.977 m of the robot, because a forward-facing sensor loses the trunks beside and behind the robot and those are the trunks the route never approaches.

The survey workflow completed a five-tree mission without intervention, captured every tree, and wrote every image with a record of where and when it was taken. Its geometry was checked against all 100 trunks of the farm rather than the five it visited, and the one framing failure it suffered was traced to a cancellation between two errors, corrected by solving the station radius for the camera and closing the range in the loop, after which the range at the first image varied by 10 mm across five trees.

Three things remain open at the end of this chapter, and each is stated where it arises rather than only here. The friction parameters have been transferred to no other terrain. The closed-loop comparisons are single trials per goal, so they carry direction and not distribution. The robot always knew where it was, its pose coming from the simulator throughout, so nothing here tests the stack under the localisation error that a real deployment would carry. The sensor comparison was run in the full evaluation world with canopy present, but it remains a simulation result rather than a test under real foliage, lighting and wind. Chapter 6 draws the conclusions these results support and sets out the work that would close these gaps.

CHAPTER 6 - CONCLUSION

6.1 Introduction

The aim of this project, stated in Section 1.3, was to enable a low-cost, skid-steer autonomous mobile robot to navigate the uneven, intercropped terrain of a Ghanaian cocoa farm reliably enough to collect localised crop data. This chapter states what has been achieved against that aim, summarises the work through which it was achieved, sets out the constraints within which the findings hold, and recommends the work that would extend them.

6.2 Summary of the Study

The literature established that the systems capable of high-quality agricultural data collection and those capable of field autonomy are both capital-intensive, and that each assumes structured row geometry, reliable satellite positioning, or both. It also established the specific technical gap this project addresses: terrain-aware navigation frameworks for unstructured ground predict motion with a no-slip kinematic model, a friction-based model that predicts skid-steer slip had not been placed inside a real-time controller, and the sensing those frameworks assume is priced beyond the research groups the robot would serve.

The design theory that followed traced one fact, that a skid-steer robot turns only by making its wheels slip, through the platform, the motion model and the navigation system. A four-wheel, two-motor skid-steer robot was designed and represented in simulation, the no-slip model was shown to be unable to describe its turning, and the friction-based kinematic model was adopted in its place, together with a procedure for identifying its parameters and a measure for judging them. Hybrid A*-guided model predictive path integral control was selected as the navigation framework because it places no restriction on the motion model used in its predictions.

The implementation corrected the simulation model against measurement, collected driving data in effort mode, identified the friction parameters by a nested least-squares procedure, and embedded the model in the controller by way of a bounded solver written for the purpose. The navigation stack was then evaluated in closed loop against the no-slip model it replaces and against a substitution of the reference 3D LiDAR by a stereo depth

camera, and the complete system was applied to an autonomous tree-image survey mission.

6.3 Conclusions

This study has established the following, in the order in which the objectives of Section 1.4 were pursued.

1. **A skid-steer farm robot has to be sized for scrubbing rather than for gradients.**
The motor sizing derived from straight-line travel up a slope has been shown to under-specify the torque the robot needs to turn in place by a factor of about six, and the achievable turn rate has been shown to depend on the width of the wheel's contact band as well as on torque. Turning, not climbing, is the binding mechanical requirement for a robot that must stop at every tree.
2. **The friction-based kinematic model has been identified for this robot and predicts its turning far more accurately than the model it replaces. On data neither model was fitted to, it has predicted yaw rate within 0.10 rad/s against the no-slip model's 0.84 rad/s, an improvement of 87.9 %. It has proved less accurate than the no-slip model on forward velocity, so the gain it delivers is specific to the rotational channel, which is the channel a planner depends on, since a controller that commands a turn must know how much of that turn will occur.**
3. **The model has been placed inside a real-time controller, which its originators had left as future work.** A bounded solver has evaluated it in 2.44 μ s per prediction, against about 7 ms for the offline solver, bringing the controller's cycle to between 16 and 19 ms within a 100 ms control period. A model previously usable only for offline analysis has therefore been shown to run inside the control loop of a sampling-based controller on ordinary hardware.
4. **Reasoning about slip has widened the range of controller tunings over which navigation succeeds.** With the seed settings the navigation framework arrived with, the friction-based prediction has reached four of six goals where the no-slip prediction reached two, with half the collisions and greater clearance; with the settings adopted in this work, both have reached every goal. The friction-based prediction has therefore acted as a safeguard against a poorly tuned controller rather than as an improvement upon a well-tuned one, and this conditional character is itself a finding: a stack that

must be tuned precisely before it works is fragile in a way that one reasoning about slip is not.

5. **A low-cost stereo depth camera has been shown to substitute for the reference 3D LiDAR without measurable cost to the task.** Across sixty trials, paired on goal and seed, both sensors have reached every goal, with time to goal and path efficiency equivalent within margins fixed before the trials began. The camera has mapped 40.8 percentage points fewer tree trunks, a deficit that follows from its narrower field of view, but the loss of obstacle clearance this predicted has not occurred: of the trunks the camera never mapped, none came within 2.977 m of the robot. The cost of the cheaper sensor has fallen on map completeness rather than on task completion, for the routes driven here.
6. **The complete system has performed the task it exists to serve.** An autonomous survey mission has visited five trees, captured twenty images and skipped none, writing each image with the record needed to trace it to its tree, time and pose. Solving the photo station for the camera rather than for the robot, and closing the range in the loop before the first image, has held the camera's range to the trunk to a spread of 10 mm across the mission, against 0.42 m beforehand.

Taken together, these conclusions fulfil the aim of Section 1.3 within the scope defined in Section 1.5: a low-cost skid-steer robot has navigated a simulated Ghanaian cocoa farm autonomously and collected a complete tree-image record without intervention, using a motion model that describes how such a robot actually moves and a range sensor its intended users could afford.

6.4 Constraints of the Study

Six constraints bound these conclusions, and each is stated with its results in the chapters above.

The work has been carried out entirely in simulation; no physical prototype was built or tested, so every conclusion describes the simulated robot and awaits confirmation on hardware. The friction parameters have been identified for one terrain, one torque limit and one wheel geometry, and no transfer of those parameters to another terrain has been demonstrated. The closed-loop comparisons of the two motion models were single trials per goal, which establishes the direction of an effect but neither a distribution nor its significance. Every closed-loop trial was run in the full evaluation world with canopy

present, so the sensor comparison includes simulated foliage returns, but not the lighting changes, wind motion, leaf movement or sensor artefacts of a real cocoa farm. The robot's pose was taken from the simulator throughout, so nothing here tests navigation under the localisation error a real robot accumulates. Finally, the survey mission demonstrates data collection rather than crop diagnosis: its images record whole trees, not the surface detail of individual pods, and the re-selection of a blocked photo station, though implemented and tested offline, never arose during a live mission.

6.5 Recommendations

The next steps are listed in the order in which they would most strengthen the conclusions above.

1. **Validate on hardware.** Build the platform designed here and repeat the identification and the closed-loop trials on real ground. This is the single step that would convert every conclusion from one about a simulated robot into one about a robot, and the friction parameters, being identified from commanded velocity and pose alone, can be re-identified on hardware by the same procedure without new instrumentation.
2. **Establish terrain transfer.** Identify the model on a second terrain and test each parameter set on the other, to determine whether the parameters describe the surface, the robot, or the pairing of the two. Until this is done, the model must be re-identified for every surface.
3. **Repeat the closed-loop comparisons.** Run several trials per goal so that the difference between the two motion models can be reported as a distribution with a test of significance rather than as a direction, and so that the conditional result of Section 6.3 can be quantified rather than described.
4. **Add localisation.** Replace the simulator's pose with an onboard estimate suited to a farm where satellite positioning is unreliable, and re-run the navigation trials under the drift that results.
5. **Improve the forward channel.** The friction-based model is less accurate than the no-slip model in forward velocity. A formulation that retains the rotational accuracy while recovering the translational accuracy of the simpler model would remove the only respect in which the substitution costs anything.

CHAPTER 7 - REFERENCES

- [1] Statistics, Research, and Information Directorate, Ministry of Food and Agriculture, Agriculture in Ghana (Facts & Figures) 2024. Nov. 2025. [Online]. Available: [https://srid.mofa.gov.gh/sites/default/files/2025-12/AGRICULTURE%20IN%20GHANA%20\(Facts%20&%20Figures\)%202024.pdf](https://srid.mofa.gov.gh/sites/default/files/2025-12/AGRICULTURE%20IN%20GHANA%20(Facts%20&%20Figures)%202024.pdf)
- [2] A. E. Owusu, B. V. Fosua, A. W. Abubakar, S. K. Hagan, and A. Matthew, 'The Application of Artificial Intelligence in Transforming Agriculture in Ghana: A Systematic Review of Empirical Evidence, Impacts, and Scaling Constraints', *Journal of Global Agriculture and Ecology*, vol. 18, no. 1, pp. 79-94, Feb. 2026, doi: 10.56557/jogae/2026/v18i110223.
- [3] P. Dwumah, K. O. Akuoko, E. H. Yeboah, N. Gyasi-Boadu, and M. S. Boampong, 'Youth Employment Paths in a Ghanaian Rural Community', *E-Journal of Humanities, Arts and Social Sciences*, vol. 4, no. 3, pp. 196-211, Mar. 2023, doi: 10.38159/ehass.2023433.
- [4] S. Vercillo and B. Frayne, 'Urban-Rural Migration, Young People, and Agricultural Development in Northern Ghana', in *New Directions in South-South Migration*, J. Crush et al., Eds. Singapore: Springer, 2025, pp. 189-208, doi: 10.1007/978-981-97-9715-8_11.
- [5] E. Atabo, *Digitalization of Agriculture: Adoption of Digital Agri-Food Technologies Among Smallholder Farmers in Rural Ghana. A Case Study of Bono East Region*. The Hague, The Netherlands: International Institute of Social Studies, Dec. 2024.
- [6] C. Jenane, 'Is Artificial Intelligence the future of farming? Exploring opportunities and challenges in Sub-Saharan Africa'. Accessed: Sep. 16, 2026. [Online]. Available: <https://blogs.worldbank.org/en/agfood/artificial-interlligence-in-the-future-of-sub-saharan-africa-far>
- [7] N. Ozor, J. Nwakaire, A. Nyambane, W. Muhatiah, and C. Nwobodo, 'Enhancing Africa's agriculture and food systems through responsible and gender inclusive AI innovation: insights from AI4AFS network', *Frontiers in Artificial Intelligence*, vol. 7, Jan. 2025, doi: 10.3389/frai.2024.1472236.
- [8] 'Algorithmic Bias in Precision Agriculture Systems'. Accessed: Sep. 16, 2026. [Online]. Available: <https://prism.sustainability-directory.com/scenario/algorithmic-bias-in-precision-agriculture-systems-2/>

- [9] R. Chandran, 'Western AI models "fail spectacularly" in farms and forests abroad'. Accessed: Sep. 16, 2026. [Online]. Available: <https://restofworld.org/2026/ai-agriculture-local-data/>
- [10] S. Menon, 'Postcolonial Differentials in Algorithmic Bias: Challenging Digital Neo-Colonialism in Africa', *SCRIPTed*, vol. 20, no. 2, pp. 383-420, Aug. 2023, doi: 10.2966/scrip.200223.101.
- [11] A. Arora, M. Barrett, E. Lee, E. Oborn, and K. Prince, 'Risk and the future of AI: Algorithmic Bias, Data Colonialism, and Marginalization', *Information and Organization*, vol. 33, no. 3, 2023, doi: 10.1016/j.infoandorg.2023.100478.
- [12] E. Makena, 'The data drought: Why Kenya's agriculture revolution will fail without local AI training data'. Accessed: Sep. 16, 2026. [Online]. Available: <https://planterraafrica.org/2025/10/08/the-data-drought-why-kenyas-agriculture-revolution-will-fail-without-local-ai-training-data/>
- [13] P. K. Mensah et al., 'CCMT: Dataset for crop pest and disease detection', *Data in Brief*, vol. 49, Aug. 2023, doi: 10.1016/j.dib.2023.109306.
- [14] M. Fontani, S. M. Luglio, L. Gagliardi, A. Peruzzi, C. Frascioni, M. Raffaelli, and M. Fontanelli, 'A Systematic Review of 59 Field Robots for Agricultural Tasks: Applications, Trends, and Future Directions', *Agronomy*, vol. 15, no. 9, Sep. 2025, doi: 10.3390/agronomy15092185.
- [15] M. Spagnuolo, G. Todde, M. Caria, N. Furnitto, G. Schillaci, and S. Failla, 'Agricultural Robotics: A Technical Review Addressing Challenges in Sustainable Crop Production', *Robotics*, vol. 14, no. 2, Jan. 2025, doi: 10.3390/robotics14020009.
- [16] J. Yang, M. Kang, S. Lee, and S. Kim, 'Hybrid A*-Guided Model Predictive Path Integral Control for Robust Navigation in Rough Terrains', *Mathematics*, vol. 13, no. 5, Feb. 2025, doi: 10.3390/math13050810.
- [17] S. Rabiee and J. Biswas, 'A Friction-Based Kinematic Model for Skid-Steer Wheeled Mobile Robots', in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2019.
- [18] C. L. Nkwocha et al., 'A Comprehensive Review of Sensing, Control, and Networking in Agricultural Robots: From Perception to Coordination', *Robotics*, vol. 14, no. 11, Oct. 2025, doi: 10.3390/robotics14110159.

- [19] A. Atefi, Y. Ge, S. Pitla, and J. Schnable, 'Robotic Technologies for High-Throughput Plant Phenotyping: Contemporary Reviews and Future Perspectives', *Frontiers in Plant Science*, vol. 12, Jun. 2021, doi: 10.3389/fpls.2021.611940.
- [20] L. Yao, R. van de Zedde, and G. Kowalchuk, 'Recent developments and potential of robotics in plant eco-phenotyping', *Emerging Topics in Life Sciences*, vol. 5, pp. 289-300, 2021, doi: 10.1042/ETLS20200275.
- [21] T. Mueller-Sim, M. Jenkins, J. Abel, and G. Kantor, 'The Robotanist: A Ground-Based Agricultural Robot for High-Throughput Crop Phenotyping', in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 3634-3639, doi: 10.1109/ICRA.2017.7989418.
- [22] N. Wang, X. Li, K. Zhang, J. Wang, and D. Xie, 'A Survey on Path Planning for Autonomous Ground Vehicles in Unstructured Environments', *Machines*, vol. 12, no. 1, Jan. 2024, doi: 10.3390/machines12010031.
- [23] A. Dixit, D. D. Fan, K. Otsu, S. Dey, A.-A. Agha-Mohammadi, and J. W. Burdick, 'STEP: Stochastic Traversability Evaluation and Planning for Risk-Aware Navigation; Results From the DARPA Subterranean Challenge', *IEEE Transactions on Field Robotics*, vol. 2, pp. 81-99, Jan. 2025, doi: 10.1109/TFR.2024.3512433.
- [24] A. Leininger, M. Ali, H. Jardali, and L. Liu, 'Gaussian Process-based Traversability Analysis for Terrain Mapless Navigation', in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, May 2024, pp. 10925-10932, doi: 10.1109/ICRA57147.2024.10610106.
- [25] R. Khan, F. M. Malik, A. Raza, and N. Mazhar, 'Comprehensive study of skid-steer wheeled mobile robots: development and challenges', *Industrial Robot*, vol. 48, no. 1, pp. 142-156, Mar. 2021, doi: 10.1108/IR-04-2020-0082.
- [26] D. Mehta, S. Hulyalkar, K. Gupta, and S. Karmakar, 'RBR-1: Design and Development of a Multi-Purpose Autonomous Rover with Modular Arm, SLAM-Based Navigation, and Integrated Sensor Systems for Smart Agriculture', *International Journal of Advanced Research*, vol. 13, no. 10, pp. 187-196, Oct. 2025, doi: 10.21474/IJAR01/21903.
- [27] G. J. Q. Vasconcelos, G. S. R. Costa, T. V. Spina, and H. Pedrini, 'Low-Cost Robot for Agricultural Image Data Acquisition', *Agriculture*, vol. 13, no. 2, Feb. 2023, doi: 10.3390/agriculture13020413.

- [28] A. Manu, J. D. Osei, V. K. Avornyo, T. Lawler, and K. A. Frimpong, 'Harnessing Regenerative Agriculture, Unmanned Aerial Systems, and AI for Sustainable Cocoa Farming in West Africa', *Drones*, vol. 10, no. 1, Jan. 2026, doi: 10.3390/drones10010075.
- [29] D. K. Cudjoe et al., 'Field phenotyping for African crops: overview and perspectives', *Frontiers in Plant Science*, vol. 14, Oct. 2023, doi: 10.3389/fpls.2023.1219673.
- [30] RCILab, 'RCILab/RCI_hybrid_astar_guided_mppi'. Accessed: Sep. 16, 2026. [Online]. Available: https://github.com/RCILab/RCI_hybrid_astar_guided_mppi
- [31] F. Esser, R. A. Rosu, A. Cornelissen, L. Klingbeil, H. Kuhlmann, and S. Behnke, 'Field Robot for High-throughput and High-resolution 3D Plant Phenotyping', *IEEE Robotics & Automation Magazine*, vol. 30, no. 4, pp. 20-29, Dec. 2023, doi: 10.1109/MRA.2023.3321402.
- [32] ros2_control Development Team, 'ros2_control'. Accessed: Sep. 16, 2026. [Online]. Available: https://control.ros.org/jazzy/doc/ros2_control/doc/index.html
- [33] M. Arguedas et al., 'REP 2000 -- ROS 2 Releases and Target Platforms'. Accessed: Sep. 16, 2026. [Online]. Available: <https://www.ros.org/reps/rep-2000.html>
- [34] A. Mandow, J. L. Martinez, J. Morales, J. L. Blanco, A. J. Garcia-Cerezo, and J. Gonzalez, 'Experimental kinematics for wheeled skid-steer mobile robots', in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, San Diego, CA, USA, Oct. 2007, pp. 1222-1227, doi: 10.1109/IROS.2007.4399139.
- [35] T. Wang, Y. Wu, J. Liang, C. Han, J. Chen, and Q. Zhao, 'Analysis and Experimental Kinematics of a Skid-Steering Wheeled Robot Based on a Laser Scanner Sensor', *Sensors*, vol. 15, no. 5, pp. 9681-9702, Apr. 2015, doi: 10.3390/s150509681.
- [36] G. C. Anousaki and K. J. Kyriakopoulos, 'Simultaneous localization and map building of skid-steered robots', *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 79-89, Mar. 2007, doi: 10.1109/MRA.2007.339625.
- [37] J. Y. Wong and C. F. Chiang, 'A general theory for skid steering of tracked vehicles on firm ground', *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering*, vol. 215, no. 3, pp. 343-355, Mar. 2001, doi: 10.1243/0954407011525683.
- [38] M. A. Branch, T. F. Coleman, and Y. Li, 'A Subspace, Interior, and Conjugate Gradient Method for Large-Scale Bound-Constrained Minimization Problems', *SIAM*

Journal on Scientific Computing, vol. 21, no. 1, pp. 1-23, Jan. 1999, doi: 10.1137/S1064827595289108.

[39] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, 'Aggressive driving with model predictive path integral control', in Proc. IEEE International Conference on Robotics and Automation (ICRA), Stockholm, Sweden, May 2016, pp. 1433-1440, doi: 10.1109/ICRA.2016.7487277.

[40] K. Kanjanawaniskul, 'Motion control of a wheeled mobile robot using model predictive control: a survey', Asia-Pacific Journal of Science and Technology, vol. 17, no. 5, pp. 811-837, Apr. 2017. [Online]. Available: <https://so01.tci-thaijo.org/index.php/APST/article/view/83293>

[41] D. Fox, W. Burgard, and S. Thrun, 'The dynamic window approach to collision avoidance', IEEE Robotics & Automation Magazine, vol. 4, no. 1, pp. 23-33, Mar. 1997, doi: 10.1109/100.580977.

[42] R. C. Coulter, Implementation of the Pure Pursuit Path Tracking Algorithm. Pittsburgh, PA, USA: Carnegie Mellon University, Jan. 1992. [Online]. Available: <https://publications.ri.cmu.edu/implementation-of-the-pure-pursuit-path-tracking-algorithm>

[43] P. Fankhauser, M. Bloesch, and M. Hutter, 'Probabilistic terrain mapping for mobile robots with uncertain localization', IEEE Robotics and Automation Letters, vol. 3, no. 4, pp. 3019-3026, Oct. 2018, doi: 10.1109/LRA.2018.2849506.

[44] P. E. Hart, N. J. Nilsson, and B. Raphael, 'A formal basis for the heuristic determination of minimum cost paths', IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, Jul. 1968, doi: 10.1109/TSSC.1968.300136.

[45] D. Dolgov, S. Thrun, M. Montemerlo, and J. Diebel, 'Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments', The International Journal of Robotics Research, vol. 29, no. 5, pp. 485-501, Apr. 2010, doi: 10.1177/0278364909359210.

[46] F. Wilcoxon, 'Individual Comparisons by Ranking Methods', Biometrics Bulletin, vol. 1, no. 6, pp. 80-83, Dec. 1945, doi: 10.2307/3001968.

[47] D. J. Schuirmann, 'A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability', Journal of

Pharmacokinetics and Biopharmaceutics, vol. 15, no. 6, pp. 657-680, Dec. 1987, doi:
10.1007/BF01068419.